

Feb 2026

Contents

Contents	i
1 Changes	1
1.1 .github/ISSUE_TEMPLATE/Bump.yml	1
1.2 PhysLean.lean	1
1.3 PhysLean/ClassicalMechanics/DampedHarmonicOscillator/Basic.lean	1
1.4 PhysLean/ClassicalMechanics/EulerLagrange.lean	2
1.5 PhysLean/ClassicalMechanics/HarmonicOscillator/Basic.lean	2
1.6 PhysLean/ClassicalMechanics/HarmonicOscillator/ConfigurationSpace.lean	6
1.7 PhysLean/ClassicalMechanics/HarmonicOscillator/Solution.lean	12
1.8 PhysLean/ClassicalMechanics/Mass/MassUnit.lean	14
1.9 PhysLean/ClassicalMechanics/RigidBody/SolidSphere.lean	14
1.10 PhysLean/CondensedMatter/TightBindingChain/Basic.lean	14
1.11 PhysLean/Electromagnetism/Charge/ChargeUnit.lean	14
1.12 PhysLean/Electromagnetism/Dynamics/IsExtrema.lean	14
1.13 PhysLean/Electromagnetism/Dynamics/KineticTerm.lean	14
1.14 PhysLean/Electromagnetism/Kinematics/FieldStrength.lean	15
1.15 PhysLean/Mathematics/Calculus/Divergence.lean	15
1.16 PhysLean/Mathematics/DataStructures/Matrix/LieTrace.lean	15
1.17 PhysLean/Mathematics/Distribution/Basic.lean	15
1.18 PhysLean/Mathematics/Fin.lean	16
1.19 PhysLean/Mathematics/Fin/Involutions.lean	16
1.20 PhysLean/Mathematics/Geometry/Metric/PseudoRiemannian/Defs.lean	16
1.21 PhysLean/Mathematics/InnerProductSpace/Submodule.lean	17
1.22 PhysLean/Mathematics/KroneckerDelta.lean	20
1.23 PhysLean/Mathematics/List.lean	20
1.24 PhysLean/Mathematics/List/InsertionSort.lean	21
1.25 PhysLean/Mathematics/S03/Basic.lean	22
1.26 PhysLean/Mathematics/Trigonometry/Tanh.lean	22
1.27 PhysLean/Mathematics/VariationalCalculus/HasVarAdjDeriv.lean	22
1.28 PhysLean/Mathematics/VariationalCalculus/IsTestFunction.lean	23
1.29 PhysLean/Meta/Basic.lean	23
1.30 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/Plane	
1.31 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/PlusU1/QuadS	

1.32	PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/Plu	
1.33	PhysLean/Particles/BeyondTheStandardModel/TwoHDM/GramMatrix.lean	24
1.34	PhysLean/Particles/BeyondTheStandardModel/TwoHDM/Potential.lean	24
1.35	PhysLean/Particles/FlavorPhysics/CKMMMatrix/PhaseFreedom.lean	39
1.36	PhysLean/Particles/FlavorPhysics/CKMMMatrix/Relations.lean	39
1.37	PhysLean/Particles/FlavorPhysics/CKMMMatrix/StandardParameterization/S	
1.38	PhysLean/Particles/StandardModel/AnomalyCancellation/Basic.lean	40
1.39	PhysLean/Particles/StandardModel/AnomalyCancellation/NoGrav/One/Lemma	
1.40	PhysLean/Particles/StandardModel/HiggsBoson/Basic.lean	40
1.41	PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/B3.lean	4
1.42	PhysLean/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/Y3.lean	4
1.43	PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/Basic.lean	40
1.44	PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/Completions.lean	
1.45	PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/MinimalSuperSet.1	
1.46	PhysLean/QFT/AnomalyCancellation/Basic.lean	.. 41
1.47	PhysLean/QFT/PerturbationTheory/CreateAnnihilate.lean	41
1.48	PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/SuperCommute.lean	
1.49	PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/TimeOrder.lean	41
1.50	PhysLean/QFT/PerturbationTheory/FieldSpecification/NormalOrder.lean	4
1.51	PhysLean/QFT/PerturbationTheory/FieldSpecification/TimeOrder.lean	41
1.52	PhysLean/QFT/PerturbationTheory/FieldStatistics/Basic.lean	42
1.53	PhysLean/QFT/PerturbationTheory/FieldStatistics/OfFinset.lean	42
1.54	PhysLean/QFT/PerturbationTheory/Koszul/KoszulSign.lean	42
1.55	PhysLean/QFT/PerturbationTheory/Koszul/KoszulSignInsert.lean	42
1.56	PhysLean/QFT/PerturbationTheory/WickAlgebra/Basic.lean	42
1.57	PhysLean/QFT/PerturbationTheory/WickAlgebra/NormalOrder/Basic.lean	43
1.58	PhysLean/QFT/PerturbationTheory/WickAlgebra/NormalOrder/Lemmas.lean	4
1.59	PhysLean/QFT/PerturbationTheory/WickAlgebra/NormalOrder/WickContracti	
1.60	PhysLean/QFT/PerturbationTheory/WickAlgebra/StaticWickTerm.lean	43
1.61	PhysLean/QFT/PerturbationTheory/WickAlgebra/SuperCommute.lean	43
1.62	PhysLean/QFT/PerturbationTheory/WickAlgebra/TimeContraction.lean	43
1.63	PhysLean/QFT/PerturbationTheory/WickAlgebra/TimeOrder.lean	44
1.64	PhysLean/QFT/PerturbationTheory/WickAlgebra/WickTerm.lean	44
1.65	PhysLean/QFT/PerturbationTheory/WickContraction/Basic.lean	44
1.66	PhysLean/QFT/PerturbationTheory/WickContraction/ExtractEquiv.lean	44
1.67	PhysLean/QFT/PerturbationTheory/WickContraction/InsertAndContract.lean	
1.68	PhysLean/QFT/PerturbationTheory/WickContraction/InsertAndContractNat.	
1.69	PhysLean/QFT/PerturbationTheory/WickContraction/Join.lean	45
1.70	PhysLean/QFT/PerturbationTheory/WickContraction/Sign/InsertNone.lean	
1.71	PhysLean/QFT/PerturbationTheory/WickContraction/Sign/InsertSome.lean	
1.72	PhysLean/QFT/PerturbationTheory/WickContraction/Sign/Join.lean	46
1.73	PhysLean/QFT/PerturbationTheory/WickContraction/Singleton.lean	46
1.74	PhysLean/QFT/PerturbationTheory/WickContraction/StaticContract.lean	4
1.75	PhysLean/QFT/PerturbationTheory/WickContraction/TimeCond.lean	46
1.76	PhysLean/QFT/PerturbationTheory/WickContraction/TimeContract.lean	46
1.77	PhysLean/QFT/PerturbationTheory/WickContraction/UncontractedList.lean	

1.78	PhysLean/QFT/QED/AnomalyCancellation/Basic.lean	47
1.79	PhysLean/QFT/QED/AnomalyCancellation/BasisLinear.lean	47
1.80	PhysLean/QFT/QED/AnomalyCancellation/ConstAbs.lean	47
1.81	PhysLean/QFT/QED/AnomalyCancellation/Even/BasisLinear.lean	47
1.82	PhysLean/QFT/QED/AnomalyCancellation/Even/LineInCubic.lean	48
1.83	PhysLean/QFT/QED/AnomalyCancellation/LineInPlaneCond.lean	48
1.84	PhysLean/QFT/QED/AnomalyCancellation/Odd/BasisLinear.lean	48
1.85	PhysLean/QuantumMechanics/DDimensions/Hydrogen/Basic.lean	48
1.86	PhysLean/QuantumMechanics/DDimensions/Hydrogen/LaplaceRungeLenzVector.lean	50
1.87	PhysLean/QuantumMechanics/DDimensions/Operators/AngularMomentum.lean	69
1.88	PhysLean/QuantumMechanics/DDimensions/Operators/Commutation.lean	72
1.89	PhysLean/QuantumMechanics/DDimensions/Operators/Momentum.lean	80
1.90	PhysLean/QuantumMechanics/DDimensions/Operators/Position.lean	81
1.91	PhysLean/QuantumMechanics/DDimensions/Operators/Unbounded.lean	84
1.92	PhysLean/QuantumMechanics/DDimensions/SpaceDHilbertSpace/Basic.lean	89
1.93	PhysLean/QuantumMechanics/DDimensions/SpaceDHilbertSpace/SchwartzSubmodule.lean	
1.94	PhysLean/QuantumMechanics/FiniteTarget/Basic.lean	94
1.95	PhysLean/QuantumMechanics/OneDimension/HilbertSpace/PlaneWaves.lean	94
1.96	PhysLean/QuantumMechanics/OneDimension/HilbertSpace/PositionStates.lean	94
1.97	PhysLean/QuantumMechanics/OneDimension/HilbertSpace/SchwartzSubmodule.lean	94
1.98	PhysLean/QuantumMechanics/OneDimension/Operators/Momentum.lean	94
1.99	PhysLean/QuantumMechanics/OneDimension/Operators/Parity.lean	95
1.100	PhysLean/Relativity/LorentzAlgebra/ExponentialMap.lean	95
1.101	PhysLean/Relativity/LorentzGroup/Boosts/Generalized.lean	95
1.102	PhysLean/Relativity/Tensors/Color/Lift.lean	96
1.103	PhysLean/Relativity/Tensors/ComplexTensor/Metrics/Basic.lean	96
1.104	PhysLean/Relativity/Tensors/ComplexTensor/Metrics/Lemmas.lean	96
1.105	PhysLean/Relativity/Tensors/ComplexTensor/OfRat.lean	97
1.106	PhysLean/Relativity/Tensors/ComplexTensor/Units/Basic.lean	97
1.107	PhysLean/Relativity/Tensors/ComplexTensor/Weyl/Two.lean	97
1.108	PhysLean/Relativity/Tensors/Contraction/Basis.lean	98
1.109	PhysLean/Relativity/Tensors/Contraction/Products.lean	98
1.110	PhysLean/Relativity/Tensors/Contraction/Pure.lean	98
1.111	PhysLean/Relativity/Tensors/Evaluation.lean	98
1.112	PhysLean/Relativity/Tensors/Product.lean	98
1.113	PhysLean/Relativity/Tensors/RealTensor/Basic.lean	99
1.114	PhysLean/Relativity/Tensors/RealTensor/Metrics/Basic.lean	99
1.115	PhysLean/Relativity/Tensors/RealTensor/ToComplex.lean	99
1.116	PhysLean/Relativity/Tensors/RealTensor/Vector/Basic.lean	112
1.117	PhysLean/Relativity/Tensors/RealTensor/VelocityEngine.lean	112
1.118	PhysLean/Relativity/Tensors/TensorSpecies/Basic.lean	112
1.119	PhysLean/SpaceAndTime/Space/Basic.lean	112
1.120	PhysLean/SpaceAndTime/Space/ConstantSliceDist.lean	112
1.121	PhysLean/SpaceAndTime/Space/Derivatives/Basic.lean	112
1.122	PhysLean/SpaceAndTime/Space/Derivatives/Curl.lean	113
1.123	PhysLean/SpaceAndTime/Space/Derivatives/Div.lean	113

1.124	PhysLean/SpaceAndTime/Space/Derivatives/Grad.lean	114
1.125	PhysLean/SpaceAndTime/Space/LengthUnit.lean	114
1.126	PhysLean/SpaceAndTime/Space/Norm.lean	114
1.127	PhysLean/SpaceAndTime/Space/Translations.lean	114
1.128	PhysLean/SpaceAndTime/SpaceTime/Derivatives.lean	115
1.129	PhysLean/SpaceAndTime/Time/TimeTransMan.lean	115
1.130	PhysLean/SpaceAndTime/Time/TimeUnit.lean	115
1.131	PhysLean/SpaceAndTime/TimeAndSpace/Basic.lean	115
1.132	PhysLean/SpaceAndTime/TimeAndSpace/ConstantTimeDist.lean	115
1.133	PhysLean/StatisticalMechanics/BoltzmannConstant.lean	116
1.134	PhysLean/StatisticalMechanics/CanonicalEnsemble/Finite.lean	116
1.135	PhysLean/StatisticalMechanics/CanonicalEnsemble/Lemmas.lean	116
1.136	PhysLean/Thermodynamics/Temperature/Basic.lean	116
1.137	PhysLean/Thermodynamics/Temperature/TemperatureUnits.lean	116
1.138	PhysLean/Units/Basic.lean	117
1.139	PhysLean/Units/Examples.lean	117
1.140	README.md	117
1.141	docs/MaintainerTeam.md	117
1.142	docs/_data/perturbationTheory.yml	118
1.143	lake-manifest.json	118
1.144	lakefile.toml	118
1.145	lean-toolchain	118

1 Changes

1.1 .github/ISSUE_TEMPLATE/Bump.yml

```
- label: Remove the `.lake` file, e.g. on unix run `rm -rf .lake`.
- label: Run `lake update`.
- label: Put a bump notice on this [thread](https://leanprover.zulipchat.com/#PhysLean/topic/Bumps.20of.20PhysLean.2E/with/572707099).
  label: Example past bumps
    [v4.20.0](https://github.com/lean-phys-community/PhysLean/pull/591),
    [v4.20.0-rc5](https://github.com/lean-phys-community/PhysLean/pull/566)
    [v4.27.0](https://github.com/lean-phys-community/PhysLean/pull/929)
```

1.2 PhysLean.lean

```
import PhysLean.ClassicalMechanics.HarmonicOscillator.ConfigurationSpace
import PhysLean.Mathematics.InnerProductSpace.Submodule
import PhysLean.Mathematics.KroneckerDelta
import PhysLean.QuantumMechanics.DDimensions.Hydrogen.Basic
import PhysLean.QuantumMechanics.DDimensions.Hydrogen.LaplaceRungeLenzVector
import PhysLean.QuantumMechanics.DDimensions.Operators.AngularMomentum
import PhysLean.QuantumMechanics.DDimensions.Operators.Commutation
import PhysLean.QuantumMechanics.DDimensions.Operators.Momentum
import PhysLean.QuantumMechanics.DDimensions.Operators.Position
import PhysLean.QuantumMechanics.DDimensions.Operators.Unbounded
import PhysLean.QuantumMechanics.DDimensions.SpaceDHilbertSpace.Basic
import PhysLean.QuantumMechanics.DDimensions.SpaceDHilbertSpace.SchwartzSubmodule
```

1.3 PhysLean/ClassicalMechanics/DampedHarmonicOscillator/Basic.lean

```
lemma k_ne_zero : S.k ≠ 0 := Ne.symm (ne_of_lt S.k_pos)
lemma m_ne_zero : S.m ≠ 0 := Ne.symm (ne_of_lt S.m_pos)
```

1.4 PhysLean/ClassicalMechanics/EulerLagrange.lean

```

apply ContDiff.differentiable (n := ∞) (by fun_prop) (by simp)
apply ContDiff.differentiable (n := ∞) (by fun_prop) (by simp)
apply ContDiff.differentiable (n := ∞) (by fun_prop) (by simp)
apply ContDiff.differentiable (n := ∞) (by fun_prop) (by simp)

```

1.5 PhysLean/ClassicalMechanics/HarmonicOscillator/Basic.lean

```

import PhysLean.ClassicalMechanics.HarmonicOscillator.ConfigurationSpace
TODO "6VZHC" "Create a new folder for the damped harmonic oscillator, initial
holder."

```

```

lemma k_ne_zero : S.k ≠ 0 := Ne.symm (ne_of_lt S.k_pos)
lemma m_ne_zero : S.m ≠ 0 := Ne.symm (ne_of_lt S.m_pos)
lemma ω_ne_zero : S.ω ≠ 0 := Ne.symm (ne_of_lt S.ω_pos)
noncomputable def kineticEnergy (xt : Time → ConfigurationSpace) : Time → ℝ
noncomputable def potentialEnergy (x : ConfigurationSpace) : ℝ :=
noncomputable def energy (xt : Time → ConfigurationSpace) : Time → ℝ := fun
lemma kineticEnergy_eq (xt : Time → ConfigurationSpace) :
lemma potentialEnergy_eq (x : ConfigurationSpace) :
lemma energy_eq (xt : Time → ConfigurationSpace) :
lemma kineticEnergy_differentiable (xt : Time → ConfigurationSpace) (hx : C
lemma potentialEnergy_differentiable (xt : Time → ConfigurationSpace) (hx :
lemma energy_differentiable (xt : Time → ConfigurationSpace) (hx : ContDiff
lemma kineticEnergy_deriv (xt : Time → ConfigurationSpace) (hx : ContDiff ℝ
lemma potentialEnergy_deriv (xt : Time → ConfigurationSpace) (hx : ContDiff
lemma energy_deriv (xt : Time → ConfigurationSpace) (hx : ContDiff ℝ ∞ xt)
noncomputable def lagrangian (t : Time) (x : ConfigurationSpace)
  (v : ConfigurationSpace) : ℝ :=
  (xt : Time → ConfigurationSpace) :
  apply ContDiff.sub
  · apply ContDiff.mul
    · apply ContDiff.mul
      · exact contDiff_const
      · exact contDiff_const
    · exact ContDiff.inner (□ := ℝ) (contDiff_snd.comp contDiff_snd)
      (contDiff_snd.comp contDiff_snd)
  · apply ContDiff.mul
    · apply ContDiff.mul
      · exact contDiff_const
      · exact contDiff_const
    · exact ContDiff.inner (□ := ℝ) (contDiff_fst.comp contDiff_snd)
      (contDiff_fst.comp contDiff_snd)

```


1.5.

PHYSLEAN/CLASSICALMECHANICS/HARMONICOSCILLATOR/BASIC.LEAN

```

lemma toDual_symm_innerSL (x : ConfigurationSpace) :
  (InnerProductSpace.toDual ℝ ConfigurationSpace).symm (innerSL ℝ x) = x := by
  apply (innerSL_inj (λ := ℝ) (E := ConfigurationSpace)).1
  ext y
  simp [InnerProductSpace.toDual_symm_apply]

lemma gradient_inner_self (x : ConfigurationSpace) :
  gradient (fun y : ConfigurationSpace => ⟨y, y⟩_ℝ) x = (2 : ℝ) • x := by
  refine ext_inner_right (λ := ℝ) fun y => ?_
  unfold gradient
  rw [InnerProductSpace.toDual_symm_apply]
  have hid : DifferentiableAt ℝ (fun y : ConfigurationSpace => y) x := differentiableAt_id
  rw [show (fun y : ConfigurationSpace => ⟨y, y⟩_ℝ) =
    fun y => ⟨(fun y => y) y, (fun y => y) y⟩_ℝ from rfl]
  rw [fderiv_inner_apply (λ := ℝ) hid hid]
  simp only [fderiv_id', ContinuousLinearMap.coe_id', id_eq,
    ConfigurationSpace.inner_def, ConfigurationSpace.smul_val]
  ring

lemma gradient_const_mul_inner_self (c : ℝ) (x : ConfigurationSpace) :
  gradient (fun y : ConfigurationSpace => c * ⟨y, y⟩_ℝ) x = (2 * c) • x := by
  calc
    gradient (fun y : ConfigurationSpace => c * ⟨y, y⟩_ℝ) x
      = (InnerProductSpace.toDual ℝ ConfigurationSpace).symm
        (fderiv ℝ (fun y : ConfigurationSpace => c * ⟨y, y⟩_ℝ) x) := rfl
    _ = (InnerProductSpace.toDual ℝ ConfigurationSpace).symm
        (c • fderiv ℝ (fun y : ConfigurationSpace => ⟨y, y⟩_ℝ) x) := by
      rw [fderiv_const_mul (ConfigurationSpace.differentiableAt_inner_self x)]
    _ = c • gradient (fun y : ConfigurationSpace => ⟨y, y⟩_ℝ) x := by
      simp only [gradient, map_smul]
    _ = c • ((2 : ℝ) • x) := by
      rw [gradient_inner_self]
    _ = (2 * c) • x := by
      rw [smul_smul, mul_comm]

private lemma gradient_add_const' {f : ConfigurationSpace → ℝ} {c : ℝ}
  (x : ConfigurationSpace) :
  gradient (fun y => f y + c) x = gradient f x := by
  unfold gradient
  rw [fderiv_add_const]

lemma gradient_lagrangian_position_eq (t : Time) (x : ConfigurationSpace)
  (v : ConfigurationSpace) :
  gradient (fun x => lagrangian S t x v) x = - S.k • x := by
  have h_eq : (fun y : ConfigurationSpace => lagrangian S t y v) =
    fun y => (-(1 / (2 : ℝ)) * S.k) * ⟨y, y⟩_ℝ + (1 / (2 : ℝ) * S.m * ⟨v, v⟩_ℝ) :=
    funext y; unfold lagrangian potentialEnergy; simp only [smul_eq_mul]; ring

```

```

rw [h_eq, gradient_add_const', gradient_const_mul_inner_self]
ext; simp only [ConfigurationSpace.smul_val]; ring

lemma gradient_lagrangian_velocity_eq (t : Time) (x : ConfigurationSpace)
  (v : ConfigurationSpace) :
  have h_eq : (fun y : ConfigurationSpace => lagrangian S t x y) =
    fun y => ((1 / (2 : ℝ)) * S.m) * ∂y, y|_ℝ + (-(1 / (2 : ℝ)) * S.k * ∂
    funext y; unfold lagrangian_potentialEnergy; simp only [smul_eq_mul]; r
    change gradient (fun y : ConfigurationSpace => lagrangian S t x y) v = S.
    rw [h_eq, gradient_add_const', gradient_const_mul_inner_self]
    ext; simp only [ConfigurationSpace.smul_val]; ring
noncomputable def gradLagrangian (x_t : Time → ConfigurationSpace) :
  Time → ConfigurationSpace :=
lemma gradLagrangian_eq_eulerLagrangeOp (x_t : Time → ConfigurationSpace)
def EquationOfMotion (x_t : Time → ConfigurationSpace) : Prop :=
lemma equationOfMotion_iff_gradLagrangian_zero (x_t : Time → ConfigurationSp
noncomputable def force (S : HarmonicOscillator) (x : ConfigurationSpace) :
  ConfigurationSpace :=
lemma force_eq_linear (x : ConfigurationSpace) : force S x = - S.k • x := b
  unfold force_potentialEnergy
  have hpot : (fun y : ConfigurationSpace => (1 / (2 : ℝ)) • S.k • ∂y, y|_ℝ
    fun y => ((1 / (2 : ℝ)) * S.k) * ∂y, y|_ℝ := by
    funext y
    simp [smul_eq_mul, mul_assoc]
  rw [hpot]
  have hgrad : gradient (fun y : ConfigurationSpace => ((1 / (2 : ℝ)) * S.k
    = S.k • x := by
    simp [smul_eq_mul, mul_assoc] using
      (gradient_const_mul_inner_self (c := (1 / (2 : ℝ)) * S.k) x)
  rw [hgrad]
  simp [neg_smul]
lemma gradLagrangian_eq_force (x_t : Time → ConfigurationSpace) (hx : ContDi
  · simp [gradient_lagrangian_position_eq, force_eq_linear]
  · conv_lhs =>
    arg 1
    ext t'
    rw [gradient_lagrangian_velocity_eq]
  show ∂_t (fun t' => S.m • ∂_t x_t t') t = S.m • ∂_t (∂_t x_t) t
  have hd : DifferentiableAt ℝ (∂_t x_t) t :=
    (deriv_differentiable_of_contDiff x_t hx).differentiableAt
  calc
    ∂_t (fun t' => S.m • ∂_t x_t t') t
      = fderiv ℝ (fun t' => S.m • ∂_t x_t t') t 1 := rfl
    _ = S.m • (fderiv ℝ (∂_t x_t) t 1) := by
      simp using congrArg (fun L => L 1) (fderiv_const_smul (c := S.m)
    _ = S.m • ∂_t (∂_t x_t) t := rfl

```

1.5.

PHYSLEAN/CLASSICALMECHANICS/HARMONICOSCILLATOR/BASIC.LEAN

```

lemma equationOfMotion_iff_newtons_2nd_law (x_t : Time → ConfigurationSpace)
lemma energy_conservation_of_equationOfMotion (x_t : Time → ConfigurationSpace)
lemma energy_conservation_of_equationOfMotion' (x_t : Time → ConfigurationSpace)
noncomputable def toCanonicalMomentum (t : Time) (x : ConfigurationSpace) :
  ConfigurationSpace ≈1[ℝ] ConfigurationSpace where
lemma toCanonicalMomentum_eq (t : Time) (x : ConfigurationSpace)
  (v : ConfigurationSpace) :
noncomputable def hamiltonian (t : Time) (p : ConfigurationSpace)
  (x : ConfigurationSpace) : ℝ :=
  have hm : S.m ≠ 0 := by exact m_ne_zero S
  apply ContDiff.add
  · apply ContDiff.mul
    · apply ContDiff.mul
      · exact contDiff_const
      · exact contDiff_const
    · exact ContDiff.inner (□ := ℝ) (contDiff_fst.comp contDiff_snd)
      (contDiff_fst.comp contDiff_snd)
  · apply ContDiff.mul
    · apply ContDiff.mul
      · exact contDiff_const
      · exact contDiff_const
    · exact ContDiff.inner (□ := ℝ) (contDiff_snd.comp contDiff_snd)
      (contDiff_snd.comp contDiff_snd)
lemma gradient_hamiltonian_position_eq (t : Time) (x : ConfigurationSpace)
  (p : ConfigurationSpace) :
  have h_eq : (fun y : ConfigurationSpace => hamiltonian S t p y) =
    fun y => ((1 / (2 : ℝ)) * S.k) * □y, y□_ℝ +
      ((1 / (2 : ℝ)) * (1 / S.m) * □p, p□_ℝ) := by
    funext y; unfold hamiltonian; simp only [toCanonicalMomentum, lagrangian,
      potentialEnergy, LinearEquiv.coe_symm_mk', ConfigurationSpace.inner_def,
      ConfigurationSpace.smul_val, smul_eq_mul]
  have hm : S.m ≠ 0 := m_ne_zero S
  field_simp
  ring
  change gradient (fun y : ConfigurationSpace => hamiltonian S t p y) x = S.k • x
  rw [h_eq, gradient_add_const', gradient_const_mul_inner_self]
  ext; simp only [ConfigurationSpace.smul_val]; ring
lemma gradient_hamiltonian_momentum_eq (t : Time) (x : ConfigurationSpace)
  (p : ConfigurationSpace) :
  have h_eq : (fun y : ConfigurationSpace => hamiltonian S t y x) =
    fun y => ((1 / (2 : ℝ)) * (1 / S.m)) * □y, y□_ℝ +
      ((1 / (2 : ℝ)) * S.k * □x, x□_ℝ) := by
    funext y; unfold hamiltonian; simp only [toCanonicalMomentum, lagrangian,
      potentialEnergy, LinearEquiv.coe_symm_mk', ConfigurationSpace.inner_def,
      ConfigurationSpace.smul_val, smul_eq_mul]
  have hm : S.m ≠ 0 := m_ne_zero S

```

```

    field_simp
    ring
    change gradient (fun y : ConfigurationSpace => hamiltonian S t y x) p = (
    rw [h_eq, gradient_add_const', gradient_const_mul_inner_self]
    ext; simp only [ConfigurationSpace.smul_val]; ring
lemma hamiltonian_eq_energy (x_t : Time → ConfigurationSpace) :
  have hsymm :
    (toCanonicalMomentum S t (x_t t)).symm (S.m • ∂_t x_t t) = ∂_t x_t t := by
    rw [← toCanonicalMomentum_eq (S := S) (t := t) (x := x_t t) (v := ∂_t x_t t)
    exact LinearEquiv.symm_apply_apply (toCanonicalMomentum S t (x_t t)) (∂_t
    unfold hamiltonian lagrangian energy kineticEnergy potentialEnergy
    simp only [toCanonicalMomentum_eq,
    ConfigurationSpace.inner_def, ConfigurationSpace.smul_val, one_div, smul
    rw [hsymm]
    ring_nf
noncomputable def hamiltonEqOp (p : Time → ConfigurationSpace)
  (q : Time → ConfigurationSpace) :=
lemma equationOfMotion_iff_hamiltonEqOp_eq_zero (x_t : Time → ConfigurationSpace)
  have hderiv_smul : ∀ t, ∂_t (fun t' => S.m • ∂_t x_t t') t = S.m • ∂_t (∂_t x_t t)
  intro t
  have hd : DifferentiableAt ℝ (∂_t x_t) t :=
    (deriv_differentiable_of_contDiff x_t hx).differentiableAt
  calc
    ∂_t (fun t' => S.m • ∂_t x_t t') t
      = fderiv ℝ (fun t' => S.m • ∂_t x_t t') t 1 := rfl
    _ = S.m • (fderiv ℝ (∂_t x_t) t 1) := by
      simpa using congrArg (fun L => L 1) (fderiv_const_smul (c := S.m)
    _ = S.m • ∂_t (∂_t x_t) t := rfl
  simp [hderiv_smul, force_eq_linear]
lemma equationOfMotion_tfae (x_t : Time → ConfigurationSpace) (hx : ContDiff

```

1.6 PhysLean/ClassicalMechanics/HarmonicOscillator/ConfigurationSpace

```

/-
Copyright (c) 2026 Nicola Bernini. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Nicola Bernini
-/
import Mathlib.Geometry.Manifold.Diffeomorph
import PhysLean.SpaceAndTime.Time.Basic
/-!
# Configuration space of the harmonic oscillator

```

The configuration space is defined as a one-dimensional smooth manifold,

1.6.

PHYSLEAN/CLASSICALMECHANICS/HARMONICOSCILLATOR/CONFIGURATIONSPACE.LEAN

modeled on $\mathbb{R}$, with a chosen coordinate.

-/

namespace ClassicalMechanics

namespace HarmonicOscillator

/-- The configuration space of the harmonic oscillator. -

/

structure ConfigurationSpace where

/-- The underlying real coordinate. -/

val : $\mathbb{R}$

namespace ConfigurationSpace

@[ext]

lemma ext {x y : ConfigurationSpace} (h : x.val = y.val) : x = y := by

cases x

cases y

simp at h

simp [h]

/-!

Algebraic and analytical structure

-/

instance : Zero ConfigurationSpace := { zero := ⟨0⟩ }

instance : OfNat ConfigurationSpace 0 := { ofNat := ⟨0⟩ }

@[simp]

lemma zero_val : (0 : ConfigurationSpace).val = 0 := rfl

instance : Add ConfigurationSpace where

add x y := ⟨x.val + y.val⟩

@[simp]

lemma add_val (x y : ConfigurationSpace) : (x + y).val = x.val + y.val := rfl

instance : Neg ConfigurationSpace where

neg x := ⟨-x.val⟩

@[simp]

lemma neg_val (x : ConfigurationSpace) : (-x).val = -x.val := rfl

instance : Sub ConfigurationSpace where

```

    sub x y := ⟨x.val - y.val⟩

@[simp]
lemma sub_val (x y : ConfigurationSpace) : (x - y).val = x.val - y.val := r

instance : SMul ℝ ConfigurationSpace where
  smul r x := ⟨r * x.val⟩

@[simp]
lemma smul_val (r : ℝ) (x : ConfigurationSpace) : (r • x).val = r * x.val :

instance : CoeFun ConfigurationSpace (fun _ => Fin 1 → ℝ) where
  coe x := fun _ => x.val

@[simp]
lemma apply_zero (x : ConfigurationSpace) : x 0 = x.val := rfl

@[simp]
lemma apply_eq_val (x : ConfigurationSpace) (i : Fin 1) : x i = x.val := rfl

instance : AddGroup ConfigurationSpace where
  add_assoc x y z := by ext; simp [add_assoc]
  zero_add x := by ext; simp [zero_add]
  add_zero x := by ext; simp [add_zero]
  neg_add_cancel x := by ext; simp [neg_add_cancel]
  nsmul := nsmulRec
  zsmul := zsmulRec

instance : AddCommGroup ConfigurationSpace where
  add_comm x y := by ext; simp [add_comm]

instance : Module ℝ ConfigurationSpace where
  one_smul x := by ext; simp
  smul_add r x y := by ext; simp [mul_add]
  smul_zero r := by ext; simp [mul_zero]
  add_smul r s x := by ext; simp [add_mul]
  mul_smul r s x := by ext; simp [mul_assoc]
  zero_smul x := by ext; simp

instance : Norm ConfigurationSpace where
  norm x := ||x.val||

instance : Dist ConfigurationSpace where
  dist x y := ||x - y||

lemma dist_eq_val (x y : ConfigurationSpace) :

```

1.6.

PHYSLEAN/CLASSICALMECHANICS/HARMONICOSCILLATOR/CONFIGURATIONSPACE.LEAN

```
dist x y = ||x.val - y.val|| := rfl

instance : SeminormedAddCommGroup ConfigurationSpace where
  dist_self x := by simp [dist_eq_val]
  dist_comm x y := by
    simp [dist_eq_val, Real.dist_eq] using (dist_comm x.val y.val)
  dist_triangle x y z := by
    simp [dist_eq_val, Real.dist_eq] using (dist_triangle x.val y.val z.val)

instance : NormedAddCommGroup ConfigurationSpace where
  eq_of_dist_eq_zero := by
    intro a b h
    ext
    have h' : dist a.val b.val = 0 := by
      simp [dist_eq_val, Real.dist_eq] using h
    exact dist_eq_zero.mp h'

instance : NormedSpace ℝ ConfigurationSpace where
  norm_smul_le r x := by
    simp [norm, smul_val, abs_mul]

open InnerProductSpace

instance : Inner ℝ ConfigurationSpace where
  inner x y := x.val * y.val

@[simp]
lemma inner_def (x y : ConfigurationSpace) :  $\langle x, y \rangle_{\mathbb{R}} = x.val * y.val$  := rfl

noncomputable instance : InnerProductSpace ℝ ConfigurationSpace where
  norm_sq_eq_re_inner := by
    intro x
    have hx : ||x|| ^ 2 = x.val ^ 2 := by
      simp [norm, sq_abs]
    simp [inner_def, pow_two] using hx
  conj_inner_symm := by
    intro x y
    simp [inner_def]
    ring
  add_left := by
    intro x y z
    simp [inner_def, add_mul]
  smul_left := by
    intro x y r
    simp [inner_def]
    ring
```

```

@[fun_prop]
lemma differentiable_inner_self :
  Differentiable ℝ (fun x : ConfigurationSpace => [x, x[ℝ]) := by
  have h_id : Differentiable ℝ (fun x : ConfigurationSpace => x) := differe
  simp using (Differentiable.inner ([:=ℝ]) (f:=fun x : ConfigurationSpace =>
    (g:=fun x : ConfigurationSpace => x) h_id h_id))

@[fun_prop]
lemma differentiableAt_inner_self (x : ConfigurationSpace) :
  DifferentiableAt ℝ (fun y : ConfigurationSpace => [y, y[ℝ]) x := by
  have h_id : DifferentiableAt ℝ (fun y : ConfigurationSpace => y) x := dif
  simp using (DifferentiableAt.inner ([:=ℝ]) (f:=fun y : ConfigurationSpace
    (g:=fun y : ConfigurationSpace => y) h_id h_id))

@[fun_prop]
lemma contDiff_inner_self (n : WithTop ℕ∞) :
  ContDiff ℝ n (fun x : ConfigurationSpace => [x, x[ℝ]) := by
  have h_id : ContDiff ℝ n (fun x : ConfigurationSpace => x) := contDiff_id
  simp using (ContDiff.inner ([:=ℝ]) (f:=fun x : ConfigurationSpace => x)
    (g:=fun x : ConfigurationSpace => x) h_id h_id))

/-- Linear map sending a configuration space element to its underlying real
/
noncomputable def toRealLM : ConfigurationSpace →ℓ[ℝ] ℝ :=
  { toFun := ConfigurationSpace.val
    map_add' := by simp
    map_smul' := by simp }

/-- Linear map embedding a real value into the configuration space. -
/
noncomputable def fromRealLM : ℝ →ℓ[ℝ] ConfigurationSpace :=
  { toFun := fun x => ⟨x⟩
    map_add' := by
      intro x y
      ext
      simp
    map_smul' := by
      intro r x
      ext
      simp }

/-- Continuous linear map sending a configuration space element to its unde
/
noncomputable def toRealCLM : ConfigurationSpace →L[ℝ] ℝ :=
  toRealLM.mkContinuous 1 (by

```


1.6.

PHYSLEAN/CLASSICALMECHANICS/HARMONICOSCILLATOR/CONFIGURATIONSPACE.LEAN

```

    intro x
    simp [toRealLM, norm])

/-- Continuous linear map embedding a real value into the configuration space. -
/
noncomputable def fromRealCLM :  $\mathbb{R} \rightarrow L[\mathbb{R}]$  ConfigurationSpace :=
  fromRealLM.mkContinuous 1 (by
    intro x
    simp [fromRealLM, norm])

/-- Homeomorphism between configuration space and  $\mathbb{R}$  given by `ConfigurationSpace.val`
/
noncomputable def valHomeomorphism : ConfigurationSpace  $\approx_t$   $\mathbb{R}$  where
  toFun := ConfigurationSpace.val
  invFun := fun t => ⟨t⟩
  left_inv := by
    intro t
    cases t
    rfl
  right_inv := by
    intro t
    rfl
  continuous_toFun := by
    simpa [toRealCLM, toRealLM] using toRealCLM.continuous
  continuous_invFun := by
    simpa [fromRealCLM, fromRealLM] using fromRealCLM.continuous

/-- The structure of a charted space on `ConfigurationSpace`. -
/
noncomputable instance : ChartedSpace  $\mathbb{R}$  ConfigurationSpace where
  atlas := { valHomeomorphism.toOpenPartialHomeomorph }
  chartAt _ := valHomeomorphism.toOpenPartialHomeomorph
  mem_chart_source := by
    simp
  chart_mem_atlas := by
    intro x
    simp

open Manifold ContDiff

/-- The structure of a smooth manifold on `ConfigurationSpace`. -
/
noncomputable instance : IsManifold  $\square(\mathbb{R}, \mathbb{R})$   $\omega$  ConfigurationSpace where
  compatible := by
    intro e1 e2 h1 h2
    simp [atlas, ChartedSpace.atlas] at h1 h2

```

```

      subst h1 h2
      exact symm_trans_mem_contDiffGroupoid valHomeomorphism.toOpenPartialHom

instance : FiniteDimensional ℝ ConfigurationSpace := by
  classical
  refine FiniteDimensional.of_injective toRealLM ?_
  intro x y h
  ext
  simpa using h

instance : CompleteSpace ConfigurationSpace := by
  classical
  simpa using (FiniteDimensional.complete ℝ ConfigurationSpace)

/-!
## Map to space
-/

/-- The position in one-dimensional space associated to the configuration.
/
def toSpace (q : ConfigurationSpace) : Space 1 := ⟨fun _ => q.val⟩

@[simp]
lemma toSpace_apply (q : ConfigurationSpace) (i : Fin 1) : q.toSpace i = q.

end ConfigurationSpace

end HarmonicOscillator

end ClassicalMechanics

```

1.7 PhysLean/ClassicalMechanics/HarmonicOscillator/Solution.lean

```

x₀ : ConfigurationSpace
v₀ : ConfigurationSpace
x_t₀ : ConfigurationSpace
v_t₀ : ConfigurationSpace
noncomputable def trajectory (IC : InitialConditions) : Time → ConfigurationSpace
· apply fun_smul
  · change ContDiff ℝ _ (((fun x => cos x) ∘ (fun y => S.ω * y)) ∘ Time.toReal)
    refine ContDiff.comp_continuousLinearMap (ContDiff.comp contDiff_cos contDiff_sin)
    fun_prop
  · have hx := contDiff_sin (n := n)
    apply fun_smul

```

1.7.

PHYSLEAN/CLASSICALMECHANICS/HARMONICOSCILLATOR/SOLUTION1LEAN

```

    · change ContDiff ℝ _ (((fun x => sin x / S.ω) ∘ (fun y => S.ω * y)) ∘ Time.toReal)
      refine ContDiff.comp_continuousLinearMap (ContDiff.comp ?_ ?_)
    · fun_prop
    · fun_prop
  field_simp [mul_div_assoc, div_self, mul_one, S.ω_ne_zero]
  have hcont : ContDiff ℝ ∞ (IC.trajectory S) := trajectory_contDiff S IC
  rw [EquationOfMotion, gradLagrangian_eq_force (S := S) (x_t := IC.trajectory S) hcont]
  ext
  have hw : S.ω ≠ 0 := ω_ne_zero S
  have hwm : S.ω ^ 2 * S.m = S.k := by
    rw [ω_sq]
    field_simp [m_ne_zero S]
  simp [trajectory_eq, smul_add, smul_smul, mul_comm, mul_left_comm]
  rw [← hwm]
  field_simp [hw]
  ring
lemma trajectories_unique (IC : InitialConditions) (x : Time → ConfigurationSpace)
  set y : Time → ConfigurationSpace := fun t => x t - IC.trajectory S t with hydef
  ContDiffAt.differentiableAt hx_contDiffAt (by simp)
  ContDiffAt.differentiableAt httraj_contDiffAt (by simp)
  exact mul_ne_zero (by norm_num) (S.k_ne_zero)
  ext
  simp only [ConfigurationSpace.add_val, ConfigurationSpace.smul_val, ConfigurationSpace.zero_val]
  field_simp [S.ω_ne_zero]
  linear_combination S.ω * IC.x_t0.val * h1
  ext
  simp only [ConfigurationSpace.add_val, ConfigurationSpace.smul_val, ConfigurationSpace.zero_val,
    neg_mul]
  field_simp [S.ω_ne_zero]
  linear_combination IC.v_t0.val * h1
  have hx : S.ω ≠ 0 := by exact ω_ne_zero S
  ext
  simp only [ConfigurationSpace.zero_val]
  ring_nf
  · simp only [ConfigurationSpace.add_val, ConfigurationSpace.smul_val, ConfigurationSpace.zero_val,
    ring_nf]
  simp [neg_smul]
  have hx' : S.ω ≠ 0 := by exact ω_ne_zero S
  ext
  simp [one_div]
  simp [mul_eq_zero, inv_eq_zero]
  ext
  · exact k_ne_zero S
    simp only [one_div, ne_eq, m_ne_zero, not_false_eq_true, inv_mul_cancel_left]
  apply ω_ne_zero

```

1.8 PhysLean/ClassicalMechanics/Mass/MassUnit.lean

```

lemma val_ne_zero (x : MassUnit) : x.val ≠ 0 := by
lemma div_ne_zero (x y : MassUnit) : ¬ x / y = (0 : ℝ≥0) := by
  · exact Ne.symm (div_ne_zero x y)
  simp [div_eq_val, x.val_ne_zero]

```

1.9 PhysLean/ClassicalMechanics/RigidBody/SolidSphere.lean

```

· apply Space.volume_closedBall_ne_top
· apply Space.volume_closedBall_ne_zero

```

1.10 PhysLean/CondensedMatter/TightBindingChain/Basic.lean

```

rw [Fin.val_sub]
  · rw [@Fin.val_sub]
calc
  _ = Complex.exp (Complex.I * k2 * 1 * T.N * T.a - Complex.I * k1 *
    ring_nf

```

1.11 PhysLean/Electromagnetism/Charge/ChargeUnit.lean

```

lemma val_ne_zero (x : ChargeUnit) : x.val ≠ 0 := by
lemma div_ne_zero (x y : ChargeUnit) : ¬ x / y = (0 : ℝ≥0) := by
  · exact Ne.symm (div_ne_zero x y)
  simp [div_eq_val, x.val_ne_zero]

```

1.12 PhysLean/Electromagnetism/Dynamics/IsExtrema.lean

```

(hJ.differentiable (by simp)) _ _).const_mul _).differentiableAt,
(hJ.differentiable (by simp)) _ _).differentiableAt,
(hJ.differentiable (by simp)) _ _).const_mul _).differentiableAt,
(hJ.differentiable (by simp)) _ _).differentiableAt,

```

1.13 PhysLean/Electromagnetism/Dynamics/KineticTerm.lean

```

(by simp)
(by simp)
(by simp)

```

1.14.

PHYSLEAN/ELECTROMAGNETISM/KINEMATICS/FIELDSTRENGTH.LEAN15

1.14 PhysLean/Electromagnetism/Kinematics/FieldStrength.lean

(by simp)

1.15 PhysLean/Mathematics/Calculus/Divergence.lean

```
simp only [Matrix.trace, Matrix.diag, LinearMap.toMatrix_apply]
rfl
```

1.16 PhysLean/Mathematics/DataStructures/Matrix/LieTrace.lean

```
(NormedSpace.exp A).BlockTriangular id := by
rw [NormedSpace.exp_eq_tsum []]
(NormedSpace.exp A).diag = fun i => NormedSpace.exp (A i i) := by
rw [NormedSpace.exp_eq_tsum [], diag_apply]
rw [NormedSpace.exp_eq_tsum []]
(NormedSpace.exp A).det = NormedSpace.exp A.trace := by
have h_exp_upper : BlockTriangular (NormedSpace.exp A) id :=
have h_diag_exp : (NormedSpace.exp A).diag = fun i => NormedSpace.exp (A i i) :=
  NormedSpace.exp ((U : Matrix m m ℝ) * A * star (U : Matrix m m ℝ)) =
    (U : Matrix m m ℝ) * NormedSpace.exp A * star (U : Matrix m m ℝ) := by
have h_units := Matrix.exp_units_conj Uu A
(NormedSpace.exp ((U : Matrix m m ℝ) * A * star (U : Matrix m m ℝ))).det =
  (NormedSpace.exp A).det := by
(NormedSpace.exp A).det = NormedSpace.exp A.trace := by
have h_det_invariant : (NormedSpace.exp A).det = (NormedSpace.exp T.val).det := by
have h_triangular_case : (NormedSpace.exp T.val).det = NormedSpace.exp T.val.trace
  (exp A).map (algebraMap ℝ ℂ) = exp (A.map (algebraMap ℝ ℂ)) := by
simp only [exp_eq_tsum ℝ]
simp
(A : Matrix n n ℝ) : (NormedSpace.exp A).det = Real.exp A.trace := by
have h_complex : (NormedSpace.exp A_ℂ).det = Complex.exp A_ℂ.trace := by
have h_det_comm : (algebraMap ℝ ℂ) ((NormedSpace.exp A).det) = (NormedSpace.exp A_ℂ).det
```

1.17 PhysLean/Mathematics/Distribution/Basic.lean

```
import Mathlib.Analysis.Distribution.TemperedDistribution
SchwartzMap.evalCLM (□ := □) E □ v <|
trans -u ((SchwartzMap.evalCLM (□ := □) E □ v1) ((fderivCLM □) E □ η) +
  (SchwartzMap.evalCLM (□ := □) E □ v2) ((fderivCLM □) E □ η))
trans u (a • (SchwartzMap.evalCLM (□ := □) E □ v1) ((fderivCLM □) E □ η))
fderivD □ u η v = - u (SchwartzMap.evalCLM (□ := □) E □ v (SchwartzMap.fderivCLM
```

```

simp only [evalCLM_apply_apply, fderivCLM_apply, neg_neg]
  change Integrable (SchwartzMap.evalCLM (□ := ℝ) E ℝ v (SchwartzMap.fder
    exact integrable ((SchwartzMap.evalCLM ℝ E ℝ v) ((fderivCLM ℝ) E ℝ η))
open TemperedDistribution ContinuousLinearMap

toPointwiseConvergenceCLM _ _ _ _ <|
  (BoundedContinuousFunction.evalCLM □ a).comp (toBoundedContinuousFuncti

```

1.18 PhysLean/Mathematics/Fin.lean

```

simp only [Fin.lt_def, Fin.val_castSucc, not_lt, Fin.val_succ] at h1 h2
  simp_all only [Nat.succ_eq_add_one, Fin.val_castSucc]
  simp_all only [Nat.succ_eq_add_one, Fin.val_castSucc, Fin.val_suc
  simp_all only [Nat.succ_eq_add_one, Fin.val_castSucc, Fin.val_suc
  simp_all only [Nat.succ_eq_add_one, Fin.val_castSucc]
  simp_all only [Nat.succ_eq_add_one, Fin.val_castSucc, Fin.val_succ]
  simp_all only [Nat.succ_eq_add_one, Fin.val_castSucc, Fin.val_suc
  Equiv.coe_refl, Sum.map_inr, finCongr_apply, Fin.val_cast]
simp_all only [Nat.succ_eq_add_one, not_lt, Fin.le_def, Fin.val_castSuc
  simp only [Nat.succ_eq_add_one, Fin.val_cast, Fin.natAdd_mk]

```

1.19 PhysLean/Mathematics/Fin/Involutions.lean

```

· simp only [hj, ↓reduceDIte, Fin.val_pred]
· simp only [hj, ↓reduceDIte, Fin.val_pred]
simp only [Fin.val_pred]

```

1.20 PhysLean/Mathematics/Geometry/Metric/PseudoRiemannian/Defs

```

unfold LinearMap.BilinForm.Nondegenerate LinearMap.Nondegenerate
  LinearMap.SeparatingLeft LinearMap.SeparatingRight
constructor
· intro v hv; simp_rw [toBilinForm_apply] at hv; exact g.nondegenerate x
· intro v hv; simp_rw [toBilinForm_apply] at hv;
  have hw : ∀ (w : TangentSpace I x), ((g.val x) v) w = 0 := by
    intro w; rw [symm]; simp [hv]
  exact g.nondegenerate x v hw
unfold LinearMap.BilinForm.Nondegenerate LinearMap.Nondegenerate
  LinearMap.SeparatingLeft LinearMap.SeparatingRight
constructor
· intro w hw
  apply cotangentMetricVal_nondegenerate g x w

```

1.21.

PHYSLEAN/MATHEMATICS/INNERPRODUCTSPACE/SUBMODULE.LEAN 17

```
    intro v
    exact hw v
  · intro ω hw
    apply cotangentMetricVal_nondegenerate g x ω
    intro v
    have hv : ∀ (y : TangentSpace I x → L[ℝ] ℝ), ((g.cotangentToBilinForm x) ω) y = 0
      intro y; rw [LinearMap.BilinForm.isSymm_def.mp (cotangentToBilinForm_isSymm g
    exact hv v
```

1.21 PhysLean/Mathematics/InnerProductSpace/Submodule.lean

/-

Copyright (c) 2026 Gregory J. Loges. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Gregory J. Loges

-/

import Mathlib.Analysis.InnerProductSpace.LinearPMap

/-!

Submodules of $E \times E$

In this module we define `submoduleToLp` which reinterprets a submodule of $E \times E$, where E is an inner product space, as a submodule of $\text{WithLp } 2 (E \times E)$. This allows us to take advantage of the inner product structure, since otherwise by default $E \times E$ is given the sup norm.

-/

namespace InnerProductSpaceSubmodule

open LinearPMap Submodule

variable

{E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℂ E]
(M : Submodule ℂ (E × E))

/-- The submodule of $\text{WithLp } 2 (E \times E)$ defined by M . -

/

def submoduleToLp : Submodule ℂ (WithLp 2 (E × E)) where

carrier := {x | x.ofLp ∈ M}

add_mem' := by

intro a b ha hb

exact Submodule.add_mem M ha hb

zero_mem' := Submodule.zero_mem M

```

smul_mem' := by
  intro c x hx
  exact Submodule.smul_mem M c hx

lemma mem_submodule_iff_mem_submoduleToLp (f : E × E) :
  f ∈ M ↔ (WithLp.toLp 2 f) ∈ submoduleToLp M := Eq.to_iff rfl

lemma submoduleToLp_closure :
  (submoduleToLp M.topologicalClosure) = (submoduleToLp M).topologicalClosure
  rw [Submodule.ext_iff]
  intro x
  rw [← mem_submodule_iff_mem_submoduleToLp]
  change x.ofLp ∈ _root_.closure M ↔ x ∈ _root_.closure (submoduleToLp M)
  repeat rw [mem_closure_iff_nhds]
  constructor
  · intro h t ht
    apply mem_nhds_iff.mp at ht
    rcases ht with ⟨t1, ht1, ht1', hx⟩
    have : ∃ t' ∈ nhds x.ofLp, (∀ y ∈ t', WithLp.toLp 2 y ∈ t1) := by
      refine Filter.eventually_iff_exists_mem.mp ?_
      apply ContinuousAt.eventually_mem (by fun_prop) (IsOpen.mem_nhds ht1')
    rcases this with ⟨t2, ht2, ht2'⟩
    rcases h t2 ht2 with ⟨w, hw⟩
    use WithLp.toLp 2 w
    exact ⟨Set.mem_preimage.mp (ht1 (ht2' w hw.1)),
      (mem_submodule_iff_mem_submoduleToLp M w).mpr hw.2⟩
  · intro h t ht
    apply mem_nhds_iff.mp at ht
    rcases ht with ⟨t1, ht1, ht1', hx⟩
    have : ∃ t' ∈ nhds x, (∀ y ∈ t', y.ofLp ∈ t1) := by
      refine Filter.eventually_iff_exists_mem.mp ?_
      exact ContinuousAt.eventually_mem (by fun_prop) (IsOpen.mem_nhds ht1')
    rcases this with ⟨t2, ht2, ht2'⟩
    rcases h t2 ht2 with ⟨w, hw⟩
    use w.ofLp
    exact ⟨Set.mem_preimage.mp (ht1 (ht2' w hw.1)), (mem_toAddSubgroup (submoduleToLp M).topologicalClosure)⟩

lemma mem_submodule_closure_iff_mem_submoduleToLp_closure (f : E × E) :
  f ∈ M.topologicalClosure ↔ (WithLp.toLp 2 f) ∈ (submoduleToLp M).topologicalClosure
  rw [← submoduleToLp_closure]
  rfl

lemma mem_submodule_adjoint_iff_mem_submoduleToLp_orthogonal (f : E × E) :
  f ∈ M.adjoint ↔ WithLp.toLp 2 (f.2, -f.1) ∈ (submoduleToLp M)⊥ := by
  constructor <=> intro h
  · rw [mem_orthogonal]

```



```

    intro u hu
    rw [mem_adjoint_iff] at h
    have h' : inner ℂ u.snd f.1 = inner ℂ u.fst f.2 := by
      rw [← sub_eq_zero]
      exact h u.fst u.snd hu
    simp [h']
  · rw [mem_adjoint_iff]
    intro a b hab
    rw [mem_orthogonal] at h
    have hab' := (mem_submodule_iff_mem_submoduleToLp M (a, b)).mp hab
    have h' : inner ℂ a f.2 = inner ℂ b f.1 := by
      rw [← sub_eq_zero, sub_eq_add_neg, ← inner_neg_right]
      exact h (WithLp.toLp 2 (a, b)) hab'
    simp [h']

lemma mem_submodule_adjoint_adjoint_iff_mem_submoduleToLp_orthogonal_orthogonal (f : E
  f ∈ M.adjoint.adjoint ↔ WithLp.toLp 2 f ∈ (submoduleToLp M)[] := by
  simp only [mem_adjoint_iff]
  trans ∀ v, (∀ w ∈ submoduleToLp M, inner ℂ w v = 0) → inner ℂ v (WithLp.toLp 2 f)
  · constructor <=> intro h
    · intro v hw
      have h' := h (-v.snd) v.fst
      rw [inner_neg_left, sub_neg_eq_add] at h'
      apply h'
      intro a b hab
      rw [inner_neg_right, neg_sub_left, neg_eq_zero]
      exact hw (WithLp.toLp 2 (a, b)) ((mem_submodule_iff_mem_submoduleToLp M (a, b))
    · intro a b h'
      rw [sub_eq_add_neg, ← inner_neg_left]
      apply h (WithLp.toLp 2 (b, -a))
      intro w hw
      have hw' := h' w.fst w.snd hw
      rw [sub_eq_zero] at hw'
      simp [hw']
  simp only [← mem_orthogonal]

lemma mem_submodule_closure_adjoint_iff_mem_submoduleToLp_closure_orthogonal (f : E
  f ∈ M.topologicalClosure.adjoint ↔
  WithLp.toLp 2 (f.2, -f.1) ∈ (submoduleToLp M).topologicalClosure[] := by
  rw [mem_submodule_adjoint_iff_mem_submoduleToLp_orthogonal, submoduleToLp_closure]

end InnerProductSpaceSubmodule

```

1.22 PhysLean/Mathematics/KroneckerDelta.lean

```

/-
Copyright (c) 2026 Gregory J. Loges. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Gregory J. Loges
-/
import Mathlib.Algebra.Ring.Basic
import PhysLean.Meta.TODO.Basic
/-!

# Kronecker delta

This file defines the Kronecker delta,  $\delta[i,j] = \text{if } (i = j) \text{ then } 1 \text{ else } 0$ .

-/
TODO "YVABB" "Build functionality for working with sums involving Kronecker

namespace KroneckerDelta

/-- The Kronecker delta function,  $\text{ite } (i = j) \ 1 \ 0$ . -/
def kroneckerDelta [Ring R] (i j : Fin d) : R := if (i = j) then 1 else 0

@[inherit_doc kroneckerDelta]
macro "δ[" i:term "," j:term "]" : term => `(kroneckerDelta $i $j)

lemma kroneckerDelta_symm [Ring R] (i j : Fin d) :
  kroneckerDelta (R := R) i j = kroneckerDelta j i :=
  ite_cond_congr (Eq.propIntro Eq.symm Eq.symm)

lemma kroneckerDelta_self [Ring R] : ∀ i : Fin d, kroneckerDelta (R := R) i
  intro i
  exact if_pos rfl

end KroneckerDelta

```

1.23 PhysLean/Mathematics/List.lean

```

import Mathlib.Data.List.TakeWhile
  · simp only [List.tail_cons, hPb, decide_false, Bool.false_eq_true, not
    List.dropWhile_cons_of_neg]
    simp_all only [decide_eq_true_eq, List.cons_ne_self, ↓reduceIte]
    next x heq => simp_all only [decide_eq_false_iff_not, ↓reduceIte, Lis
[Std.Total le1] [IsTrans I le1] (r0 : I) (r : List I) (hs : List.Pairwi

```

```

simp only [Fin.eta, Fin.val_cast, Fin.val_fin_lt] at hx
  simp only [Fin.succAbove, Fin.castSucc_mk, Fin.mk_lt_mk, Fin.succ_mk, Fin.val_ca
    Fin.castSucc_mk, Fin.mk_lt_mk, Fin.succ_mk, Fin.val_cast]
    Fin.cast_zero, Fin.val_cast]
    Fin.castOrderIso_apply, Fin.cast_mk, Fin.val_cast]
  simp only [orderedInsertPos_sigma, Fin.val_cast]
set_option maxHeartbeats 350000
  simp only [List.getElem_cons_zero, Fin.zero_eta, orderedInsertEquiv_zero,
    List.getElem_insertIdx_self]
  simp only [Fin.succAbove, Fin.val_cast]
  · simp only [hi, ↓reduceIte, Fin.val_castSucc, dite_eq_left_iff, not_lt]
  simp_all only [Fin.val_castSucc, Fin.val_cast]
  simp only [Fin.val_castSucc, Fin.val_cast, not_lt] at hi
  [Std.Total l1] [IsTrans I l1] :
  [Std.Total l1] [IsTrans I l1] (r : List I) (n : Fin r.length) :
  [Std.Total r] [IsTrans α r] (i : α) (l : List α) :

```

1.24 PhysLean/Mathematics/List/InsertionSort.lean

```

[Std.Total r] [IsTrans α r] (a : α) (l : List α)
  simp only [List.length_cons, Function.comp_apply, List.get_eq_getElem, Fin.val_c
simp only [List.length_cons, ne_eq, Fin.ext_iff, Fin.val_cast]
[Std.Total r] [IsTrans α r] (l1 : List α) :
[Std.Total r] [IsTrans α r] (a b : α) (hr : ¬ r a b) : (l : List α) →
  have ht := Std.Total.total (r := r) a b
  have ht := Std.Total.total (r := r) a b
[Std.Total r] [IsTrans α r] (a : α) : (l1 l2 : List α) →
[Std.Total r] [IsTrans α r] : (l1 l2 : List α) →
[Std.Total r] [IsTrans α r] : (l1 l2 l3 : List α) →
[Std.Total r] [IsTrans α r]
  have ht := Std.Total.total (r := r) a b
[Std.Total r] [IsTrans α r]
  have ht := Std.Total.total (r := r) a b
  have ht := Std.Total.total (r := r) a b
[Std.Total r] [IsTrans α r] (a b : α) (hr : ¬ r a b) (n : ℕ) : (l : List α) →
  have ht := Std.Total.total (r := r) a b
simp only [Fin.eta, Fin.val_cast]
simp only [Fin.val_cast, Fin.eta, Fin.cast_cast]
  [Std.Total r] [IsTrans α r] (a a2 : α) : (l1 l2 : List α) →
  [Std.Total r] [IsTrans α r] (a : α) : (l1 l2 : List α) →
lemma insertionSort_filter {α : Type} (r : α → α → Prop) [DecidableRel r] [Std.Total
  [Std.Total r] [IsTrans α r] (a : α) : (l : List α) →
  [Std.Total r] [IsTrans α r] (a : α) (l1 l l2 : List α)
  [Std.Total r] [IsTrans α r] (a : α) (l1 l2 : List α) :

```

1.25 PhysLean/Mathematics/S03/Basic.lean

```
by simp only [transpose_transpose, mul_eq_one_comm.mpr A.2.2]}
inv_mul_cancel A := Subtype.ext (mul_eq_one_comm.mpr A.2.2)
(inv_eq_left_inv (mul_eq_one_comm.mpr A.2.2)).symm
toFun A := (A.1, (A-1).1, A.2.2, mul_eq_one_comm.mpr A.2.2)
```

1.26 PhysLean/Mathematics/Trigonometry/Tanh.lean

```
import Mathlib.Analysis.Distribution.SchwartzSpace.Deriv
· exact le_of_lt (neg_one_lt_tanh x)
```

1.27 PhysLean/Mathematics/VariationalCalculus/HasVarAdjDeriv.lean

```
exact fun x => (h.smooth_linear hφ).differentiable (by simp) x
  rw [← fderiv_apply_one_eq_deriv]
  rw [← fderiv_apply_one_eq_deriv]
  rw [← fderiv_apply_one_eq_deriv]
  rw [← fderiv_apply_one_eq_deriv]
  · rw [← fderiv_apply_one_eq_deriv, ← fderiv_apply_one_eq_deriv, DifferentiableAt.comp]
    conv => enter [3, 1, x, y]; rw [← fderiv_apply_one_eq_deriv]
    conv => enter [3, 1, x, y]; rw [← fderiv_apply_one_eq_deriv]
  · rw [← fderiv_apply_one_eq_deriv, ← fderiv_apply_one_eq_deriv, DifferentiableAt.comp]
    rw [← fderiv_apply_one_eq_deriv, ← fderiv_apply_one_eq_deriv, DifferentiableAt.comp]
    rw [← fderiv_apply_one_eq_deriv, fderiv_comp']
    rw [← fderiv_apply_one_eq_deriv]
    rw [← fderiv_apply_one_eq_deriv, fderiv_comp', fderiv_fst]
  rw [← fderiv_apply_one_eq_deriv, fderiv_comp']
  rw [← fderiv_apply_one_eq_deriv]
  rw [← fderiv_apply_one_eq_deriv, fderiv_comp', fderiv_snd]
  rw [← fderiv_apply_one_eq_deriv]
  rw [← fderiv_apply_one_eq_deriv]
  rw [← fderiv_apply_one_eq_deriv]
  rw [← fderiv_apply_one_eq_deriv]
  have := hφ.smooth.differentiable (by simp)
  have := hu.differentiable (by simp)
  · change DifferentiableAt ℝ ((fun sx : ℝ × X => F (φ sx.1) sx.2) ∘ fun
    apply DifferentiableAt.comp
  · have hf := hF.diff φ hφ
    apply ContDiff.differentiable hf (by simp)
  · fun_prop
```

1.28.

PHYSLEAN/MATHEMATICS/VARIATIONALCALCULUS/ISTESTFUNCTION21LEAN

```

· change DifferentiableAt  $\mathbb{R}$  ((fun sx :  $\mathbb{R} \times X \Rightarrow G$  ( $\phi$  sx.1) sx.2)  $\circ$  fun s' => (s'
  apply DifferentiableAt.comp
  · have hg := hG.diff  $\phi$  h $\phi$ 
    apply ContDiff.differentiable hg (by simp)
  · fun_prop
    |>.differentiable (by simp) 0
    |>.differentiable (by simp) 0
· change DifferentiableAt  $\mathbb{R}$  ((fun sx :  $\mathbb{R} \times X \Rightarrow F$  ( $\phi$  sx.1) sx.2)  $\circ$  fun s' => (s'
  apply DifferentiableAt.comp
  · have hf := hF.diff  $\phi$  h $\phi$ 
    apply ContDiff.differentiable hf (by simp)
  · fun_prop
· change DifferentiableAt  $\mathbb{R}$  ((fun sx :  $\mathbb{R} \times X \Rightarrow G$  ( $\phi$  sx.1) sx.2)  $\circ$  fun s' => (s'
  apply DifferentiableAt.comp
  · have hg := hG.diff  $\phi$  h $\phi$ 
    apply ContDiff.differentiable hg (by simp)
  · fun_prop
    |>.differentiable (by simp) 0
    |>.differentiable (by simp) 0
  · convert hF.adjoint
    rw [deriv_smul_const, deriv_id'', one_smul]
    fun_prop
  · exact apply_smooth_self hG
  · convert hG.adjoint
    rw [deriv_smul_const, deriv_id'', one_smul]
    fun_prop
  · exact apply_smooth_self hF
rw [← fderiv_apply_one_eq_deriv]
simp only [Space.deriv]
rw [← fderiv_apply_one_eq_deriv]
simp only [Space.deriv]
rw [← fderiv_apply_one_eq_deriv]
rw [← fderiv_apply_one_eq_deriv]
rw [← fderiv_apply_one_eq_deriv, fderiv_comp]

```

1.28 PhysLean/Mathematics/VariationalCalculus/IsTestFunction.lean

```
Differentiable  $\mathbb{R}$  f := hf.1.differentiable (by simp)
```

1.29 PhysLean/Meta/Basic.lean

```
import ImportGraph.Imports.RequiredModules
```

1.30 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancel

```
import Mathlib.Tactic.LinearCombination
```

1.31 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancel

```
lemma genericToQuad_ne_zero (S : (PlusU1 n).QuadSols) (h :  $\alpha_1 \in S.1 \neq 0$ ) :
  rw [genericToQuad_ne_zero C S h]
```

1.32 PhysLean/Particles/BeyondTheStandardModel/RHN/AnomalyCancel

```
lemma quadSolToSolInv_α1_α2_ne_zero (S : (PlusU1 n).Sols) (h :  $\alpha_1 \in S.1 \neq 0$ ) :
  · rw [quadSolToSol, dif_neg (quadSolToSolInv_α1_α2_ne_zero S h)]
```

1.33 PhysLean/Particles/BeyondTheStandardModel/TwoHDM/GramMatrix

```
/-- The lemma manifesting the definitional equality for the gramVector. -
/
lemma gramVector_eq (H : TwoHiggsDoublet) : H.gramVector = fun μ =>
  2 * PauliMatrix.pauliBasis.repr (gramMatrix H, gramMatrix_selfAdjoint H)

lemma mem_orbit_gaugeGroupI_iff_gramVector (H1 H2 : TwoHiggsDoublet) :
  H1 ∈ MulAction.orbit GaugeGroupI H2 ↔ H1.gramVector = H2.gramVector :=
  rw [mem_orbit_gaugeGroupI_iff_gramMatrix]
  constructor
  · intro h
    rw [gramVector_eq, gramVector_eq]
    funext μ
    congr
  · intro h
    rw [gramMatrix_eq_gramVector_sum_pauliMatrix,
      gramMatrix_eq_gramVector_sum_pauliMatrix, h]
```

1.34 PhysLean/Particles/BeyondTheStandardModel/TwoHDM/Potential

```
# The potential of the Two Higgs doublet model
In this module we give the define the parameters of the 2HDM potential, and
give stability properties of the potential.
```

```
## ii. Key results
```

1.34.

PHYSLEAN/PARTICLES/BEYONDTHESTANDARDMODEL/TWOHDM/POTENTIAL.LEAN

- `PotentialParameters` : The parameters of the 2HDM potential.
- `massTerm` : The mass term of the 2HDM potential.
- `quarticTerm` : The quartic term of the 2HDM potential.
- `potential` : The full potential of the 2HDM.
- `PotentialIsStable` : The condition that the potential is stable.

iii. Table of contents

- A. The parameters of the potential
 - A.1. The potential parameters corresponding to zero
 - A.2. Gram parameters
 - A.3. Specific cases
- B. The mass term
- C. The quartic term
- D. The full potential
- E. Stability of the potential
 - E.1. The stability condition
 - E.2. Instability of the stabilityCounterExample potential
 - E.3. The reduced mass term
 - E.4. The reduced quartic term
 - E.5. Stability in terms of the gram vectors
 - E.6. Strong stability implies stability
 - E.7. Showing step in hep-ph/0605184 is invalid

iv. References

For the parameterization of the potential we follow the convention of

- <https://arxiv.org/pdf/1605.03237>

Stability arguments of the potential follow, in part, those from

- <https://arxiv.org/abs/hep-ph/0605184>

Although we note that we explicitly prove that one of the steps in this paper is not

/-!

A. The parameters of the potential

We define a type for the parameters of the Higgs potential in the 2HDM.

We follow the convention of `1605.03237`, which is highlighted in the explicit const of the potential itself.

We relate these parameters to the ` $\xi$ ` and ` $\eta$ ` parameters used in the gram vector for given in arXiv:hep-ph/0605184.

-/

```
/-!
```

```
### A.1. The potential parameters corresponding to zero
```

We define an instance of `Zero` for the potential parameters, corresponding parameters being zero, and therefore the potential itself being zero.

```
-/
```

```
instance : Zero PotentialParameters where
```

```
  zero :=
    { m112 := 0
      m222 := 0
      m122 := 0
       $\square_1$  := 0
       $\square_2$  := 0
       $\square_3$  := 0
       $\square_4$  := 0
       $\square_5$  := 0
       $\square_6$  := 0
       $\square_7$  := 0 }
```

```
@[simp] lemma zero_m112 : (0 : PotentialParameters).m112 = 0 := rfl
```

```
@[simp] lemma zero_m222 : (0 : PotentialParameters).m222 = 0 := rfl
```

```
@[simp] lemma zero_m122 : (0 : PotentialParameters).m122 = 0 := rfl
```

```
@[simp] lemma zero_ $\square_1$  : (0 : PotentialParameters). $\square_1$  = 0 := rfl
```

```
@[simp] lemma zero_ $\square_2$  : (0 : PotentialParameters). $\square_2$  = 0 := rfl
```

```
@[simp] lemma zero_ $\square_3$  : (0 : PotentialParameters). $\square_3$  = 0 := rfl
```

```
@[simp] lemma zero_ $\square_4$  : (0 : PotentialParameters). $\square_4$  = 0 := rfl
```

```
@[simp] lemma zero_ $\square_5$  : (0 : PotentialParameters). $\square_5$  = 0 := rfl
```

```
@[simp] lemma zero_ $\square_6$  : (0 : PotentialParameters). $\square_6$  = 0 := rfl
```

```
@[simp] lemma zero_ $\square_7$  : (0 : PotentialParameters). $\square_7$  = 0 := rfl
```

```
/-!
```

```
### A.2. Gram parameters
```


1.34.

PHYSLEAN/PARTICLES/BEYONDTHESTANDARDMODEL/TWOHDM/POTENTIAL.LEAN

A reparameterization of the potential parameters corresponding to ξ and η in arXiv:hep-ph/0605184.

-/

```
noncomputable def  $\xi$  (P : PotentialParameters) ( $\mu$  : Fin 1  $\oplus$  Fin 3) :  $\mathbb{R}$  :=
| .inl 0 => (P.m112 + P.m222) / 2
| .inr 0 => -Complex.re P.m122
| .inr 1 => Complex.im P.m122
| .inr 2 => (P.m112 - P.m222) / 2
```

@[simp]

lemma ξ_zero : (θ : PotentialParameters). ξ = 0 := by

ext μ

fin_cases μ <=> simp [ξ]

| .inl 0, .inl 0 => (P. $\square_1$ + P. $\square_2$ + 2 * P. $\square_3$) / 8

| .inl 0, .inr 0 => (P. $\square_6$.re + P. $\square_7$.re) / 4

| .inl 0, .inr 1 => - (P. $\square_6$.im + P. $\square_7$.im) / 4

| .inl 0, .inr 2 => (P. $\square_1$ - P. $\square_2$) / 8

| .inr 0, .inl 0 => (P. $\square_6$.re + P. $\square_7$.re) / 4

| .inr 1, .inl 0 => - (P. $\square_6$.im + P. $\square_7$.im) / 4

| .inr 2, .inl 0 => (P. $\square_1$ - P. $\square_2$) / 8

| .inr 0, .inr 0 => (P. $\square_5$.re + P. $\square_4$) / 4

| .inr 1, .inr 1 => (P. $\square_4$ - P. $\square_5$.re) / 4

| .inr 2, .inr 2 => (P. $\square_1$ + P. $\square_2$ - 2 * P. $\square_3$) / 8

| .inr 0, .inr 1 => - P. $\square_5$.im / 4

| .inr 2, .inr 0 => (P. $\square_6$.re - P. $\square_7$.re) / 4

| .inr 2, .inr 1 => (P. $\square_7$.im - P. $\square_6$.im) / 4

| .inr 1, .inr 0 => - P. $\square_5$.im / 4

| .inr 0, .inr 2 => (P. $\square_6$.re - P. $\square_7$.re) / 4

| .inr 1, .inr 2 => (P. $\square_7$.im - P. $\square_6$.im) / 4

@[simp]

lemma η_zero : (θ : PotentialParameters). η = 0 := by

ext μ v

fin_cases μ <=> fin_cases v <=> simp [η]

/-!

A.3. Specific cases

-/

-- An example of potential parameters that serve as a counterexample to the stability condition given in arXiv:hep-ph/0605184.

This corresponds to the potential:

$\|H.\Phi_1\|^2 * (\|H.\Phi_1\|, H.\Phi_2\|_{\mathbb{C}}).im + \|H.\Phi_1 - H.\Phi_2\|^4$

which has the property that the quartic term is non-negative and only zero when the mass term is also zero, but the potential is not stable.

In the proof that ``stabilityCounterExample_not_potentialIsStable``, we give explicit vectors ``H.Φ1`` and ``H.Φ2`` that show this potential is not stable.

This is the first occurrence of such a counterexample in the literature to the author's knowledge.

-/

```
def stabilityCounterExample : PotentialParameters := {(θ : PotentialParameters
  m12 := Complex.I
  φ1 := 2
  φ2 := 2
  φ3 := 2
  φ4 := 2
  φ5 := 2
  φ6 := -2
  φ7 := -2}
```

```
lemma stabilityCounterExample_ξ :
  stabilityCounterExample.ξ = fun
  | .inl 0 => 0
  | .inr 0 => 0
  | .inr 1 => 1
  | .inr 2 => 0 := by
  funext μ
  simp [stabilityCounterExample, ξ]
```

```
lemma stabilityCounterExample_η :
  stabilityCounterExample.η = fun μ => fun v =>
  match μ, v with
  | .inl 0, .inl 0 => 1
  | .inl 0, .inr 0 => -1
  | .inl 0, .inr 1 => 0
  | .inl 0, .inr 2 => 0
  | .inr 0, .inl 0 => -1
  | .inr 1, .inl 0 => 0
  | .inr 2, .inl 0 => 0
  | .inr 0, .inr 0 => 1
  | .inr 1, .inr 1 => 0
  | .inr 2, .inr 2 => 0
  | .inr 0, .inr 1 => 0
  | .inr 2, .inr 0 => 0
  | .inr 2, .inr 1 => 0
  | .inr 1, .inr 0 => 0
  | .inr 0, .inr 2 => 0
```

1.34.

PHYSLEAN/PARTICLES/BEYONDTHESTANDARDMODEL/TWOHDM/POTENTIAL.LEAN

```

    | .inr 1, .inr 2 => 0 := by
  funext μ v
  simp [stabilityCounterExample, η]
  ring_nf

```

/-!

B. The mass term

We define the mass term of the potential, write it in terms of the gram vector, and prove that it is gauge invariant.

-/

@[simp]

```

lemma massTerm_zero : massTerm 0 = 0 := by
  ext H
  simp [massTerm]

```

```

lemma massTerm_stabilityCounterExample (H : TwoHiggsDoublet) :

```

```

  massTerm PotentialParameters.stabilityCounterExample H =

```

```

  2 * (⟦H.Φ1, H.Φ2⟧_C).im := by

```

```

  simp [massTerm, PotentialParameters.stabilityCounterExample]

```

```

  rw [show ⟦H.Φ2, H.Φ1⟧_C = conj ⟦H.Φ1, H.Φ2⟧_C from Eq.symm (conj_inner_symm H.Φ2 H

```

```

  rw [Complex.conj_im]

```

```

  ring_nf

```

/-!

C. The quartic term

We define the quartic term of the potential, write it in terms of the gram vector, and prove that it is gauge invariant.

-/

@[simp]

```

lemma quarticTerm_zero : quarticTerm 0 = 0 := by
  ext H
  simp [quarticTerm]

```

```

lemma quarticTerm_stabilityCounterExample (H : TwoHiggsDoublet) :

```

```

  quarticTerm .stabilityCounterExample H =

```

```

  (⟦H.Φ1⟧ ^ 2 + ⟦H.Φ2⟧ ^ 2 - 2 * (⟦H.Φ1, H.Φ2⟧_C).re) ^ 2 := by

```

```

  /- Proof by calculation. -/

```

```

  calc _ = (⟦H.Φ1⟧ ^ 2 + ⟦H.Φ2⟧ ^ 2) ^ 2

```

```

+ 2 * ||H.ϕ1, H.ϕ2_C|| ^ 2
+ (H.ϕ1, H.ϕ2_C ^ 2 + H.ϕ2, H.ϕ1_C ^ 2).re
- 2 * (||H.ϕ1|| ^ 2 + ||H.ϕ2|| ^ 2) * ((H.ϕ1, H.ϕ2_C).re + (H.ϕ2, H.ϕ1_C).re)
  simp [quarticTerm, PotentialParameters.stabilityCounterExample, Complex.ofReal_pow]
  ring
  _ = (||H.ϕ1|| ^ 2 + ||H.ϕ2|| ^ 2) ^ 2
+ 4 * (H.ϕ1, H.ϕ2_C).re ^ 2
- 2 * (||H.ϕ1|| ^ 2 + ||H.ϕ2|| ^ 2) * ((H.ϕ1, H.ϕ2_C).re + (H.ϕ2, H.ϕ1_C).re)
  have h1 : 2 * ||H.ϕ1, H.ϕ2_C|| ^ 2
    + (H.ϕ1, H.ϕ2_C ^ 2 + H.ϕ2, H.ϕ1_C ^ 2).re = 4 * (H.ϕ1, H.ϕ2_C).re ^ 2 := by
    rw [show H.ϕ2, H.ϕ1_C = conj H.ϕ1, H.ϕ2_C from Eq.symm (conj_inj)]
    generalize H.ϕ1, H.ϕ2_C = z
    have hz : z = z.re + z.im * Complex.I := by exact Eq.symm (Complex.ofReal_ofIm)
    generalize z.re = x at hz
    generalize z.im = y at hz
    subst hz
    have h0 : ||x + y * Complex.I|| ^ 2 = x ^ 2 + y ^ 2 := by
      rw [Complex.norm_add_mul_I, Real.sq_sqrt]
      positivity
    rw [h0]
    simp [Complex.add_re, sq]
    ring
  rw [← h1]
  ring
  _ = (||H.ϕ1|| ^ 2 + ||H.ϕ2|| ^ 2 - 2 * (H.ϕ1, H.ϕ2_C).re) ^ 2 := by
  rw [show H.ϕ2, H.ϕ1_C = conj H.ϕ1, H.ϕ2_C from Eq.symm (conj_inj)]
  rw [Complex.conj_re]
  ring

lemma quarticTerm_stabilityCounterExample_eq_norm_pow_four (H : TwoHiggsDoublets) :
  quarticTerm .stabilityCounterExample H = ||H.ϕ1 - H.ϕ2|| ^ 4 := by
/- Proof by calculation. -/
calc
  _ = (||H.ϕ1|| ^ 2 + ||H.ϕ2|| ^ 2 - 2 * (H.ϕ1, H.ϕ2_C).re) ^ 2 := by
  rw [quarticTerm_stabilityCounterExample]
  _ = (||H.ϕ1 - H.ϕ2|| ^ 2) ^ 2 := by
  congr
  have h1 (v : HiggsVec) : ||v|| ^ 2 = (v, v_C).re := by
    rw [inner_self_eq_norm_sq_to_K]
    simp [← Complex.ofReal_pow]
  rw [h1, h1, h1]
  simp only [inner_sub_right, inner_sub_left, Complex.sub_re]
  rw [show H.ϕ2, H.ϕ1_C = conj H.ϕ1, H.ϕ2_C from Eq.symm (conj_inj)]
  rw [Complex.conj_re]
  ring

```

1.34.

PHYSLEAN/PARTICLES/BEYONDTHESTANDARDMODEL/TWOHDM/POTENTIAL.LEAN

```
_ = ||H.ϕ1 - H.ϕ2|| ^ 4 := by ring
```

```
lemma quarticTerm_stabilityCounterExample_nonneg (H : TwoHiggsDoublet) :
  0 ≤ quarticTerm .stabilityCounterExample H := by
  rw [quarticTerm_stabilityCounterExample_eq_norm_pow_four]
  positivity
```

```
lemma massTerm_zero_of_quarticTerm_zero_stabilityCounterExample (H : TwoHiggsDoublet)
  (h : quarticTerm .stabilityCounterExample H = 0) :
  massTerm .stabilityCounterExample H = 0 := by
  rw [quarticTerm_stabilityCounterExample_eq_norm_pow_four] at h
  rw [massTerm_stabilityCounterExample]
  simp at h
  have h1 : H.ϕ1 = H.ϕ2 := by grind
  simp [← Complex.ofReal_pow, h1]
```

```
/-!
```

```
## D. The full potential
```

We define the full potential as the sum of the mass and quartic terms, and prove that it is gauge invariant.

```
-/
```

```
@[simp]
```

```
lemma potential_zero : potential 0 = 0 := by
  ext H
  simp [potential]
```

```
lemma potential_stabilityCounterExample (H : TwoHiggsDoublet) :
  potential .stabilityCounterExample H = 2 * (||H.ϕ1, H.ϕ2||_C).im + ||H.ϕ1 - H.ϕ2|| ^ 4
  simp [potential, massTerm_stabilityCounterExample,
  quarticTerm_stabilityCounterExample_eq_norm_pow_four]
```

```
lemma potential_eq_gramVector (P : PotentialParameters) (H : TwoHiggsDoublet) :
  potential P H = ∑ μ, P.ξ μ * H.gramVector μ +
  ∑ a, ∑ b, H.gramVector a * H.gramVector b * P.η a b := by
  rw [potential, massTerm_eq_gramVector, quarticTerm_eq_gramVector]
```

```
/-!
```

```
## E. Stability of the potential
```

```
-/
```

E.1. The stability condition

We define the condition that the potential is stable, that is, bounded from
 /-- The condition that the potential is stable. -/
 def PotentialIsStable (P : PotentialParameters) : Prop :=
 /-!

E.2. Instability of the stabilityCounterExample potential

-/

open Real

/-- The potential `stabilityCounterExample` is not stable. -
 /

```
lemma stabilityCounterExample_not_potentialIsStable :
  ¬ PotentialIsStable .stabilityCounterExample := by
  simp [PotentialIsStable]
  intro c
  /- The angle t and properties thereof. -/
  let t := Real.arctan (2 * Real.sqrt (|c| + 1))⁻¹
  have t_pos : 0 < t := by
    simp [t]
    grind
  have t_le_pi_div_2 : t ≤ Real.pi / 2 := by
    simp [t] using le_of_lt <| arctan_lt_pi_div_two ((√(|c| + 1))⁻¹ * 2⁻¹)
  have t_ne_zero : t ≠ 0 := by
    simp [t]
    grind
  have sin_t_pos : 0 < sin t := by
    simp [t]
    grind
  have cos_t_pos : 0 < cos t := by
    simp [t]
    exact cos_arctan_pos ((√(|c| + 1))⁻¹ * 2⁻¹)
  have t_mul_sin_t_nonneg : 0 ≤ 2 * t * sin t - t ^ 2 := by
    rw [sub_nonneg]
    trans 2 * t * (2 / Real.pi * t)
    · ring_nf
    · rw [mul_assoc]
    · apply le_mul_of_one_le_right
    · positivity
    · field_simp
    · exact Real.pi_le_four
    · have := Real.mul_le_sin (le_of_lt t_pos) t_le_pi_div_2
```

1.34.

PHYSLEAN/PARTICLES/BEYONDTHESTANDARDMODEL/TWOHDM/POTENTIAL.LEAN

```

nlinarith
/- The Two Higgs doublet violating stability.
The two Higgs doublet is constructed so that for the gram vector
`v` we have:
- `v_0 = cos t / (2 * t * (sin t)^2)`
- `v_1 / v_0 = (1 - t * sin t)`
- `v_2 / v_0 = - t * cos t`
- `v_3 = 0` -/
let H : TwoHiggsDoublet := {
  Φ1 := !₂[√(cos t / (4 * t * (sin t)^2)), 0]
  Φ2 := √(cos t / (4 * t * (sin t)^2)) • !₂[1 - t * sin t - Complex.I * t * cos t,
    √(2 * t * sin t - t ^ 2)] }
have Φ1_norm_sq : ||H.Φ1|| ^ 2 = cos t / (4 * t * (sin t)^2) := by
  simp [H, PiLp.norm_sq_eq_of_L2]
  rw [sq_sqrt]
  positivity
have Φ2_norm_sq : ||H.Φ2|| ^ 2 = cos t / (4 * t * (sin t)^2) := by
  simp [H, norm_smul, mul_pow]
  rw [sq_sqrt (by positivity)]
  simp [PiLp.norm_sq_eq_of_L2]
  rw [sq_sqrt (by positivity)]
have h0 : ||1 - t * Complex.sin t - Complex.I * t * Complex.cos t|| ^ 2 =
  1 + t ^ 2 - 2 * t * sin t := by
  rw [← Complex.normSq_eq_norm_sq]
  trans Complex.normSq (Complex.ofReal (1 - t * sin t) +
    Complex.ofReal (-t * cos t) * Complex.I)
  · simp
    ring_nf
  rw [Complex.normSq_add_mul_I]
  trans 1 + t ^ 2 * (sin t ^ 2 + cos t ^ 2) - 2 * (t * sin t)
  · ring
  rw [sin_sq_add_cos_sq]
  ring
  rw [h0]
  field_simp
  ring
have Φ1_inner_Φ2 : ⟨H.Φ1, H.Φ2⟩_ℂ = Complex.ofReal (cos t / (4 * t * (sin t)^2) *
  (1 - t * sin t)) + Complex.I *
  Complex.ofReal (cos t / (4 * t * (sin t)^2) * (- t * cos t)) := by
  simp [H, PiLp.inner_apply]
  trans Complex.ofReal ((√(cos t / (4 * t * sin t ^ 2))) ^ 2) *
    (1 - t * Complex.sin t - Complex.I * t * Complex.cos t)
  · simp
    ring
  rw [sq_sqrt (by positivity)]
  simp only [Complex.ofReal_div, Complex.ofReal_cos, Complex.ofReal_mul, Complex.ofReal_sin]

```

E.3. The reduced mass term

- /

```
noncomputable def massTermReduced (P : PotentialParameters) (k : EuclideanSpace ℝ (Fin n)) : ℝ :=
  P.ξ (Sum.inl 0) + ∑ μ, P.ξ (Sum.inr μ) * k μ
```

```
lemma massTermReduced_lower_bound (P : PotentialParameters) (k : EuclideanSpace ℝ (fin 3))
  (hk : ||k|| ^ 2 ≤ 1) : P.ξ (Sum.inl 0) - √(∑ a, |P.ξ (Sum.inr a)| ^ 2) ≤
  simp only [Fin.isValue, massTermReduced]
  have h1 (a b c : ℝ) (h : - b ≤ c) : a - b ≤ a + c := by grind
```


1.34.

PHYSLEAN/PARTICLES/BEYONDTHESTANDARDMODEL/TWOHDM/POTENTIAL.LEAN

```

apply h1
let ξEuclid : EuclideanSpace ℝ (Fin 3) := WithLp.toLp 2 (fun a => P.ξ (Sum.inr a))
trans - ||ξEuclid||
· simp [PiLp.norm_eq_of_L2, ξEuclid]
trans - (||k|| * ||ξEuclid||)
· simp
  simp at hk
  have ha (a b : ℝ) (h : a ≤ 1) (ha : 0 ≤ a) (hb : 0 ≤ b) : a * b ≤ b := by nlinarith
  apply ha
  · exact hk
  · exact norm_nonneg k
  · exact norm_nonneg ξEuclid
trans - ||k, ξEuclid||_ℝ
· simp
  exact abs_real_inner_le_norm k ξEuclid
trans k, ξEuclid_ℝ
· simp
  grind
simp [PiLp.inner_apply, ξEuclid]

@[simp]
lemma massTermReduced_zero : massTermReduced 0 = 0 := by
  ext k
  simp [massTermReduced]

lemma massTermReduced_stabilityCounterExample (k : EuclideanSpace ℝ (Fin 3)) :
  massTermReduced .stabilityCounterExample k = k 1 := by
  simp [massTermReduced, PotentialParameters.ξ, Fin.isValue,
    PotentialParameters.stabilityCounterExample, Fin.sum_univ_three]

/-!

### E.4. The reduced quartic term

The reduced quartic term is a function that helps express the stability condition.
It is the function `J4` in https://arxiv.org/abs/hep-ph/0605184.

-/

/-- A function related to the quartic term of the potential, used in the stableness
condition and equivalent to the term `J4` in
https://arxiv.org/abs/hep-ph/0605184. -/
noncomputable def quarticTermReduced (P : PotentialParameters) (k : EuclideanSpace ℝ
  P.η (Sum.inl 0) (Sum.inl 0) + 2 * ∑ b, k b * P.η (Sum.inl 0) (Sum.inr b) +
  ∑ a, ∑ b, k a * k b * P.η (Sum.inr a) (Sum.inr b)

```

```

@[simp]
lemma quarticTermReduced_zero : quarticTermReduced 0 = 0 := by
  ext k
  simp [quarticTermReduced]

lemma quarticTermReduced_stabilityCounterExample (k : EuclideanSpace ℝ (Fin 3)) :
  quarticTermReduced .stabilityCounterExample k = (1 - k 0) ^ 2 := by
  simp [quarticTermReduced, PotentialParameters.η, Fin.isValue,
    PotentialParameters.stabilityCounterExample, Fin.sum_univ_three]
  ring

lemma quarticTermReduced_stabilityCounterExample_nonneg (k : EuclideanSpace ℝ (Fin 3)) :
  0 ≤ quarticTermReduced .stabilityCounterExample k := by
  rw [quarticTermReduced_stabilityCounterExample]
  apply sq_nonneg

/-!

### E.5. Stability in terms of the gram vectors

We give some necessary and sufficient conditions for the potential to be stable
in terms of the gram vectors.

This follows the analysis in https://arxiv.org/abs/hep-ph/0605184.

We also give some necessary conditions.

-/

lemma potentialIsStable_iff_forall_gramVector (P : PotentialParameters) :
  PotentialIsStable P ↔ ∃ c : ℝ, ∀ K : Fin 1 ⊕ Fin 3 → ℝ, 0 ≤ K (Sum.inl 0)
lemma potentialIsStable_iff_forall_euclid (P : PotentialParameters) :
  PotentialIsStable P ↔ ∃ c, ∀ K0 : ℝ, ∀ K : EuclideanSpace ℝ (Fin 3), 0 ≤ K 0
  rw [potentialIsStable_iff_forall_gramVector]
lemma potentialIsStable_iff_forall_euclid_lt (P : PotentialParameters) :
  PotentialIsStable P ↔ ∃ c ≤ 0, ∀ K0 : ℝ, ∀ K : EuclideanSpace ℝ (Fin 3), 0 ≤ K 0
  rw [potentialIsStable_iff_forall_euclid]
lemma potentialIsStable_iff_exists_forall_forall_reduced (P : PotentialParameters) :
  PotentialIsStable P ↔ ∃ c ≤ 0, ∀ K0 : ℝ, ∀ k : EuclideanSpace ℝ (Fin 3), 0 ≤ K 0
  rw [potentialIsStable_iff_forall_euclid_lt]
lemma quarticTermReduced_nonneg_of_potentialIsStable (P : PotentialParameters) :
  (hP : PotentialIsStable P) (k : EuclideanSpace ℝ (Fin 3)) : 0 ≤ quarticTermReduced k
  rw [potentialIsStable_iff_exists_forall_forall_reduced] at hP
lemma potentialIsStable_iff_massTermReduced_sq_le_quarticTermReduced (P : PotentialParameters) :
  PotentialIsStable P ↔ ∃ c, 0 ≤ c ∧ ∀ k : EuclideanSpace ℝ (Fin 3), ||k||^2 ≤ c
  rw [potentialIsStable_iff_exists_forall_forall_reduced]

```

1.34.

PHYSLEAN/PARTICLES/BEYONDTHESTANDARDMODEL/TWOHDM/POTENTIAL.LEAN

```

    refine quarticTermReduced_nonneg_of_potentialIsStable P ?_ k hk
    rw [potentialIsStable_iff_exists_forall_forall_reduced]
lemma massTermReduced_pos_of_quarticTermReduced_zero_potentialIsStable (P : PotentialParameters)
  (hP : PotentialIsStable P) (k : EuclideanSpace ℝ (Fin 3))
  rw [potentialIsStable_iff_massTermReduced_sq_le_quarticTermReduced] at hP
/-!

```

E.6. Strong stability implies stability

Stability in terms of the positivity of the quartic term, implies that the whole potential is stable.

-/

-- The potential is stable if it is strongly stable, i.e. its quartic term is always positive. The proof of this result relies on the compactness of the closed unit ball in $\text{EuclideanSpace } \mathbb{R} (\text{Fin } 3)$, and the 'extreme value theorem'. -

/

```

lemma potentialIsStable_of_strong (P : PotentialParameters)
  (h :  $\forall k, \|k\|^2 \leq 1 \rightarrow 0 < \text{quarticTermReduced } P k$ ) :
  PotentialIsStable P := by
  rw [potentialIsStable_iff_massTermReduced_sq_le_quarticTermReduced]
  let S := Metric.closedBall (0 : EuclideanSpace ℝ (Fin 3)) 1
  have S_isCompact : IsCompact S := isCompact_closedBall 0 1
  have S_nonEmpty : S.Nonempty := {0, by simp [S]}
  obtain (kmax, kmax_S, kmax_isMax) := IsCompact.exists_isMaxOn
    (isCompact_closedBall 0 1) S_nonEmpty
  (f := fun k => (massTermReduced P k ^ 2) / (4 * quarticTermReduced P k)) <| by
  apply ContinuousOn.div
  · apply Continuous.continuousOn
    simp only [massTermReduced, Fin.isValue]
    fun_prop
  · apply Continuous.continuousOn
    simp only [quarticTermReduced, Fin.isValue]
    fun_prop
  · intro x hx
    specialize h x (by simp using hx)
    linarith
  use (massTermReduced P kmax) ^ 2 / (4 * quarticTermReduced P kmax)
  apply And.intro
  · refine (le_div_iff ?_).mpr ?_
    · specialize h kmax (by simp using kmax_S)
      linarith
    · simp only [zero_mul]
      exact sq_nonneg (massTermReduced P kmax)
  · intro k hk

```

```

    apply And.intro
    · specialize h k hk
      linarith
    · intro hq
      rw [isMaxOn_iff] at kmax_isMax
      refine (div_le_iff' ?_).mp (kmax_isMax k (by simp using hk))
      grind

/-!

### E.7. Showing step in hep-ph/0605184 is invalid

-/

/-- A lemma invalidating the step in https://arxiv.org/pdf/hep-ph/0605184 leading to
equation (4.4). -/
lemma forall_reduced_exists_not_potentialIsStable :
   $\exists P, \neg \text{PotentialIsStable } P \wedge (\forall k : \text{EuclideanSpace } \mathbb{R} \text{ (Fin 3)}, \|k\|^2 \leq 0 \leq \text{quarticTermReduced } P \ k \wedge (\text{quarticTermReduced } P \ k = 0 \rightarrow 0 \leq \text{massTermReduced } P \ k))$ 
/- Construction of the explicit counter example.
The reason that this counter example works is that:
- There is a zero of the quartic term `z` on the boundary.
- The quartic term is equal to  $((k - z) \cdot z)^2$ , as `k - z` approaches this becomes small on two accounts: the abs of `k - z` has to become the boundary, and the angle between `k - z` and `z` also becomes small.
- The mass term is of the form  $-(k - z) \cdot w$  for some `w` orthogonal to `z`, approaches orthogonal to `z`, the mass term becomes small only on the boundary where `k - z` becomes small. -/
use .stabilityCounterExample
apply And.intro
/- The condition that P is not stable. -/
· exact stabilityCounterExample_not_potentialIsStable
/- The condition on the reduced terms. -/
· refine fun k hk => And.intro (quarticTermReduced_stabilityCounterExample k)
  (fun hq => ?_)
  simp [quarticTermReduced_stabilityCounterExample] at hq
  simp only [PiLp.norm_sq_eq_of_L2, Real.norm_eq_abs, sq_abs, Fin.sum_uni_val,
    Fin.isValue] at hk
  have hk1 : k 1 = 0 := by nlinarith
  rw [massTermReduced_stabilityCounterExample, hk1]

```

1.35.

PHYSLean/PARTICLES/FLAVORPHYSICS/CKMMATRIX/PHASEFREEDOM31LEAN

1.35 PhysLean/Particles/FlavorPhysics/CKMMatrix/PhaseFreedom.lean

```
have h1:= (ud_us_ne_zero_iff_ub_ne_one V).mpr.mt hb
have hx := Vabs_sq_add_ne_zero hb
have hx := Vabs_sq_add_ne_zero hb
```

1.36 PhysLean/Particles/FlavorPhysics/CKMMatrix/Relations.lean

```
lemma ud_us_ne_zero_iff_ub_ne_one (V : CKMMatrix) :
lemma normSq_Vud_plus_normSq_Vus_ne_zero_ℝ {V : CKMMatrix} (hb : [V]ud ≠ 0 v [V]us ≠ 0) :
  rw [ud_us_ne_zero_iff_ub_ne_one] at hb
lemma VAbsub_ne_zero_Vud_Vus_ne_zero {V : Quotient CKMMatrixSetoid}
  rw [← ud_us_ne_zero_iff_ub_ne_one V] at hV
  simp [← Complex.sq_norm] using (normSq_Vud_plus_normSq_Vus_ne_zero_ℝ hV)
lemma VAbsub_ne_zero_sqrt_Vud_Vus_ne_zero {V : Quotient CKMMatrixSetoid}
  rw [← ud_us_ne_zero_iff_ub_ne_one V] at hV
  simp [← Complex.sq_norm] using (normSq_Vud_plus_normSq_Vus_ne_zero_ℝ hV)
lemma normSq_Vud_plus_normSq_Vus_ne_zero_ℂ {V : CKMMatrix} (hb : [V]ud ≠ 0 v [V]us ≠ 0) :
  have h1 := normSq_Vud_plus_normSq_Vus_ne_zero_ℝ hb
lemma Vabs_sq_add_ne_zero {V : CKMMatrix} (hb : [V]ud ≠ 0 v [V]us ≠ 0) :
  have h1 := normSq_Vud_plus_normSq_Vus_ne_zero_ℂ hb
  have h1 := normSq_Vud_plus_normSq_Vus_ne_zero_ℂ h
  have h1 := normSq_Vud_plus_normSq_Vus_ne_zero_ℂ h
lemma cb_tb_ne_zero_iff_ub_ne_one (V : CKMMatrix) :
```

1.37 PhysLean/Particles/FlavorPhysics/CKMMatrix/StandardParameterizati

```
lemma S23_of_Vub_ne_one {V : Quotient CKMMatrixSetoid} (ha : VubAbs V ≠ 1) :
  exact VAbsub_ne_zero_Vud_Vus_ne_zero ha
lemma C23_of_Vub_ne_one {V : Quotient CKMMatrixSetoid} (ha : VubAbs V ≠ 1) :
  rw [C23, θ23, Real.cos_arcsin, S23_of_Vub_ne_one ha, div_pow, Real.sq_sqrt]
  exact VAbsub_ne_zero_Vud_Vus_ne_zero ha
  exact (mul_div_cancel₀ (VudAbs V) (VAbsub_ne_zero_sqrt_Vud_Vus_ne_zero ha)).symm
  • have h2 := VAbsub_ne_zero_sqrt_Vud_Vus_ne_zero ha
  • rw [S23_of_Vub_ne_one ha, C13_eq_add_sq, mul_comm]
    exact (mul_div_cancel₀ (VcbAbs V) (VAbsub_ne_zero_sqrt_Vud_Vus_ne_zero ha)).symm
  • rw [C23_of_Vub_ne_one ha, C13_eq_add_sq, mul_comm]
    exact (mul_div_cancel₀ (VtbAbs V) (VAbsub_ne_zero_sqrt_Vud_Vus_ne_zero ha)).symm
lemma mulExpδ13_on_param_ne_zero_arg (V : CKMMatrix) (δ13 : ℝ)
  have habs_ne_zero :
  rw [← mul_right_inj' habs_ne_zero]
  rw [ud_us_ne_zero_iff_ub_ne_one] at hb
  have hx := Vabs_sq_add_ne_zero hb
```

```

rw [S23_eq_Csin_θ23 [[V]], S23_of_Vub_ne_one hb', C23_eq_Ccos_θ23 [[V]],
    C23_of_Vub_ne_one hb', S13_eq_Csin_θ13 [[V]], S13]
rw [C12_eq_Vud_div_sqrt hb', C23_of_Vub_ne_one hb', S12, S13, S23_of_
    (δ13 [[V]])] (by rw [← mulExpδ13_on_param_ne_zero_arg V δ13' h,

```

1.38 PhysLean/Particles/StandardModel/AnomalyCancellation/Basic

```
import Mathlib.Tactic.LinearCombination
```

1.39 PhysLean/Particles/StandardModel/AnomalyCancellation/NoGrav

```

lemma accGrav_Q_ne_zero {S : (SMNoGrav 1).Sols} (hQ : Q S.val (0 : Fin 1) ≠ 0) :
  · exact accGrav_Q_ne_zero hQ

```

1.40 PhysLean/Particles/StandardModel/HiggsBoson/Basic.lean

```
import Mathlib.Geometry.Manifold.VectorBundle.SmoothSection
```

1.41 PhysLean/Particles/SuperSymmetry/MSSMnu/AnomalyCancellation

```
import Mathlib.Tactic.LinearCombination
```

1.42 PhysLean/Particles/SuperSymmetry/MSSMnu/AnomalyCancellation

```
import Mathlib.Tactic.LinearCombination
```

1.43 PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/Basic

```
import Mathlib.Data.Finset.Option
```

1.44 PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/Complex

```

have Q10_ne_zero : Q10 ≠ 0 := by
simp [Q10_ne_zero]

```

1.45.

PHYSLEAN/PARTICLES/SUPERSYMMETRY/SU5/CHARGESPECTRUM/MINIMALSUPERSET.LEAN

1.45 PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/MinimalSuperS

```
lemma self_ne_mem_minimalSuperSet (S5 S10 : Finset  $\square$ ) (x y : ChargeSpectrum  $\square$ )
```

1.46 PhysLean/QFT/AnomalyCancellation/Basic.lean

```
Function.Injective  $\chi$ .linSolsIncl := fun _ _ h => LinSols.ext h
```

1.47 PhysLean/QFT/PerturbationTheory/CreateAnnihilate.lean

```
instance : Std.Total normalOrder where
```

1.48 PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/SuperCommute.l

```
lemma statistic_ne_of_superCommuteF_fermionic { $\phi$ s  $\phi$ s' : List  $\square$ .CrAnFieldOp}
```

1.49 PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/TimeOrder.lean

```
have hx := Std.Total.total (r := timeOrderRel)  $\psi$   $\phi$   
have h1 := Std.Total.total (r := crAnTimeOrderRel)  $\phi$   $\psi$   
  Fin.val_cast  
simp only [Fin.lt_def, Fin.val_cast]
```

1.50 PhysLean/QFT/PerturbationTheory/FieldSpecification/NormalOrder.le

```
instance : Std.Total  $\square$ .normalOrderRel where
```

1.51 PhysLean/QFT/PerturbationTheory/FieldSpecification/TimeOrder.lean

```
instance : Std.Total  $\square$ .timeOrderRel where  
  simp only [maxTimeFieldPos, List.length_cons, Order.lt_add_one_iff]  
  exact Fin.is_le (insertionSortMinPos timeOrderRel  $\phi$   $\phi$ s)  
instance : Std.Total  $\square$ .crAnTimeOrderRel where  
  total a b := Std.Total.total (r :=  $\square$ .timeOrderRel) a.1 b.1  
  exact (Std.Total.to_refl (r :=  $\square$ .crAnTimeOrderRel)).refl  $\phi$   
instance : Std.Total  $\square$ .normTimeOrderRel where  
  match Std.Total.total (r :=  $\square$ .crAnTimeOrderRel) a b,  
    Std.Total.total (r :=  $\square$ .normalOrderRel) a b with
```

1.52 PhysLean/QFT/PerturbationTheory/FieldStatistics/Basic.lean

```
lemma bosonic_ne_iff_fermionic_eq (a : FieldStatistic) : ¬ bosonic = a ↔ fermionic = a
lemma fermionic_ne_iff_bosonic_eq (a : FieldStatistic) : ¬ fermionic = a ↔ bosonic = a
```

1.53 PhysLean/QFT/PerturbationTheory/FieldStatistics/OfFinset.lean

```
exact Finset.filter_union_filter_not_eq p a
exact Finset.disjoint_filter_filter_not a a p
```

1.54 PhysLean/QFT/PerturbationTheory/Koszul/KoszulSign.lean

```
lemma koszulSign_insertIdx [Std.Total le] [IsTrans [] le] (φ : []) :
  simp only [Fin.eta, Fin.val_cast]
  · simp only [hn, ↓reduceIte, Fin.val_castSucc]
lemma koszulSign_eraseIdx [Std.Total le] [IsTrans [] le] (φs : List []) (n : ℕ) :
  Equiv.coe_fn_mk, Fin.cast_mk, Fin.eta, Fin.val_cast]
lemma koszulSign_eraseIdx_insertionSortMinPos [Std.Total le] [IsTrans [] le]
lemma koszulSign_of_insertionSort [Std.Total le] [IsTrans [] le] (φs : List [])
lemma koszulSign_of_append_eq_insertionSort_left [Std.Total le] [IsTrans [] le]
  Fin.val_cast]
  simp only [finCongr_apply, Fin.val_cast]
lemma koszulSign_of_append_eq_insertionSort [Std.Total le] [IsTrans [] le] :
```

1.55 PhysLean/QFT/PerturbationTheory/Koszul/KoszulSignInsert.lean

```
lemma koszulSignInsert_eq_cons [Std.Total le] (φ : []) (φs : List []) :
  simp only [or_self] using Std.Total.total (r := le) φ φ
lemma koszulSignInsert_eq_exchangeSign_take [Std.Total le] [IsTrans [] le] (
  simp only [decide_not, (Std.Total.to_refl le).refl φ, not_true_eq_false,
```

1.56 PhysLean/QFT/PerturbationTheory/WickAlgebra/Basic.lean

```
rcases statistic_ne_of_superCommuteF_fermionic h with h | h
```


1.57.

PHYSLEAN/QFT/PERTURBATIONTHEORY/WICKALGEBRA/NORMALORDER/BASIC.LEAN

1.57 PhysLean/QFT/PerturbationTheory/WickAlgebra/NormalOrder/Basic.lean

```
rw [← sub_eq_zero, ← map_sub, ← LinearMap.map_sub]
```

1.58 PhysLean/QFT/PerturbationTheory/WickAlgebra/NormalOrder/Lemmas.lean

```
simp only [instCommGroup.eq_1, Fin.val_cast, Fin.getElem_fin,
```

1.59 PhysLean/QFT/PerturbationTheory/WickAlgebra/NormalOrder/WickContr

```
((φsΛ.uncontractedListOrderPos i), by simp [uncontractedListGet]),  
smul_smul]  
  simp only [Fin.val_castSucc] at h  
  simp only [Fin.val_castSucc, not_lt] at h'  
Fin.val_cast, uncontractedListGet]
```

1.60 PhysLean/QFT/PerturbationTheory/WickAlgebra/StaticWickTerm.lean

```
instCommGroup.eq_1, Fin.val_cast, Fin.getElem_fin, smul_eq_zero]
```

1.61 PhysLean/QFT/PerturbationTheory/WickAlgebra/SuperCommute.lean

```
rw [← sub_eq_zero, ← map_sub, ← LinearMap.map_sub]
```

1.62 PhysLean/QFT/PerturbationTheory/WickAlgebra/TimeContraction.lean

```
have h1 := Std.Total.total (r := [].timeOrderRel) φ ψ  
  have h1 := Std.Total.total (r := [].timeOrderRel) φ ψ  
  have ht := Std.Total.total (r := [].timeOrderRel) φ ψ  
  have ht : timeOrderRel φ ψ v timeOrderRel ψ φ := Std.Total.total (r := [].timeO  
lemma timeOrder_timeContract_ne_time {φ ψ : [].FieldOp}  
  apply timeOrder_superCommute_ne_time  
  apply timeOrder_superCommute_ne_time  
  apply timeOrder_superCommute_ne_time  
  apply timeOrder_superCommute_ne_time
```

1.63 PhysLean/QFT/PerturbationTheory/WickAlgebra/TimeOrder.lean

```

lemma ι_timeOrderF_superCommuteF_ne_time {φ ψ : [].CrAnFieldOp}
  · rw [ι_timeOrderF_superCommuteF_ne_time heqt]
  · rw [ι_timeOrderF_superCommuteF_ne_time heqt]
  · rw [ι_timeOrderF_superCommuteF_ne_time heqt]
  rw [← sub_eq_zero, ← map_sub, ← LinearMap.map_sub]
lemma timeOrder_superCommute_ne_time {φ ψ : [].CrAnFieldOp}
  rw [ι_timeOrderF_superCommuteF_ne_time]
lemma timeOrder_superCommute_anPart_ofFieldOp_ne_time {φ ψ : [].FieldOp}
  apply timeOrder_superCommute_ne_time
  apply timeOrder_superCommute_ne_time

```

1.64 PhysLean/QFT/PerturbationTheory/WickAlgebra/WickTerm.lean

```

Equiv.coe_trans, Option.map_some, Function.comp_apply, finCongr_app

```

1.65 PhysLean/QFT/PerturbationTheory/WickContraction/Basic.lean

```

lemma self_ne_getDual?_get (i : Fin n) (h : (c.getDual? i).isSome) :
lemma fstFieldOfContract_ne_sndFieldOfContract (c : WickContraction n) (a :
  (c.fstFieldOfContract_ne_sndFieldOfContract a)
  · exact Ne.symm <| fstFieldOfContract_ne_sndFieldOfContract c a
  exact Ne.symm <| fstFieldOfContract_ne_sndFieldOfContract c a

```

1.66 PhysLean/QFT/PerturbationTheory/WickContraction/ExtractEqu

```

decide +kernel
decide +kernel

```

1.67 PhysLean/QFT/PerturbationTheory/WickContraction/InsertAndC

```

simp_all only [Nat.succ_eq_add_one, Fin.val_fin_lt, not_lt, finCongr_
simp_all only [Nat.succ_eq_add_one, Fin.val_fin_lt, not_lt, finCongr_
simp only [Fin.val_cast, ne_eq]
  simp only [Fin.ext_iff, Fin.val_cast] at hx

```

1.68 PhysLean/QFT/PerturbationTheory/WickContraction/InsertAndC

```

lemma insertAndContractNat_some_getDual?_ne_none (c : WickContraction n) (i

```

1.69

PHYSLean/QFT/PERTURBATIONTHEORY/WICKCONTRACTION/JOIN.LEAN

```
lemma insertAndContractNat_some_getDual?_ne_isSome (c : WickContraction n) (i : Fin n)
  simp [hkj, insertAndContractNat_some_getDual?_ne_none]
lemma insertAndContractNat_some_getDual?_ne_isSome_get (c : WickContraction n) (i : Fin n)
  (by simp [hkj, insertAndContractNat_some_getDual?_ne_isSome] using h)) := by
  (by simp [hkj, insertAndContractNat_some_getDual?_ne_isSome] using h))) := by
  (by simp [hkj, insertAndContractNat_some_getDual?_ne_isSome] using h)))
  rw [← insertAndContractNat_some_getDual?_ne_isSome_get c i j k hkj]
  simp [hkj, insertAndContractNat_some_getDual?_ne_isSome] using h
simp only [ne_eq, hkj, not_false_eq_true, insertAndContractNat_some_getDual?_ne_isSome]
simp only [Nat.succ_eq_add_one, ne_eq, self_ne_getDual?_get, not_false_eq_true]
```

1.69 PhysLean/QFT/PerturbationTheory/WickContraction/Join.lean

```
lemma jointLiftLeft_ne_joinLiftRight {φs : List [].FieldOp} {φsΛ : WickContraction φs}
  have h1 := jointLiftLeft_ne_joinLiftRight a b
  have h1 := jointLiftLeft_ne_joinLiftRight b a
```

1.70 PhysLean/QFT/PerturbationTheory/WickContraction/Sign/InsertNone.lean

```
Fin.val_cast, or_iff_right_iff_imp]
simp only [Fin.val_cast, Fin.val_fin_lt]
simp only [Fin.val_cast, Fin.val_fin_lt]
finCongr_apply, Fin.getElem_fin, Fin.val_cast, insertIdx_getElem_fin,
simp only [instCommGroup, Nat.succ_eq_add_one, finCongr_apply, Fin.getElem_fin,
```

1.71 PhysLean/QFT/PerturbationTheory/WickContraction/Sign/InsertSome.lean

```
simp only [Fin.val_cast, Fin.val_fin_lt, and_congr_right_iff]
simp only [Fin.val_cast, Fin.val_fin_lt, Nat.succ_eq_add_one, finCongr_apply,
simp only [Fin.val_cast, Fin.val_fin_lt]
simp only [Fin.ext_iff, Fin.val_cast]
simp only [Fin.ext_iff, Fin.val_cast] at h
simp only [Fin.ext_iff, Fin.val_cast]
simp only [Fin.val_cast, Fin.val_fin_lt]
simp only [Fin.val_cast, Option.get_map, Function.comp_apply, Fin.val_fin_lt]
finCongr_apply, Fin.getElem_fin, Fin.val_cast, insertIdx_getElem_fin,
simp only [instCommGroup, Fin.getElem_fin, Fin.val_cast, List.getElem_insertIdx]
simp only [instCommGroup, Fin.getElem_fin, Fin.val_cast, insertIdx_getElem_fin]
simp only [Fin.ext_iff, Fin.val_cast]
simp only [Fin.ext_iff, Fin.val_cast]
simp only [Fin.val_cast, Fin.val_fin_lt]
simp only [Fin.ext_iff, Fin.val_cast] at hy2
```

```

simp only [Fin.ext_iff, Fin.val_cast]
  simp only [Fin.ext_iff, Fin.val_cast]
  simp only [Fin.val_cast, Fin.val_fin_lt]
    simp only [Fin.val_cast, Fin.val_fin_lt] at h2
  simp only [Fin.ext_iff, Fin.val_cast] at hy2
    simp only [Fin.lt_def, Fin.val_cast, gt_iff_lt]
  simp only [Option.get_map, Function.comp_apply, Fin.val_cast, F

```

1.72 PhysLean/QFT/PerturbationTheory/WickContraction/Sign/Join.

```

have hjneqfst := singleton_uncontractedEmd_ne_right h (φsucΛ.fstFieldOfCo
have hjneqsnd := singleton_uncontractedEmd_ne_right h (φsucΛ.sndFieldOfCo
have hineqfst := singleton_uncontractedEmd_ne_left h (φsucΛ.fstFieldOfCon
have hineqsnd := singleton_uncontractedEmd_ne_left h (φsucΛ.sndFieldOfCon
have hjneqsnd := singleton_uncontractedEmd_ne_right h (φsucΛ.sndFieldOfCo

```

1.73 PhysLean/QFT/PerturbationTheory/WickContraction/Singleton.

```

lemma singleton_uncontractedEmd_ne_left {φs : List [].FieldOp} {i j : Fin φs
lemma singleton_uncontractedEmd_ne_right {φs : List [].FieldOp} {i j : Fin φ

```

1.74 PhysLean/QFT/PerturbationTheory/WickContraction/StaticCont

```

Equiv.coe_trans, Option.map_some, Function.comp_apply, finCongr_apply,

```

1.75 PhysLean/QFT/PerturbationTheory/WickContraction/TimeCond.1

```

rw [timeOrder_timeContract_ne_time]
rw [timeOrder_superCommute_anPart_ofFieldOp_ne_time]

```

1.76 PhysLean/QFT/PerturbationTheory/WickContraction/TimeContra

```

Equiv.coe_trans, Option.map_some, Function.comp_apply, finCongr_apply,
Equiv.coe_trans, Option.map_some, Function.comp_apply, finCongr_apply,
have ht := Std.Total.total (r := timeOrderRel) φs[k.1] φ

```

1.77.

PHYSLEAN/QFT/PERTURBATIONTHEORY/WICKCONTRACTION/UNCONTRACTEDLIST.LEAN

1.77 PhysLean/QFT/PerturbationTheory/WickContraction/UncontractedList.lean

```
Equiv.coe_fn_mk, Fin.val_cast, Function.Embedding.coe_subtype]
lemma uncontractedListOrderPos_le_length (c : WickContraction n) (i : Fin n.succ) :
  c.uncontractedListOrderPos i ≤ c.uncontractedList.length := by
  · simp only [Fin.lt_def, Fin.val_castSucc]
    simp_all only [Fin.lt_def, Fin.val_castSucc, not_lt, Fin.val_succ]
```

1.78 PhysLean/QFT/QED/AnomalyCancellation/Basic.lean

```
import Mathlib.Tactic.LinearCombination
```

1.79 PhysLean/QFT/QED/AnomalyCancellation/BasisLinear.lean

```
simp only [Fin.val_castSucc, Fin.val_last] at h1 h2
simp only [Nat.succ_eq_add_one, PureU1_numberCharges, Equiv.invFun_as_coe,
  Finsupp.equivFunOnFinite_symm_apply_apply, Function.comp_apply]
simp only [Nat.succ_eq_add_one, Finsupp.equivFunOnFinite_symm_apply_apply, Funct
```

1.80 PhysLean/QFT/QED/AnomalyCancellation/ConstAbs.lean

```
simp only [PureU1_numberCharges, Fin.val_cast, Fin.val_castAdd, mul_eq, Fin.val_ca
simp only [mul_eq, Fin.val_succ, PureU1_numberCharges, Fin.val_cast, Fin.val_natAd
```

1.81 PhysLean/QFT/QED/AnomalyCancellation/Even/BasisLinear.lean

```
simp only [succ_eq_add_one, evenSnd, Fin.ext_iff, Fin.val_cast, Fin.val_natAdd
simp only [succ_eq_add_one, evenShiftLast, Fin.isValue, Fin.val_cast, Fin.val_natA
simp only [Fin.val_cast, Fin.val_natAdd, Fin.val_castAdd, Fin.val_succ]
simp only [Fin.val_cast, Fin.val_natAdd, Fin.val_castAdd, Fin.val_castSucc]
  simp only [Fin.val_castAdd, Fin.val_addNat] at h2
  simp only [Fin.val_castAdd, Fin.val_addNat] at h2
simp_all only [Fin.natAdd_eq_addNat, Fin.cast_inj, Fin.val_cast, Fin.val_natAdd,
  Fin.val_castAdd, add_right_inj, Fin.val_addNat, add_eq_left]
simp_all only [Fin.val_natAdd, Fin.val_castAdd, Fin.val_addNat, not_true_eq_fals
Fin.val_cast, Fin.val_castAdd, Fin.val_eq_zero, Fin.val_natAdd] at h
  Fin.val_cast, Fin.val_castAdd, Fin.val_eq_zero, Fin.val_natAdd] at h2
simp only [succ_eq_add_one, evenShiftLast, Fin.isValue, Fin.val_cast, Fin.val_nat
  Fin.val_eq_zero, add_zero, evenShiftFst, Fin.val_castAdd, add_right_inj] at h
  simp only [succ_eq_add_one, evenShiftLast, Fin.isValue, Fin.val_cast, Fin.val_
    Fin.val_eq_zero, add_zero, evenShiftSnd, Fin.val_castAdd, add_right_inj] at
```

1.82 PhysLean/QFT/QED/AnomalyCancellation/Even/LineInCubic.lean

```
Fin.val_cast, Fin.val_natAdd, Fin.val_castAdd, Fin.val_last, Fin.val_eq
```

1.83 PhysLean/QFT/QED/AnomalyCancellation/LineInPlaneCond.lean

```
Fin.val_castSucc]
(by simp only [Nat.succ_eq_add_one, ne_eq, Fin.ext_iff, Fin.val_castS
```

1.84 PhysLean/QFT/QED/AnomalyCancellation/Odd/BasisLinear.lean

```
simp only [succ_eq_add_one, oddShiftShiftFst, Fin.val_cast, Fin.val_castA
simp only [succ_eq_add_one, oddShiftShiftMid, Fin.isValue, Fin.val_cast,
  Fin.val_natAdd, Fin.val_eq_zero, add_zero, oddMid]
simp only [succ_eq_add_one, oddShiftShiftSnd, Fin.val_cast, Fin.val_natAdd
simp only [oddSnd, Fin.val_cast, Fin.val_natAdd, oddShiftSnd, add_left_in
  simp only [Fin.val_castAdd, Fin.val_natAdd] at h2
simp_all only [Fin.cast_inj, Fin.val_cast, Fin.val_castAdd, Fin.val_nat
simp_all only [Fin.val_natAdd, Fin.val_castAdd, add_eq_right,
simp only [oddMid, Fin.isValue, Fin.val_cast, Fin.val_castAdd, Fin.val_
  simp only [oddMid, Fin.isValue, Fin.val_cast, Fin.val_castAdd, Fin.va
  simp only [Fin.val_castAdd, Fin.val_natAdd] at h2
simp only [oddShiftZero, Fin.isValue, Fin.val_cast, Fin.val_castAdd, Fi
  oddShiftFst, Fin.val_natAdd] at h
  simp only [oddShiftZero, Fin.isValue, Fin.val_cast, Fin.val_castAdd,
    oddShiftSnd, Fin.val_natAdd] at h2
```

1.85 PhysLean/QuantumMechanics/DDimensions/Hydrogen/Basic.lean

```
/-
Copyright (c) 2026 Gregory J. Loges. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Gregory J. Loges
-/
import PhysLean.QuantumMechanics.DDimensions.Operators.Momentum
import PhysLean.QuantumMechanics.DDimensions.Operators.Position
/-!

# Hydrogen atom
```

1.85.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/HYDROGEN/BASIC.LEAN

This module introduces the d -dimensional hydrogen atom with $1/r$ potential.

In addition to the dimension d , the quantum mechanical system is characterized by a mass $m > 0$ and constant k appearing in the potential $V = -k/r$.

The standard hydrogen atom has $d=3$, $m = m_e m_p / (m_e + m_p) \approx m_e$ and $k = e^2 / 4\pi\epsilon_0$.

The potential $V = -k/r$ is singular at the origin. To address this we define a regularized Hamiltonian in which the potential is replaced by $-k(r(\epsilon)^{-1} + \frac{1}{2}\epsilon^2 r(\epsilon)^{-3})$, where $r(\epsilon)^2 = \|x\|^2 + \epsilon^2$. The ϵ^2/r^3 term limits to the zero distribution for $\epsilon \rightarrow 0$ but is convenient to include for two reasons:

- It improves the convergence: $r(\epsilon)^{-1} + \frac{1}{2}\epsilon^2 r(\epsilon)^{-3} = r^{-1}(1 + O(\epsilon^4/r^4))$ with no $O(\epsilon)$
- It is what appears in the commutators of the (regularized) LRL vector components.

-/

```
namespace QuantumMechanics
open SchwartzMap
```

```
/-- A hydrogen atom is characterized by the number of spatial dimensions `d`,
    the mass `m` and the coefficient `k` for the `1/r` potential. -
```

```
/
```

```
structure HydrogenAtom where
```

```
  /-- Number of spatial dimensions -/
```

```
  d : ℕ
```

```
  /-- Mass (positive) -/
```

```
  m : ℝ
```

```
  hm : 0 < m
```

```
  /-- Coefficient in potential (positive for attractive) -
```

```
/
```

```
  k : ℝ
```

```
namespace HydrogenAtom
noncomputable section
```

```
variable (H : HydrogenAtom)
```

```
@[simp]
```

```
lemma m_ne_zero : H.m ≠ 0 := by linarith [H.hm]
```

```
/-- The hydrogen atom Hamiltonian regularized by  $\epsilon > 0$  is defined to be
     $\square(\epsilon) = (2m)^{-1}\square^2 - k(\square(\epsilon)^{-1} + \frac{1}{2}\epsilon^2\square(\epsilon)^{-3})$ . -/
```

```
def hamiltonianReg (ε : ℝ) :  $\square(\text{Space } H.d, \mathbb{C}) \rightarrow L[\mathbb{C}] \square(\text{Space } H.d, \mathbb{C}) :=$ 
```

```

    (2 * H.m)-1 •  $\square^2$  - H.k • ( $\square[\varepsilon, -1]$  + (2-1 *  $\varepsilon^2$ ) •  $\square[\varepsilon, -3]$ )

end
end HydrogenAtom
end QuantumMechanics

```

1.86 PhysLean/QuantumMechanics/DDimensions/Hydrogen/LaplaceRunge

```

/-
Copyright (c) 2026 Gregory J. Loges. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Gregory J. Loges
-/
import PhysLean.QuantumMechanics.DDimensions.Hydrogen.Basic
import PhysLean.QuantumMechanics.DDimensions.Operators.Commutation
/-!

# Laplace-Runge-Lenz vector

In this file we define
- The (regularized) LRL vector operator for the quantum mechanical hydrogen
   $\square(\varepsilon)_i = \frac{1}{2}(\square_j \square_{ij} + \square_{ij} \square_j) - mk \cdot \square(\varepsilon)^{-1} \square_i$ .
- The square of the LRL vector operator,  $\square(\varepsilon)^2 = \square(\varepsilon)_i \square(\varepsilon)_i$ .

The main results are
- The commutators  $\{\square_{ij}, \square(\varepsilon)_k\} = i\hbar(\delta_{ik}\square(\varepsilon)_j - \delta_{jk}\square(\varepsilon)_i)$  in `angularMomen
- The commutators  $\{\square(\varepsilon)_i, \square(\varepsilon)_j\} = -i\hbar \frac{2m}{\square(\varepsilon)} \square_{ij}$  in `lrl_commutation_lrl
- The commutators  $\{\square(\varepsilon), \square(\varepsilon)_i\} = i\hbar \varepsilon^2(\dots)$  in `hamiltonianReg_commutation_
- The relation  $\square(\varepsilon)^2 = 2m \square(\varepsilon)(\square^2 + \frac{1}{4}\hbar^2(d-1)^2) + m^2 k^2 + \varepsilon^2(\dots)$  in `lrlOper

-/

namespace QuantumMechanics
namespace HydrogenAtom
noncomputable section
open Constants
open KroneckerDelta
open ContinuousLinearMap SchwartzMap

variable (H : HydrogenAtom)

/-- The (regularized) Laplace-Runge-Lenz vector operator for the `d`-
dimensional hydrogen atom,

```


1.86.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/HYDROGEN/LAPLACERUNGELENZVECTOR.LEAN

```

  `(ε)i = ½(∑j ∑j + ∑j ∑j) - mk·(ε)-1∑i` - /
def lrlOperator (ε : ℝ) (i : Fin H.d) : (Space H.d, ℂ) → L[ℂ] (Space H.d, ℂ) :=
  (2 : ℝ)-1 • ∑ j, (∑ [j] •L ∑ [i,j] + ∑ [i,j] •L ∑ [j]) - (H.m * H.k) • ∑ [ε, -
1] •L ∑ [i]

/-- The square of the LRL vector operator, `(ε)2 = (ε)i(ε)i` -
/
def lrlOperatorSqr (ε : ℝ) : (Space H.d, ℂ) → L[ℂ] (Space H.d, ℂ) :=
  ∑ i, (H.lrlOperator ε i) •L (H.lrlOperator ε i)

/-- `(ε)i = ∑j ∑j - (∑j ∑j)∑i + ½iħ(d-1)∑i - mk·(ε)-1∑i` -
/
lemma lrlOperator_eq (i : Fin H.d) :
  H.lrlOperator ε i = ∑ [i] •L ∑2 - (∑ j, ∑ [j] •L ∑ [j]) •L ∑ [i]
  + (2-1 * Complex.I * ħ * (H.d - 1)) • ∑ [i] - (H.m * H.k) • ∑ [ε, -
1] •L ∑ [i] := by
  unfold lrlOperator angularMomentumOperator
  congr
  conv_lhs =>
    enter [2, 2, j]
    rw [comp_sub, sub_comp]
    repeat rw [← comp_assoc, momentum_comp_position_eq, sub_comp, smul_comp, id_comp]
    repeat rw [comp_assoc]
    rw [momentum_comp_commute i j]

  simp only [Finset.sum_add_distrib, Finset.sum_sub_distrib]
  rw [← ContinuousLinearMap.comp_finset_sum]
  simp only [← comp_assoc, ← ContinuousLinearMap.finset_sum_comp]
  rw [← momentumOperatorSqr]
  unfold kroneckerDelta
  simp only [mul_ite_zero, ite_zero_smul, Finset.sum_ite_eq, Finset.mem_univ, ↑reduc
    Finset.sum_const, Finset.card_univ, Fintype.card_fin, ← smul_assoc]
  ext ψ x
  simp only [mul_one, nsmul_eq_mul, smul_add, ContinuousLinearMap.add_apply, coe_smu
    coe_comp', Pi.smul_apply, Pi.sub_apply, Function.comp_apply, SchwartzMap.add_app
    SchwartzMap.smul_apply, SchwartzMap.sub_apply, smul_eq_mul, Complex.real_smul,
    Complex.ofReal_inv, Complex.ofReal_ofNat]
  ring

  /- `(ε)i = ∑j ∑j + ½iħ(d-1)∑i - mk·(ε)-1∑i` - /
lemma lrlOperator_eq' (i : Fin H.d) : H.lrlOperator ε i = ∑ j, ∑ [i,j] •L ∑ [j]
  + (2-1 * Complex.I * ħ * (H.d - 1)) • ∑ [i] - (H.m * H.k) • ∑ [ε, -
1] •L ∑ [i] := by
  unfold lrlOperator
  congr
  conv_lhs =>

```

```

    enter [2, 2, j]
    rw [comp_eq_comp_sub_commute [][j] [][i,j], angularMomentum_commutation_m
repeat rw [Finset.sum_add_distrib, Finset.sum_sub_distrib, Finset.sum_sub
unfold kroneckerDelta
ext ψ x
simp only [ContinuousLinearMap.add_apply, coe_smul', coe_sum', coe_comp',
    Finset.sum_apply, Function.comp_apply, coe_sub', Pi.sub_apply, Schwartz
    SchwartzMap.smul_apply, SchwartzMap.sum_apply, SchwartzMap.sub_apply]
simp only [mul_ite, mul_one, mul_zero, smul_eq_mul, ite_mul, zero_mul, Fi
    Finset.mem_univ, ↓reduceIte, Finset.sum_const, Finset.card_univ, Fintyp
    nsmul_eq_mul, smul_add, Complex.real_smul, Complex.ofReal_inv, Complex.
ring

/--  $\epsilon_i = \sum_j \epsilon_{ij} - \frac{1}{2} i \hbar (d-1) \epsilon_i - m_k \epsilon_i^{-1} \epsilon_i$  -/
lemma lrlOperator_eq'' (i : Fin H.d) : H.lrlOperator ε i = ∑ j, [][j] • L [][i
    - (2-1 * Complex.I * ħ * (H.d - 1)) • [][i] - (H.m * H.k) • [][ε,-
1] • L [][i] := by
  unfold lrlOperator
  congr
  conv_lhs =>
    enter [2, 2, j]
    rw [comp_eq_comp_add_commute [][i,j] [][j], angularMomentum_commutation_m
  rw [Finset.sum_add_distrib, Finset.sum_add_distrib, Finset.sum_sub_distrib
  ext ψ x
  unfold kroneckerDelta
  simp only [ContinuousLinearMap.add_apply, coe_smul', coe_sum', coe_comp',
    Pi.smul_apply, Finset.sum_apply, Function.comp_apply, coe_sub', Pi.sub_
    SchwartzMap.add_apply, SchwartzMap.smul_apply, SchwartzMap.sum_apply, C
    Complex.ofReal_inv, Complex.ofReal_ofNat, SchwartzMap.sub_apply]
  simp only [mul_ite, mul_one, mul_zero, smul_eq_mul, ite_mul, zero_mul, Fi
    Finset.mem_univ, ↓reduceIte, Finset.sum_const, Finset.card_univ, Fintyp
    nsmul_eq_mul]
  ring

/-- The operator  $\epsilon_i \epsilon_i$  on Schwartz maps. -/
private def positionDotMomentum {d} := ∑ i : Fin d, [][i] • L [][i]

/-- The operator  $\epsilon_i \epsilon_i^2$  on Schwartz maps. -/
private def positionCompMomentumSqr {d} (i : Fin d) := [][i] • L []^2

/-- The operator  $(\sum_j \epsilon_j) \epsilon_i$  on Schwartz maps. -/
private def positionDotMomentumCompMomentum {d} (i : Fin d) := positionDotM

/-- The operator  $\frac{1}{2} i \hbar (d-1) \epsilon_i$  on Schwartz maps. -/
private def constMomentum {d} (i : Fin d) := (2-1 * Complex.I * ħ * (d - 1)

```

1.86.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/HYDROGEN/LAPLACEREGION/ANGELNZVECTOR.LEAN

```

/-- The operator `mk·(ε)-1·` on Schwartz maps. -/
private def constRadiusRegInvCompPosition (ε : ℝ) (i : Fin H.d) := (H.m * H.k) • (ε
1] ·L (i]

/-
## Angular momentum / LRL vector commutators
-/

/-- `{i,j}, (ε)k} = iħ(δik(ε)j - δjk(ε)i)` -/
lemma angularMomentum_commutation_lrl (hε : 0 < ε) (i j k : Fin H.d) :
  {i,j}, H.lrlOperator ε k} = (Complex.I * ħ * δ[i,k]) • H.lrlOperator ε j
  - (Complex.I * ħ * δ[j,k]) • H.lrlOperator ε i := by
  rcases eq_or_ne i j with (⟨hij, rfl⟩ | hij)
  · rw [angularMomentumOperator_eq_zero, zero_lie, sub_self]

  unfold lrlOperator
  simp only [lie_sub, lie_smul, lie_sum, lie_add, lie_leibniz]
  simp only [angularMomentum_commutation_angularMomentum, angularMomentum_commutation_angularMomentum_commutation_position, angularMomentum_commutation_radiusRegPow h
  simp only [comp_add, comp_sub, add_comp, sub_comp, comp_smul, smul_comp, zero_comp
  ext ψ x
  simp only [ContinuousLinearMap.sub_apply, ContinuousLinearMap.smul_apply,
    ContinuousLinearMap.sum_apply, ContinuousLinearMap.add_apply, ContinuousLinearMap
  simp only [SchwartzMap.sub_apply, SchwartzMap.smul_apply, SchwartzMap.sum_apply,
    SchwartzMap.add_apply, SchwartzMap.smul_apply, smul_eq_mul]
  simp only [Finset.sum_add_distrib, Finset.sum_sub_distrib]
  dsimp only [kroneckerDelta]
  simp only [mul_ite_zero, mul_one, ite_mul, zero_mul, Finset.sum_ite_irrel, Complex
    Finset.sum_const_zero, Finset.sum_ite_eq, Finset.mem_univ, ↓reduceIte, smul_add]
  simp only [angularMomentumOperator_antisymm k _]
  simp only [mul_sub, mul_add, mul_ite_zero, Finset.mul_sum]
  simp only [ContinuousLinearMap.neg_apply, map_neg, SchwartzMap.neg_apply]

  rcases eq_or_ne i k with (⟨_, rfl⟩ | hik)
  · simp only [↓reduceIte, angularMomentumOperator_eq_zero]
    repeat rw [ite_cond_eq_false _ _ (eq_false hij.symm)]
    simp only [ContinuousLinearMap.zero_apply, SchwartzMap.zero_apply]
    ring_nf
  · rcases eq_or_ne j k with (⟨_, rfl⟩ | hjk)
    · simp only [↓reduceIte]
      repeat rw [ite_cond_eq_false _ _ (eq_false hij)]
      ring_nf
    · repeat rw [ite_cond_eq_false _ _ (eq_false hik)]
      repeat rw [ite_cond_eq_false _ _ (eq_false hjk)]
      ring

```

```

/-- `{[i,j], [(ε)²]} = 0` -/
lemma angularMomentum_commutation_lrlSqr (hε : 0 < ε) (i j : Fin H.d) :
  {[i,j], H.lrlOperatorSqr ε} = 0 := by
  unfold lrlOperatorSqr
  simp only [lie_sum, lie_leibniz, H.angularMomentum_commutation_lrl hε]
  simp only [comp_sub, comp_smul, sub_comp, smul_comp]
  dsimp only [kroneckerDelta]
  simp [Finset.sum_add_distrib, Finset.sum_sub_distrib]

/-- `{[²], [(ε)²]} = 0` -/
lemma angularMomentumSqr_commutation_lrlSqr (hε : 0 < ε) :
  {angularMomentumOperatorSqr (d := H.d), H.lrlOperatorSqr ε} = 0 := by
  unfold angularMomentumOperatorSqr
  simp [sum_lie, leibniz_lie, H.angularMomentum_commutation_lrlSqr hε]

/-

## LRL / LRL commutators

To compute the commutator `{[i(ε), j(ε)]}` we take the following approach:
- Write `{(ε)}_i = [i]² - ([j][j])[i] + 1/2 iħ(d-1)[i] - mk·[(ε)]⁻¹[i] = f1_i - f2_i + f
- Organize the sixteen terms which result from expanding `{f1_i -
f2_i+f3_i-f4_i, f1_j-f2_j+f3_j-f4_j}`
  into four diagonal terms such as `{f1_i, f1_j}` and six off-
diagonal pairs such as
  `{f1_i, f3_j} + {f3_i, f1_j} = {f1_i, f3_j} - {f1_j, f3_i}`.
- Compute the diagonal commutators and off-diagonal pairs individually. Man
  that don't are all of the form `iħ (...) [i]_j` (as they must to be antisymme
- Collect terms.

-

private lemma positionDotMomentum_commutation_position {d} (i : Fin d) :
  {positionDotMomentum (d := d), [i]} = (-Complex.I * ħ) • [i] := by
  unfold positionDotMomentum
  simp only [← lie_skew [i] _, position_commutation_position, position_com
    leibniz_lie, kroneckerDelta, sum_lie, comp_neg, comp_smul]
  simp

private lemma positionDotMomentum_commutation_momentum {d} (i : Fin d) :
  {positionDotMomentum (d := d), [i]} = (Complex.I * ħ) • [i] := by
  unfold positionDotMomentum
  simp only [momentum_commutation_momentum, position_commutation_momentum,
    sum_lie, leibniz_lie, smul_comp]
  simp

```

1.86.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/HYDROGEN/LAPLACERUNGELENZVECTOR.LEAN

```
private lemma positionDotMomentum_commutation_momentumSqr {d} :
  [positionDotMomentum (d := d), momentumOperatorSqr (d := d)] = (2 * Complex.I *
  unfold momentumOperatorSqr
  simp only [positionDotMomentum_commutation_momentum, lie_sum, lie_leibniz, comp_smul,
  smul_comp]
  rw [Finset.smul_sum]
  congr
  ext i ψ x
  simp only [ContinuousLinearMap.add_apply, coe_smul', coe_comp', Pi.smul_apply,
  Function.comp_apply, SchwartzMap.add_apply, SchwartzMap.smul_apply, smul_eq_mul]
  ring

private lemma positionDotMomentum_commutation_radiusRegPow {d} (hε : 0 < ε) :
  [positionDotMomentum (d := d), radiusRegPowOperator (d := d) ε s] =
  (-s * Complex.I * ħ) • (□[ε,s] - ε ^ 2 • □[ε,s-2]) := by
  unfold positionDotMomentum
  rw [sum_lie]
  conv_lhs =>
  enter [2, i]
  rw [leibniz_lie, position_commutation_radiusRegPow hε, zero_comp, add_zero]
  rw [← lie_skew, radiusRegPow_commutation_momentum hε, comp_neg, comp_smul]
  rw [comp_eq_comp_sub_commute □[ε,_] □[_], position_commutation_radiusRegPow hε,
  ← comp_assoc]
  rw [Finset.sum_neg_distrib, ← Finset.smul_sum, ← finset_sum_comp]
  rw [positionOperatorSqr_eq hε]
  rw [sub_comp, smul_comp, id_comp, radiusRegPowOperator_comp_eq hε]
  simp

private lemma positionCompMomentumSqr_comm {d} (i j : Fin d) :
  [positionCompMomentumSqr (d := d) i, positionCompMomentumSqr j] =
  (-2 * Complex.I * ħ) • □² ∘L □[i,j] := by
  unfold positionCompMomentumSqr
  rw [leibniz_lie, lie_leibniz, lie_leibniz]
  rw [lie_self, ← lie_skew □² □[j]]
  rw [position_commutation_position, momentumSqr_comp_angularMomentum_commute]
  repeat rw [position_commutation_momentumSqr]
  unfold angularMomentumOperator
  ext ψ x
  simp only [comp_zero, neg_comp, smul_comp, zero_add, comp_neg, comp_smul_s1, RingHom
  zero_comp, add_zero, ContinuousLinearMap.add_apply, ContinuousLinearMap.neg_appl
  coe_comp', Pi.smul_apply, Function.comp_apply, SchwartzMap.add_apply, SchwartzMa
  SchwartzMap.smul_apply, smul_eq_mul, neg_mul, sub_comp, neg_smul, coe_sub', Pi.s
  SchwartzMap.sub_apply]
  ring

private lemma positionCompMomentumSqr_comm_positionDotMomentumCompMomentum_add {d} (
```

```

    {positionCompMomentumSqr (d := d) i, positionDotMomentumCompMomentum j}
    {positionDotMomentumCompMomentum i, positionCompMomentumSqr j} =
    (-Complex.I * ħ) •  $\square^2$  •L  $\square[i,j]$  := by
  unfold positionCompMomentumSqr positionDotMomentumCompMomentum
  nth_rw 2 [← lie_skew]
  repeat rw [leibniz_lie, lie_leibniz, lie_leibniz]
  repeat rw [← lie_skew _ positionDotMomentum]
  repeat rw [position_commutation_momentum]
  repeat rw [momentumSqr_commutation_momentum]
  repeat rw [positionDotMomentum_commutation_position]
  repeat rw [positionDotMomentum_commutation_momentumSqr]
  simp only [neg_comp, smul_comp, add_comp, comp_zero, zero_add, comp_smul,
  repeat rw [comp_eq_comp_add_commute  $\square^2$   $\square[_]$ , momentumSqr_commutation_mome
  rw [kroneckerDelta_symm j i]
  trans (-Complex.I * ħ) •  $\square[i,j]$  •L  $\square^2$ 
  · ext  $\psi$  x
    unfold angularMomentumOperator
    simp only [add_zero, comp_neg, comp_smuls1, RingHom.id_apply, neg_mul,
      coe_sub', Pi.sub_apply, ContinuousLinearMap.add_apply, ContinuousLine
      coe_smul', coe_comp', Pi.smul_apply, Function.comp_apply, SchwartzMap
      SchwartzMap.add_apply, SchwartzMap.neg_apply, SchwartzMap.smul_apply,
    ring
  rw [comp_eq_comp_sub_commute  $\square^2$  _, angularMomentum_commutation_momentumSqr

private lemma positionDotMomentumCompMomentum_comm {d} (i j : Fin d) :
  {positionDotMomentumCompMomentum i, positionDotMomentumCompMomentum j}
  unfold positionDotMomentumCompMomentum
  rw [leibniz_lie, lie_leibniz, lie_leibniz, lie_self,
    ← lie_skew _ positionDotMomentum]
  ext  $\psi$  x
  simp [momentum_commutation_momentum, positionDotMomentum_commutation_mome
    momentum_comp_commute i j]

private lemma positionCompMomentumSqr_comm_constMomentum_add {d} (i j : Fin
  {positionCompMomentumSqr i, constMomentum j} +
  {constMomentum i, positionCompMomentumSqr j} = 0 := by
  unfold positionCompMomentumSqr constMomentum
  nth_rw 2 [← lie_skew]
  repeat rw [lie_smul, leibniz_lie, momentumSqr_commutation_momentum, comp_
    position_commutation_momentum, smul_comp, id_comp]
  rw [kroneckerDelta_symm j i, add_neg_eq_zero]

private lemma positionDotMomentumCompMomentum_comm_constMomentum_add {d} (i
  {positionDotMomentumCompMomentum (d := d) i, constMomentum j} +
  {constMomentum i, positionDotMomentumCompMomentum j} = 0 := by
  unfold positionDotMomentumCompMomentum constMomentum

```

1.86.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/HYDROGEN/LAPLACER&NGELENZVECTOR.LEAN

```
nth_rw 2 [← lie_skew]
repeat rw [lie_smul, leibniz_lie, momentum_commutation_momentum, comp_zero, zero_a
  positionDotMomentum_commutation_momentum, smul_comp]
rw [momentum_comp_commute, add_neg_eq_zero]
```

```
private lemma constMomentum_comm_constMomentum {d} (i j : Fin d) :
  {constMomentum i, constMomentum j} = 0 := by
  unfold constMomentum
  simp [momentum_commutation_momentum]
```

```
private lemma positionCompMomentumSqr_comm_constRadiusRegInvCompPosition_add
  (hε : 0 < ε) (i j : Fin H.d) :
  {positionCompMomentumSqr i, constRadiusRegInvCompPosition H ε j} +
  {constRadiusRegInvCompPosition H ε i, positionCompMomentumSqr j} =
  - (2 * H.m * H.k * Complex.I * ħ) • [ε, -1] •L [i, j] := by
  unfold positionCompMomentumSqr constRadiusRegInvCompPosition
  nth_rw 2 [← lie_skew]
  repeat rw [lie_smul, leibniz_lie, lie_leibniz, lie_leibniz]
  repeat rw [position_commutation_position]
  repeat rw [position_commutation_radiusRegPow hε]
  repeat rw [← lie_skew [2] _]
  repeat rw [position_commutation_momentumSqr]
  rw [radiusRegPow_commutation_momentumSqr hε]
  rw [← positionDotMomentum]
```

```
simp only [zero_comp, comp_zero, add_zero, comp_smul, comp_add, comp_neg, smul_sub
  smul_neg, neg_comp, add_comp, smul_comp, comp_assoc, sub_comp, comp_sub]
repeat rw [comp_eq_comp_add_commute positionDotMomentum [_, ],
  positionDotMomentum_commutation_position]
```

```
have hxr : ∀ i : Fin H.d, ∀ s, ∀ (A : [Space H.d, ℂ] →L[ℂ] [Space H.d, ℂ]),
  [i] •L [ε, s] •L A = [ε, s] •L [i] •L A := by
  intro i p A
  rw [← comp_assoc, position_comp_radiusRegPow_commute hε, comp_assoc]
  repeat rw [hxr]
  simp only [comp_add, comp_smul]
  rw [position_comp_commute j i]
```

```
have hxx_xp : [j] •L [i] •L positionDotMomentum = [i] •L [j] •L positionDotMomentum
  rw [← comp_assoc, position_comp_commute, comp_assoc]
  rw [hxx_xp]
```

```
unfold angularMomentumOperator
ext ψ x
simp only [Complex.ofReal_neg, Complex.ofReal_one, mul_neg, mul_one, neg_mul, neg_
  smul_neg, neg_neg, one_mul, sub_neg_eq_add, neg_add_rev, ContinuousLinearMap.add
```

```

ContinuousLinearMap.neg_apply, coe_smul', coe_comp', Pi.smul_apply, Fun
SchwartzMap.add_apply, SchwartzMap.neg_apply, SchwartzMap.smul_apply, s
Complex.real_smul, Complex.ofReal_mul, Complex.ofReal_pow, Complex.ofRe
Complex.ofReal_ofNat, Complex.ofReal_add, Complex.ofReal_natCast, comp
Pi.sub_apply, SchwartzMap.sub_apply]
ring

private lemma positionDotMomentumCompMomentum_comm_constRadiusRegInvCompPos
(hε : 0 < ε) (i j : Fin H.d) :
  [positionDotMomentumCompMomentum i, constRadiusRegInvCompPosition H ε j
  [constRadiusRegInvCompPosition H ε i, positionDotMomentumCompMomentum j
  (H.m * H.k * Complex.I * ħ * ε ^ 2) • [][ε,-3] ∘L [][i,j] := by
unfold positionDotMomentumCompMomentum constRadiusRegInvCompPosition
nth_rw 2 [← lie_skew]
repeat rw [lie_smul, leibniz_lie, lie_leibniz, lie_leibniz]
repeat rw [← lie_skew [][_] [][_], position_commutation_momentum]
repeat rw [positionDotMomentum_commutation_position]
repeat rw [← lie_skew [][_] _, radiusRegPow_commutation_momentum hε]
repeat rw [positionDotMomentum_commutation_radiusRegPow hε]
simp only [smul_comp, neg_comp, comp_assoc]
rw [position_comp_commute j i, kroneckerDelta_symm j i]
unfold angularMomentumOperator
ext ψ x
simp only [comp_neg, comp_smulsl, RingHom.id_apply, comp_id, Complex.ofRe
Complex.ofReal_one, neg_mul, one_mul, neg_smul, neg_neg, comp_add, sub
add_comp, neg_comp, smul_add, smul_neg, neg_add_rev, ContinuousLinearMa
ContinuousLinearMap.neg_apply, coe_smul', coe_comp', Pi.smul_apply, Fun
coe_sub', Pi.sub_apply, SchwartzMap.add_apply, SchwartzMap.neg_apply, S
smul_eq_mul, Complex.real_smul, Complex.ofReal_mul, SchwartzMap.sub_app
comp_sub]
ring_nf

private lemma constMomentum_comm_constRadiusRegInvCompPosition_add (hε : 0
  [constMomentum i, constRadiusRegInvCompPosition H ε j] +
  [constRadiusRegInvCompPosition H ε i, constMomentum j] = 0 := by
unfold constMomentum constRadiusRegInvCompPosition
nth_rw 2 [← lie_skew]
repeat rw [smul_lie, lie_smul, lie_leibniz]
repeat rw [← lie_skew [][_] _]
repeat rw [position_commutation_momentum, radiusRegPow_commutation_moment
simp only [neg_comp, smul_comp, comp_assoc]
rw [position_comp_commute j i, kroneckerDelta_symm j i]
ext ψ x
simp only [comp_neg, comp_smulsl, RingHom.id_apply, comp_id, Complex.ofRe
Complex.ofReal_one, neg_mul, one_mul, neg_smul, neg_neg, smul_add, smul
ContinuousLinearMap.add_apply, ContinuousLinearMap.neg_apply, coe_smul'

```


1.86.

PHYS/LEAN/QUANTUMMECHANICS/DDIMENSIONS/HYDROGEN/LAPLACEREGENGELENZVECTOR.LEAN

```

    coe_comp', Function.comp_apply, SchwartzMap.add_apply, SchwartzMap.neg_apply,
    SchwartzMap.smul_apply, smul_eq_mul, Complex.real_smul, Complex.ofReal_mul,
    ContinuousLinearMap.zero_apply, SchwartzMap.zero_apply]
ring

private lemma constRadiusRegInvCompPosition_comm_constRadiusRegInvCompPosition
  (hε : 0 < ε) (i j : Fin H.d) :
  {constRadiusRegInvCompPosition H ε i, constRadiusRegInvCompPosition H ε j} = 0 :
  unfold constRadiusRegInvCompPosition
  rw [lie_smul, smul_lie, leibniz_lie, lie_leibniz, lie_leibniz]
  rw [← lie_skew [ε, -1] _]
  rw [position_commutation_position]
  rw [radiusRegPow_commutation_radiusRegPow hε]
  repeat rw [position_commutation_radiusRegPow hε]
  simp only [comp_zero, zero_comp, add_zero, neg_zero, smul_zero]

/-- `{ε}_i, {ε}_j = -iħ 2m {ε}_{ij} - /-
lemma lrl_commutation_lrl (hε : 0 < ε) (i j : Fin H.d) : {H.lrlOperator ε i, H.lrlOperator ε j} =
  (-2 * Complex.I * ħ * H.m) • (H.hamiltonianReg ε) • L [i, j] := by
  repeat rw [lrlOperator_eq]

let f_x_p_sqr := positionCompMomentumSqr (d := H.d)
let f_xdotp_p := positionDotMomentumCompMomentum (d := H.d)
let f_const_p := constMomentum (d := H.d)
let f_c_rinvx := constRadiusRegInvCompPosition H ε
trans [f_x_p_sqr i, f_x_p_sqr j] + [f_xdotp_p i, f_xdotp_p j]
  + [f_const_p i, f_const_p j] + [f_c_rinvx i, f_c_rinvx j]
  - ([f_x_p_sqr i, f_xdotp_p j] + [f_xdotp_p i, f_x_p_sqr j])
  + ([f_x_p_sqr i, f_const_p j] + [f_const_p i, f_x_p_sqr j])
  - ([f_x_p_sqr i, f_c_rinvx j] + [f_c_rinvx i, f_x_p_sqr j])
  - ([f_xdotp_p i, f_const_p j] + [f_const_p i, f_xdotp_p j])
  + ([f_xdotp_p i, f_c_rinvx j] + [f_c_rinvx i, f_xdotp_p j])
  - ([f_const_p i, f_c_rinvx j] + [f_c_rinvx i, f_const_p j])
· unfold f_x_p_sqr f_xdotp_p f_const_p f_c_rinvx
  unfold positionCompMomentumSqr positionDotMomentumCompMomentum constMomentum
  constRadiusRegInvCompPosition positionDotMomentum
  simp only [lie_add, lie_sub, add_lie, sub_lie]
  ext ψ x
  simp only [ContinuousLinearMap.sub_apply, ContinuousLinearMap.add_apply, SchwartzMap.add_apply]
  ring

rw [positionCompMomentumSqr_comm]
rw [positionDotMomentumCompMomentum_comm]
rw [positionCompMomentumSqr_comm_positionDotMomentumCompMomentum_add]
rw [positionCompMomentumSqr_comm_constMomentum_add]

```

```

rw [positionDotMomentumCompMomentum_comm_constMomentum_add]
rw [constMomentum_comm_constMomentum]
rw [positionCompMomentumSqr_comm_constRadiusRegInvCompPosition_add H hε]
rw [positionDotMomentumCompMomentum_comm_constRadiusRegInvCompPosition_ad
rw [constMomentum_comm_constRadiusRegInvCompPosition_add H hε]
rw [constRadiusRegInvCompPosition_comm_constRadiusRegInvCompPosition H hε]

unfold hamiltonianReg
ext ψ x
simp only [ContinuousLinearMap.add_apply, coe_smul', coe_comp', Pi.smul_a
Function.comp_apply, SchwartzMap.add_apply, SchwartzMap.smul_apply, smu
coe_sub', Pi.sub_apply, SchwartzMap.sub_apply, Complex.real_smul, Compl
Complex.ofReal_inv, Complex.ofReal_pow, ContinuousLinearMap.zero_apply,
ring_nf
simp

/-
## Hamiltonian / LRL vector commutators
-/

private lemma pSqr_comm_pL_Lp (i : Fin H.d) :
  [momentumOperatorSqr (d := H.d), ∑ j, (□[j] °L □[i,j] + □[i,j] °L □[j])
rw [lie_sum]
conv_lhs =>
  enter [2, j]
  rw [lie_add, lie_leibniz, lie_leibniz]
  rw [momentumSqr_commutation_momentum]
  rw [← lie_skew, angularMomentum_commutation_momentumSqr]
simp only [neg_zero, comp_zero, zero_comp, add_zero, Finset.sum_const_zero]

private lemma rr_comm_rx (hε : 0 < ε) (i : Fin H.d) :
  [radiusRegPowOperator (d := H.d) ε (-1) + (2-1 * ε ^ 2) • □[ε, -
3], □[ε, -1] °L □[i]} = 0 := by
  rw [add_lie, smul_lie, lie_leibniz, lie_leibniz]
  repeat rw [radiusRegPow_commutation_radiusRegPow hε]
  repeat rw [← lie_skew, position_commutation_radiusRegPow hε]
  simp only [neg_zero, comp_zero, zero_comp, add_zero, smul_zero]

private lemma pSqr_comm_rx (hε : 0 < ε) (i : Fin H.d) :
  [momentumOperatorSqr (d := H.d), □[ε, -1] °L □[i]} =
  (-2 * Complex.I * ħ) • □[ε, -1] °L □[i]
  + (ħ ^ 2 * (H.d - 3) : ℝ) • □[ε, -3] °L □[i]
  + (3 * ħ ^ 2 * ε ^ 2) • □[ε, -5] °L □[i]
  + (2 * Complex.I * ħ) • □[ε, -3] °L (∑ j, □[j] °L □[j]) °L □[i] := by
  rw [lie_leibniz]
  rw [← lie_skew, position_commutation_momentumSqr]

```

1.86.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/HYDROGEN/LAPLACERUNGELENZVECTOR.LEAN

```

rw [← lie_skew, radiusRegPow_commutation_momentumSqr hε]
ext ψ x
simp only [comp_neg, comp_smulsl, RingHom.id_apply, Complex.ofReal_neg, Complex.of
mul_neg, mul_one, neg_mul, neg_smul, one_mul, neg_add_rev, neg_neg, add_comp, sm
sub_comp, ContinuousLinearMap.add_apply, ContinuousLinearMap.neg_apply, coe_smul
Pi.smul_apply, Function.comp_apply, coe_sub', Pi.sub_apply, coe_sum', Finset.sum
SchwartzMap.add_apply, SchwartzMap.neg_apply, SchwartzMap.smul_apply, smul_eq_mu
SchwartzMap.sub_apply, Complex.real_smul, Complex.ofReal_sub, Complex.ofReal_add
Complex.ofReal_natCast, Complex.ofReal_ofNat, Complex.ofReal_mul, Complex.ofReal
SchwartzMap.sum_apply]
ring_nf

private lemma rr_comm_pL_Lp (hε : 0 < ε) (i : Fin H.d) :
  [radiusRegPowOperator (d := H.d) ε (-1) + (2-1 * ε ^ 2) • □[ε,-
3],
  ∑ j, (□[j] •L □[i,j] + □[i,j] •L □[j])} =
  (- Complex.I * ħ) •
  (□[ε,-3] + (3 * 2-1 * ε ^ 2) • □[ε,-5]) •L ∑ j, (□[j] •L □[i,j] + □[i,j] •L □[j])
rw [lie_smul]
conv_lhs =>
  enter [2, j]
  simp only [add_lie, lie_add, smul_lie, lie_leibniz]
  repeat rw [← lie_skew _ □[_,_], angularMomentum_commutation_radiusRegPow hε]
  repeat rw [radiusRegPow_commutation_momentum hε]
  simp only [neg_zero, comp_zero, zero_comp, zero_add, add_zero]
  simp only [smul_comp, comp_smul, smul_add, ← comp_assoc]
  repeat rw [comp_eq_comp_add_commute □[_,_] □[ε,_],
    angularMomentum_commutation_radiusRegPow hε]
  simp only [comp_assoc]
  simp only [Finset.sum_add_distrib, ← Finset.smul_sum, ← comp_finset_sum]
ext ψ x
simp only [Complex.ofReal_neg, Complex.ofReal_one, neg_mul, one_mul, neg_smul,
Complex.ofReal_ofNat, smul_neg, add_zero, ContinuousLinearMap.add_apply,
ContinuousLinearMap.neg_apply, coe_smul', coe_comp', coe_sum', Pi.smul_apply,
Function.comp_apply, Finset.sum_apply, map_sum, SchwartzMap.add_apply, SchwartzM
SchwartzMap.smul_apply, SchwartzMap.sum_apply, smul_eq_mul, Complex.real_smul,
Complex.ofReal_mul, Complex.ofReal_inv, Complex.ofReal_pow, comp_add, add_comp,
smul_add]
ring_nf

private lemma xL_Lx_eq (hε : 0 < ε) (i : Fin H.d) : ∑ j, (□[j] •L □[i,j] + □[i,j] •L
(2 : ℝ) • (∑ j, □[j] •L □[j]) •L □[i] - (2 : ℝ) • □[ε,2] •L □[i] + (2 * ε ^ 2) •
- (Complex.I * ħ * (H.d - 3)) • □[i] := by
  conv_lhs =>
    enter [2, j]
    calc

```

```

_ = [][j] •L ([][i] •L [][j] - [][j] •L [][i])
  + ([][i] •L [][j] - [][j] •L [][i]) •L [][j] := rfl
_ = [][j] •L [][i] •L [][j] + [][i] •L [][j] •L [][j]
  - [][j] •L [][j] •L [][i] - [][j] •L [][i] •L [][j] := by
  rw [comp_sub, sub_comp]
ext ψ x
simp only [ContinuousLinearMap.add_apply, coe_sub', coe_comp', Pi.sub_apply,
  Function.comp_apply, SchwartzMap.add_apply, SchwartzMap.sub_apply]
ring
_ = [][j] •L [][j] •L [][i] + [][i] •L [][j] •L [][j] - (2 : ℝ) • [][j] •L [][i]
  + (2 * Complex.I * ħ * δ[i,j]) • [][j] - (Complex.I * ħ) • [][i] :=
  rw [comp_eq_comp_add_commute [][i] [][j], position_commutation_moment]
  rw [comp_eq_comp_sub_commute [][i] [][j], position_commutation_moment]
  rw [comp_eq_comp_add_commute [][j] [][j], position_commutation_moment]
  rw [kroneckerDelta_symm j i, kroneckerDelta_self]
ext ψ x
simp only [comp_add, comp_smulsl, RingHom.id_apply, comp_id, comp_sub_apply,
  coe_smul', Pi.sub_apply, ContinuousLinearMap.add_apply, Function.comp_apply,
  Pi.smul_apply, SchwartzMap.sub_apply, SchwartzMap.add_apply, SchwartzMap.smul_eq_mul,
  mul_one, Complex.real_smul, Complex.ofReal_ofNat]
ring
_ = [][j] •L [][j] •L [][i] + [][j] •L [][i] •L [][j] - (2 : ℝ) • [][j] •L [][i]
  + (2 * Complex.I * ħ * δ[i,j]) • [][j] - (Complex.I * ħ) • [][i] :=
  nth_rw 2 [← comp_assoc]
  rw [position_comp_commute i j, comp_assoc]
_ = (2 : ℝ) • ([][j] •L [][j]) •L [][i] - (2 : ℝ) • ([][j] •L [][j]) •L [][i]
  + (3 * Complex.I * ħ * δ[i,j]) • [][j] - (Complex.I * ħ) • [][i] :=
  rw [comp_eq_comp_add_commute [][i] [][j], position_commutation_moment]
ext ψ x
simp only [comp_add, comp_smulsl, RingHom.id_apply, comp_id, coe_sub_apply,
  Pi.sub_apply, ContinuousLinearMap.add_apply, coe_comp', Function.comp_apply,
  Pi.smul_apply, SchwartzMap.sub_apply, SchwartzMap.add_apply, SchwartzMap.smul_eq_mul,
  Complex.real_smul, Complex.ofReal_ofNat, sub_left_inj]
ring
simp only [Finset.sum_sub_distrib, Finset.sum_add_distrib, ← Finset.smul_eq_mul,
  rw [positionOperatorSqr_eq hε, sub_comp, smul_comp, id_comp]]

unfold kroneckerDelta
ext ψ x
simp only [ContinuousLinearMap.sub_apply, ContinuousLinearMap.add_apply,
  ContinuousLinearMap.smul_apply, ContinuousLinearMap.sum_apply, SchwartzMap.add_apply,
  SchwartzMap.smul_apply, SchwartzMap.sum_apply]
simp only [coe_comp', coe_sum', Function.comp_apply, Finset.sum_apply, SchwartzMap.smul_eq_mul,
  Complex.real_smul, Complex.ofReal_ofNat, Complex.ofReal_pow, mul_ite, mul_eq_mul,
  ite_mul, zero_mul, Finset.sum_ite_eq, Finset.mem_univ, Finset.sum_const,
  Finset.card_univ, Fintype.card_fin, nsmul_eq_mul, Complex.ofReal_ofNat]

```

1.86.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/HYDROGEN/LAPLACERUNGELENZVECTOR.LEAN

```

ring

/-- `[(ε), (ε)i] = iħkε2(3/4[(ε)-5([j[ij + [ij[j] + 3iħ/2 [(ε)-5[i + [(ε)-3[i)]` -
/
lemma hamiltonianReg_commutation_lrl (hε : 0 < ε) (i : Fin H.d) :
  {H.hamiltonianReg ε, H.lrlOperator ε i} = (Complex.I * ħ * H.k * ε ^ 2) •
    ((3 * 4-1 : ℝ) • [(ε, -5] •L ∑ j, ([j] •L [i, j] + [i, j] •L [j])
      + (3 * 2-1 * Complex.I * ħ) • [(ε, -5] •L [i] + [(ε, -
3] •L [i]) := by
  unfold hamiltonianReg lrlOperator
  rw [sub_lie, lie_sub, lie_sub]
  simp only [lie_smul, smul_lie]

  rw [pSqr_comm_pL_Lp]
  rw [rr_comm_rx H hε]
  rw [pSqr_comm_rx H hε]
  rw [rr_comm_pL_Lp H hε]
  rw [xL_Lx_eq H hε]

  simp only [smul_zero, sub_zero, zero_sub, smul_smul, smul_add, smul_sub, comp_smul
    add_comp, comp_sub, comp_add]
  simp only [← comp_assoc, radiusRegPow0Operator_comp_eq hε]
  rw [comp_assoc]
  field_simp
  rw [← sub_eq_zero]

ext ψ x
simp only [neg_smul, smul_neg, neg_add_rev, neg_neg, Complex.I_sq, neg_mul, one_mu
  Pi.sub_apply, ContinuousLinearMap.add_apply, ContinuousLinearMap.neg_apply, coe_
  coe_comp', coe_sum', Pi.smul_apply, Function.comp_apply, Finset.sum_apply, map_s
  SchwartzMap.sub_apply, SchwartzMap.add_apply, SchwartzMap.neg_apply, SchwartzMap
  SchwartzMap.sum_apply, smul_eq_mul, Complex.real_smul, Complex.ofReal_div, Compl
  Complex.ofReal_mul, Complex.ofReal_pow, Complex.ofReal_sub, Complex.ofReal_natCa
  ContinuousLinearMap.zero_apply, SchwartzMap.zero_apply]
ring_nf
rw [Complex.I_sq]
simp

```

LRL vector squared

To compute $[(\epsilon)^2]$ we take the following approach:

- Write $[(\epsilon)_i] = [\epsilon]_i + \frac{1}{2}i\hbar(d-1)[\epsilon]_i - mk \cdot [(\epsilon)^{-1}]_i$ for the first term and $[(\epsilon)_i] = [\epsilon]_i + \frac{1}{2}i\hbar(d-1)[\epsilon]_i - mk \cdot [(\epsilon)^{-1}]_i$ for the second.
- Expand out to nine terms: one is a triple sum, two are double sums and the rest are

```

- Compute each term, symmetrizing the sums (see `sum_symmetrize` and `sum_s
- Collect terms.

-/

private lemma sum_symmetrize (f : Fin H.d → Fin H.d →  $\mathbb{C}$  → L[ $\mathbb{C}$ ]
   $\sum i, \sum j, f i j = (2 : \mathbb{C})^{-1} \cdot \sum i, \sum j, (f i j + f j i) :=$  by
  simp only [Finset.sum_add_distrib]
  nth_rw 3 [Finset.sum_comm]
  ext  $\psi$  x
  rw [ContinuousLinearMap.smul_apply, SchwartzMap.smul_apply, ContinuousLin
    SchwartzMap.add_apply, smul_eq_mul]
  ring

private lemma sum_symmetrize'
  (f : Fin H.d → Fin H.d → Fin H.d →  $\mathbb{C}$  → L[ $\mathbb{C}$ ]  $\mathbb{C}$  → L[ $\mathbb{C}$ ]
   $\sum i, \sum j, \sum k, f i j k = (2 : \mathbb{C})^{-1} \cdot \sum i, \sum k, \sum j, (f i j k + f k j i) :=$  by
  simp only [Finset.sum_add_distrib]
  nth_rw 3 [Finset.sum_comm]
  conv_rhs =>
    enter [2, 1, 2, i]
    rw [Finset.sum_comm]
  conv_rhs =>
    enter [2, 2, 2, i]
    rw [Finset.sum_comm]
  ext  $\psi$  x
  rw [ContinuousLinearMap.smul_apply, SchwartzMap.smul_apply, ContinuousLin
    SchwartzMap.add_apply, smul_eq_mul]
  ring

private lemma sum_Lpp_zero :  $\sum i : \text{Fin } H.d, \sum j, [i, j] \circ L [j] \circ L [i] = 0$ 
  rw [sum_symmetrize]
  conv_lhs =>
    enter [2, 2, i, 2, j]
    rw [angularMomentumOperator_antisymm j i, momentum_comp_commute j i, ne
  simp

private lemma sum_ppL_zero :  $\sum i : \text{Fin } H.d, \sum j, [i] \circ L [j] \circ L [i, j] = 0$ 
  rw [sum_symmetrize]
  conv_lhs =>
    enter [2, 2, i, 2, j]
    rw [ $\leftarrow$  comp_assoc,  $\leftarrow$  comp_assoc]
    rw [angularMomentumOperator_antisymm j i, momentum_comp_commute j i, co
  simp

private lemma sum_LppL :  $\sum i : \text{Fin } H.d, \sum j, \sum k, [i, j] \circ L [j] \circ L [k] \circ L$ 

```

1.86.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/HYDROGEN/LAPLACEREGULARGELENZVECTOR.LEAN

```

    =  $\square^2 \circ L \square^2$  := by
-- Apply a particular symmetrization to the triple sum
rw [sum_symmetrize']
conv_lhs =>
  enter [2, 2, i, 2, j, 2, k]
  rw [angularMomentumOperator_antisymm j i]
  repeat rw [comp_neg]
  repeat rw [← comp_assoc]
  rw [← sub_eq_add_neg, ← sub_comp]
  enter [1]
  unfold angularMomentumOperator
  calc
    _ =  $\square[i] \circ L \square[k] \circ L \square[k] \circ L \square[j] - \square[k] \circ L \square[i] \circ L \square[k] \circ L \square[j]$ 
      -  $\square[j] \circ L \square[k] \circ L \square[k] \circ L \square[i] + \square[k] \circ L \square[j] \circ L \square[k] \circ L \square[i]$  := by
      simp only [sub_comp, comp_assoc, sub_add]

    _ =  $\square[i] \circ L \square[k] \circ L \square[k] \circ L \square[j] - \square[j] \circ L \square[k] \circ L \square[k] \circ L \square[i]$  := by
    nth_rw 2 [momentum_comp_commute k j]
    nth_rw 2 [momentum_comp_commute k i]
    nth_rw 4 [← comp_assoc]
    rw [momentum_comp_commute i j, comp_assoc]
    ext  $\psi$  x
    simp only [ContinuousLinearMap.add_apply, coe_sub', coe_comp', Pi.sub_apply,
      Function.comp_apply, SchwartzMap.add_apply, SchwartzMap.sub_apply]
    ring

-- Back out of inner sum
conv_lhs =>
  enter [2, 2, i, 2, j]
  rw [← finset_sum_comp, Finset.sum_sub_distrib, ← comp_finset_sum, ← comp_finset]
  simp only [← comp_assoc, ← finset_sum_comp]
  rw [← momentumOperatorSqr]
  repeat rw [comp_eq_comp_add_commute  $\square[\_]$   $\square^2$ , position_commutation_momentumSqr, a
    smul_comp, comp_assoc]
  rw [momentum_comp_commute j i]
  rw [add_sub_add_right_eq_sub]
  rw [← comp_sub, ← angularMomentumOperator, comp_assoc]

simp only [← comp_finset_sum]
rw [← comp_smul, ← angularMomentumOperatorSqr]

private lemma sum_Lprx (h $\epsilon$  :  $0 < \epsilon$ ) :
   $\sum i : \text{Fin } H.d, \sum j, \square[i,j] \circ L \square[j] \circ L \square[\epsilon,-1] \circ L \square[i] = \square[\epsilon,-$ 
1]  $\circ L \square^2$  := by
  simp only [comp_eq_comp_sub_commute  $\square[\epsilon,-1]$   $\square[\_]$ , position_commutation_radiusRegPo
    sub_zero]
```

```

simp only [← comp_assoc, ← finset_sum_comp _ [][ε,-1]]
rw [sum_symmetrize]
conv_lhs =>
  enter [1, 2, 2, i, 2, j]
  rw [angularMomentumOperator_antisymm j i, neg_comp, neg_comp, ← sub_eq]
  rw [comp_assoc, comp_assoc, ← comp_sub]
  repeat rw [comp_eq_comp_sub_commute [][_] [][_], position_commutation_mom]
  rw [kroneckerDelta_symm j i, sub_sub_sub_cancel_right]
  rw [← angularMomentumOperator]
  rw [← angularMomentumOperatorSqr, ← sub_eq_zero]
  exact angularMomentumSqr_commutation_radiusRegPow hε

private lemma sum_rxpL :
  ∑ i : Fin H.d, ∑ j, [][ε,-1] °L [][i] °L [][j] °L [][i,j] = [][ε,-1] °L []² := by
  simp only [← comp_finset_sum [][ε,-1]]
  rw [sum_symmetrize]
  conv_lhs =>
    enter [2, 2, 2, i, 2, j]
    rw [angularMomentumOperator_antisymm j i, comp_neg, comp_neg, ← sub_eq]
    rw [← comp_assoc, ← comp_assoc, ← sub_comp, ← angularMomentumOperator]
    rw [← angularMomentumOperatorSqr]

private lemma sum_prx (hε : 0 < ε) : ∑ i : Fin H.d, [][i] °L [][ε,-1] °L [][i] =
  [][ε,-1] °L ∑ i : Fin H.d, [][i] °L [][i] - (Complex.I * ħ * (H.d - 1)) •
  1] - (Complex.I * ħ * ε ^ 2) • [][ε,-3] := by
  conv_lhs =>
    enter [2, i]
    rw [← comp_assoc, comp_eq_comp_sub_commute [][_] [][ε,-1], radiusRegPow_commutation_momentum hε]
    rw [sub_comp, smul_comp, comp_assoc, comp_assoc]
    rw [comp_eq_comp_sub_commute [][_] [][_], position_commutation_momentum]
    rw [kroneckerDelta_self]
    rw [comp_sub, comp_smul, comp_id]
  repeat rw [Finset.sum_sub_distrib, ← Finset.smul_sum, ← comp_finset_sum]
  rw [positionOperatorSqr_eq hε, comp_sub, radiusRegPowOperator_comp_eq hε,

ext ψ x
simp only [ContinuousLinearMap.sub_apply, SchwartzMap.sub_apply, ContinuousLinearMap.smul_apply, SchwartzMap.smul_apply, ContinuousLinearMap.sum_apply, SchwartzMap.sum_apply, mul_one, Finset.sum_const, Finset.card_univ, Finset.smul_eq_mul, smul_eq_mul, Complex.ofReal_neg, Complex.ofReal_one, neg_sub_add_cancel, Complex.real_smul, Complex.ofReal_pow, sub_neg_eq_add]

```


1.86.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/HYDROGEN/LAPLACEREGULARIZEDANGELNZVECTOR.LEAN

```

ring_nf

private lemma sum_rxp :  $\sum i : \text{Fin } H.d, \llbracket \varepsilon, -1 \rrbracket \circ_L \llbracket i \rrbracket \circ_L \llbracket i \rrbracket =$ 
   $\llbracket \varepsilon, -1 \rrbracket \circ_L \sum i : \text{Fin } H.d, \llbracket i \rrbracket \circ_L \llbracket i \rrbracket :=$  by rw [comp_finset_sum]

private lemma sum_rxr (hε : 0 < ε) :  $\sum i, \llbracket \varepsilon, -1 \rrbracket \circ_L \llbracket i \rrbracket \circ_L \llbracket \varepsilon, -1 \rrbracket \circ_L \llbracket i \rrbracket =$ 
  ContinuousLinearMap.id  $\mathbb{C} \llbracket (\text{Space } H.d, \mathbb{C}) - (\varepsilon^2) \cdot \llbracket \varepsilon, -2 \rrbracket$ 
:= by
  conv_lhs =>
    enter [2, i]
    calc
      _ =  $\llbracket \varepsilon, -1 \rrbracket \circ_L \llbracket \varepsilon, -1 \rrbracket \circ_L \llbracket i \rrbracket \circ_L \llbracket i \rrbracket :=$  by
        nth_rw 2 [← comp_assoc]
        rw [comp_eq_comp_add_commute  $\llbracket i \rrbracket \llbracket \varepsilon, -1 \rrbracket$ , position_commutation_radiusRegPow0
          add_zero, comp_assoc]
      _ =  $\llbracket \varepsilon, -2 \rrbracket \circ_L \llbracket i \rrbracket \circ_L \llbracket i \rrbracket :=$  by
        rw [← comp_assoc, radiusRegPow0operator_comp_eq hε]
        congr
        ring
      rw [← comp_finset_sum, positionOperatorSqr_eq hε, comp_sub, comp_smul, comp_id,
        radiusRegPow0operator_comp_eq hε, neg_add_cancel, radiusRegPow0operator_zero hε]

/-- The square of the (regularized) LRL vector operator is related to the (regularized)
   $\llbracket \varepsilon \rrbracket$  of the hydrogen atom, square of the angular momentum  $\llbracket \mathbf{L}^2 \rrbracket$  and powers of  $\llbracket \varepsilon \rrbracket$ 
   $\llbracket \varepsilon \rrbracket^2 = 2m \llbracket \varepsilon \rrbracket (\llbracket \mathbf{L}^2 \rrbracket + \frac{1}{4} \hbar^2 (d-1)^2) + m^2 k^2 - m^2 k^2 \varepsilon^2 \llbracket \varepsilon \rrbracket^{-2} + m k \varepsilon^2 \llbracket \varepsilon \rrbracket^{-3} (\llbracket \mathbf{L}^2 \rrbracket + \frac{1}{4} \hbar^2 (d-1)(d-3))$ . -/
lemma lrlOperatorSqr_eq (hε : 0 < ε) :  $H.lrlOperatorSqr \varepsilon =$ 
   $(2 * H.m) \cdot (H.hamiltonianReg \varepsilon) \circ_L$ 
   $(\llbracket \mathbf{L}^2 \rrbracket + (4^{-1} * \hbar^2 * (H.d - 1)^2 : \mathbb{R})) \cdot \text{ContinuousLinearMap.id } \mathbb{C} \llbracket (\text{Space } H.d,$ 
   $+ (H.m * H.k) ^ 2 \cdot \text{ContinuousLinearMap.id } \mathbb{C} \llbracket (\text{Space } H.d, \mathbb{C})$ 
   $- ((H.m * H.k) ^ 2 * \varepsilon ^ 2) \cdot \llbracket \varepsilon, -2 \rrbracket$ 
   $+ (H.m * H.k * \varepsilon ^ 2) \cdot \llbracket \varepsilon, -3 \rrbracket \circ_L$ 
   $(\llbracket \mathbf{L}^2 \rrbracket + (4^{-1} * \hbar^2 * (H.d - 1) * (H.d - 3) : \mathbb{R})) \cdot$ 
  ContinuousLinearMap.id  $\mathbb{C} \llbracket (\text{Space } H.d, \mathbb{C}) \rrbracket :=$  by
  unfold lrlOperatorSqr

let a :=  $(2^{-1} * \text{Complex.I} * \hbar * (H.d - 1))$ 

-- Replace the two copies of  $\llbracket \varepsilon \rrbracket$  in different ways and expand to nine terms
conv_lhs =>
  enter [2, i, 1]
  rw [lrlOperator_eq']
conv_lhs =>
  enter [2, i]
  rw [lrlOperator_eq'']

```

```

calc
_ = (∑ j, [i,j] ⋅L [j]) ⋅L (∑ k, [k] ⋅L [i,k])
- a ⋅ (∑ j, [i,j] ⋅L [j]) ⋅L [i]
+ a ⋅ [i] ⋅L (∑ k, [k] ⋅L [i,k])
- (a * a) ⋅ [i] ⋅L [i]
- (H.m * H.k) ⋅ (∑ j, [i,j] ⋅L [j]) ⋅L [ε,-1] ⋅L [i]
- (H.m * H.k) ⋅ [ε,-1] ⋅L [i] ⋅L (∑ k, [k] ⋅L [i,k])
- (a * H.m * H.k) ⋅ [i] ⋅L [ε,-1] ⋅L [i]
+ (a * H.m * H.k) ⋅ [ε,-1] ⋅L [i] ⋅L [i]
+ (H.m * H.k) ^ 2 ⋅ [ε,-1] ⋅L [i] ⋅L [ε,-1] ⋅L [i] := by
simp only [comp_sub, add_comp, sub_comp, comp_smul, smul_comp]
ext ψ x
simp only [coe_sub', coe_smul', coe_comp', coe_sum', Pi.sub_apply,
ContinuousLinearMap.add_apply, Finset.sum_apply, Pi.smul_apply, S
SchwartzMap.add_apply, SchwartzMap.sum_apply, SchwartzMap.smul_ap
smul_eq_mul, Complex.real_smul, Complex.ofReal_mul, Complex.ofRea
ring
_ = ∑ j, ∑ k, [i,j] ⋅L [j] ⋅L [k] ⋅L [i,k]
- a ⋅ (∑ j, [i,j] ⋅L [j] ⋅L [i])
+ a ⋅ (∑ k, [i] ⋅L [k] ⋅L [i,k])
- (a * a) ⋅ [i] ⋅L [i]
- (H.m * H.k) ⋅ (∑ j, [i,j] ⋅L [j] ⋅L [ε,-1] ⋅L [i])
- (H.m * H.k) ⋅ (∑ k, [ε,-1] ⋅L [i] ⋅L [k] ⋅L [i,k])
- (a * H.m * H.k) ⋅ [i] ⋅L [ε,-1] ⋅L [i]
+ (a * H.m * H.k) ⋅ [ε,-1] ⋅L [i] ⋅L [i]
+ (H.m * H.k) ^ 2 ⋅ [ε,-1] ⋅L [i] ⋅L [ε,-1] ⋅L [i] := by
repeat rw [finset_sum_comp, comp_finset_sum]
ext ψ x
simp only [ContinuousLinearMap.add_apply, coe_sub', coe_smul', coe_
Pi.sub_apply, Finset.sum_apply, Function.comp_apply, map_sum, Pi.
SchwartzMap.add_apply, SchwartzMap.sub_apply, SchwartzMap.sum_app
SchwartzMap.smul_apply, Complex.real_smul, Complex.ofReal_mul, Co

-- Split the outer sum
simp only [Finset.sum_add_distrib, Finset.sum_sub_distrib, ← Finset.smul_

rw [sum_LppL] -- ∑∑ LppL = p²L²
rw [sum_Lpp_zero, smul_zero] -- ∑∑ Lpp = 0
rw [sum_ppL_zero, smul_zero] -- ∑∑ ppL = 0
rw [← momentumOperatorSqr] -- ∑ pp = p²
rw [sum_Lprx H hε] -- ∑∑ Lpr⁻¹x = r⁻¹L²
rw [sum_rxpL] -- ∑∑ r⁻¹xpL = r⁻¹L²
rw [sum_prx H hε] -- ∑ pr⁻¹x = r⁻¹ ∑ xp - iħ(d-1)r⁻¹ - iħε²r⁻³
rw [sum_rxp] -- ∑ r⁻¹xp = r⁻¹ ∑ xp
rw [sum_rxx H hε] -- ∑ r⁻¹xr⁻¹x = 1 - ε²r⁻²

```

1.87.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/ANGULARMOMENTUM.LEAN

```
unfold a hamiltonianReg
ext  $\psi$  x
simp only [ContinuousLinearMap.add_apply, coe_sub', coe_comp', coe_smul', Schwartz
  Pi.sub_apply, Function.comp_apply, Pi.smul_apply, SchwartzMap.sub_apply, smul_eq
  SchwartzMap.smul_apply, Complex.real_smul, Complex.ofReal_mul, Complex.ofReal_of
ring_nf
rw [Complex.I_sq]
simp only [neg_mul, one_mul, one_div, sub_neg_eq_add, Complex.ofReal_mul, Complex.
  coe_id', id_eq, Complex.ofReal_inv, Complex.ofReal_ofNat, map_add, map_smul_of_t
  SchwartzMap.add_apply, SchwartzMap.smul_apply, Complex.real_smul, Complex.ofReal
  Complex.ofReal_natCast, Complex.ofReal_div, Complex.ofReal_neg, Complex.ofReal_o
  Complex.ofReal_sub, ne_eq, Complex.ofReal_eq_zero, m_ne_zero, not_false_eq_true,
  mul_inv_cancel_left, add_left_inj]
ring

end
end HydrogenAtom
end QuantumMechanics
```

1.87 PhysLean/QuantumMechanics/DDimensions/Operators/AngularMomentum.L

/-

Copyright (c) 2026 Gregory J. Loges. All rights reserved.

Released under Apache 2.0 license as described in the file LICENSE.

Authors: Gregory J. Loges

-/

import PhysLean.QuantumMechanics.DDimensions.Operators.Position

import PhysLean.QuantumMechanics.DDimensions.Operators.Momentum

/-!

Angular momentum operator

In this module we define:

- The angular momentum operator on Schwartz maps, component-wise.

- The angular momentum squared operator.

- The angular momentum scalar operator in 2d and angular momentum vector operator in

-/

namespace QuantumMechanics

noncomputable section

open Constants

open ContDiff SchwartzMap

```

/-
# Definition
-/

/-- Component `i j` of the angular momentum operator is the continuous linear map from  $\mathbb{C}^d$  to itself defined by  $\mathbb{L}_{ij} = \mathbb{L}_i \circ \mathbb{L}_j - \mathbb{L}_j \circ \mathbb{L}_i$ . -
/
def angularMomentumOperator {d : ℕ} (i j : Fin d) :  $\mathbb{C}^d \rightarrow \mathbb{C}^d$  :=  $\lambda \psi, \mathbb{L}_i \mathbb{L}_j \psi - \mathbb{L}_j \mathbb{L}_i \psi$ 

@[inherit_doc angularMomentumOperator]
macro "L[" i:term "," j:term "]" : term => `(angularMomentumOperator $i $j)

lemma angularMomentumOperator_apply_fun {d : ℕ} (i j : Fin d) ( $\psi : \mathbb{C}^d$ ) :  $\mathbb{L}_{ij} \psi = \mathbb{L}_i (\mathbb{L}_j \psi) - \mathbb{L}_j (\mathbb{L}_i \psi)$  := rfl

lemma angularMomentumOperator_apply {d : ℕ} (i j : Fin d) ( $\psi : \mathbb{C}^d$ ) (x :  $\mathbb{C}^d$ ) :  $\mathbb{L}_{ij} \psi x = \mathbb{L}_i (\mathbb{L}_j \psi) x - \mathbb{L}_j (\mathbb{L}_i \psi) x$  := rfl

/-- The square of the angular momentum operator,  $\mathbb{L}^2 = \frac{1}{2} \sum_{i,j} \mathbb{L}_{ij} \circ \mathbb{L}_{ij}$ . -
/
def angularMomentumOperatorSqr {d : ℕ} :  $\mathbb{C}^d \rightarrow \mathbb{C}^d$  :=  $(2 : \mathbb{C})^{-1} \cdot \sum i, \sum j, \mathbb{L}_{ij} \circ \mathbb{L}_{ij}$ 

@[inherit_doc angularMomentumOperatorSqr]
notation "L^2" => angularMomentumOperatorSqr

lemma angularMomentumOperatorSqr_apply_fun {d : ℕ} ( $\psi : \mathbb{C}^d$ ) :  $\mathbb{L}^2 \psi = (2 : \mathbb{C})^{-1} \cdot \sum i, \sum j, \mathbb{L}_{ij} (\mathbb{L}_{ij} \psi)$  := by
  dsimp only [angularMomentumOperatorSqr]
  simp only [ContinuousLinearMap.coe_sum', ContinuousLinearMap.coe_smul', ContinuousLinearMap.coe_comp', Finset.sum_apply, Pi.smul_apply, Function.comp_apply]

lemma angularMomentumOperatorSqr_apply {d : ℕ} ( $\psi : \mathbb{C}^d$ ) (x :  $\mathbb{C}^d$ ) :  $\mathbb{L}^2 \psi x = (2 : \mathbb{C})^{-1} \cdot \sum i, \sum j, \mathbb{L}_{ij} (\mathbb{L}_{ij} \psi) x$  := by
  rw [angularMomentumOperatorSqr_apply_fun]
  simp only [smul_apply, sum_apply, smul_eq_mul]

/-
## Basic properties
-/

```

1.87.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/ANGULARMOMENTUM.LEAN

```

/-- The angular momentum operator is antisymmetric, ` $\hat{L}_{ij} = -\hat{L}_{ji}$ ` -/
lemma angularMomentumOperator_antisymm {d : ℕ} (i j : Fin d) :  $\hat{L}[i,j] = -\hat{L}[j,i]$  :=
  Eq.symm (neg_sub _ _)

/-- Angular momentum operator components with repeated index vanish, ` $\hat{L}_{ii} = 0$ ` -/
lemma angularMomentumOperator_eq_zero {d : ℕ} (i : Fin d) :  $\hat{L}[i,i] = 0$  := sub_self _

/-

## Special cases in low dimensions

• d = 1 : The angular momentum operator is trivial.

• d = 2 : The angular momentum operator has only one independent component,  $\hat{L}_{01}$ , which
  be thought of as a (pseudo)scalar operator.

• d = 3 : The angular momentum operator has three independent components,  $\hat{L}_{01}, \hat{L}_{12}, \hat{L}_{20}$ .
  Dualizing using the Levi-Civita symbol produces the familiar (pseudo)vector
  momentum operator with components  $\hat{L}_0 = \hat{L}_{20}, \hat{L}_1 = \hat{L}_{20}$  and  $\hat{L}_2 = \hat{L}_{01}$ .

-/

/-- In one dimension the angular momentum operator is trivial. -/
lemma angularMomentumOperator1D_trivial :  $\forall (i j : \text{Fin } 1), \hat{L}[i,j] = 0$  := by
  intro i j
  fin_cases i, j
  exact angularMomentumOperator_eq_zero 0

/-- The angular momentum (pseudo)scalar operator in two dimensions, ` $\hat{L} = \hat{L}_{01}$ ` -/
def angularMomentumOperator2D :  $(\text{Space } 2, \mathbb{C}) \rightarrow L[\mathbb{C}] (\text{Space } 2, \mathbb{C}) := \hat{L}[0,1]$ 

/-- The angular momentum (pseudo)vector operator in three dimension, ` $\hat{L}_i = \frac{1}{2} \sum_{jk} \epsilon_{ijk} \hat{L}_{jk}$ ` -/
def angularMomentumOperator3D (i : Fin 3) :  $(\text{Space } 3, \mathbb{C}) \rightarrow L[\mathbb{C}] (\text{Space } 3, \mathbb{C}) :=$ 
  match i with
  | 0 =>  $\hat{L}[1,2]$ 
  | 1 =>  $\hat{L}[2,0]$ 
  | 2 =>  $\hat{L}[0,1]$ 

end
end QuantumMechanics

```

1.88 PhysLean/QuantumMechanics/DDimensions/Operators/Commutatio

```

/-
Copyright (c) 2026 Gregory J. Loges. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Gregory J. Loges
-/
import PhysLean.Mathematics.KroneckerDelta
import PhysLean.QuantumMechanics.DDimensions.Operators.AngularMomentum
/-!

# Commutation relations

-/

namespace QuantumMechanics
noncomputable section
open Constants
open KroneckerDelta
open SchwartzMap ContinuousLinearMap

private lemma ite_cond_symm (i j : Fin d) :
  (if i = j then A else B) = (if j = i then A else B) :=
  ite_cond_congr (Eq.propIntro Eq.symm Eq.symm)

lemma leibniz_lie {d : ℕ} (A B C :  $\mathbb{C}(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathbb{C}(\text{Space } d, \mathbb{C})$ ) :
  {A  $\circ_L$  B, C} = A  $\circ_L$  {B, C} + {A, C}  $\circ_L$  B := by
  dsimp only [Bracket.bracket]
  simp only [ContinuousLinearMap.mul_def, comp_assoc, comp_sub, sub_comp, s

lemma lie_leibniz {d : ℕ} (A B C :  $\mathbb{C}(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathbb{C}(\text{Space } d, \mathbb{C})$ ) :
  {A, B  $\circ_L$  C} = B  $\circ_L$  {A, C} + {A, B}  $\circ_L$  C := by
  dsimp only [Bracket.bracket]
  simp only [ContinuousLinearMap.mul_def, comp_assoc, comp_sub, sub_comp, s

lemma comp_eq_comp_add_commute (A B :  $\mathbb{C}(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathbb{C}(\text{Space } d, \mathbb{C})$ ) :
  A  $\circ_L$  B = B  $\circ_L$  A + {A, B} := by
  dsimp only [Bracket.bracket]
  simp only [ContinuousLinearMap.mul_def, add_sub_cancel]

lemma comp_eq_comp_sub_commute (A B :  $\mathbb{C}(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \mathbb{C}(\text{Space } d, \mathbb{C})$ ) :
  A  $\circ_L$  B = B  $\circ_L$  A - {B, A} := by
  dsimp only [Bracket.bracket]

```

1.88.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/COMMUTATION.LEAN

```
    simp only [ContinuousLinearMap.mul_def, sub_sub_cancel]

/-
## Position / position commutators
-/

/-- Position operators commute:  $[x_i, x_j] = 0$ . -/
lemma position_commutation_position {d : ℕ} (i j : Fin d) : {⊖[i], ⊖[j]} = 0 := by
  dsimp only [Bracket.bracket]
  ext ψ x
  simp only [coe_sub', coe_mul, Pi.sub_apply, Function.comp_apply, SchwartzMap.sub_a
    ContinuousLinearMap.zero_apply, SchwartzMap.zero_apply, positionOperator_apply]
  ring

lemma position_comp_commute {d : ℕ} (i j : Fin d) : ⊖[i] ∘L ⊖[j] = ⊖[j] ∘L ⊖[i] := by
  rw [← sub_eq_zero]
  exact position_commutation_position i j

lemma position_commutation_radiusRegPow (hε : 0 < ε) (i : Fin d) :
  {⊖[i], radiusRegPowOperator (d := d) ε s} = 0 := by
  dsimp only [Bracket.bracket]
  ext ψ x
  simp only [coe_sub', coe_mul, Pi.sub_apply, Function.comp_apply, SchwartzMap.sub_a
  simp only [positionOperator_apply, radiusRegPowOperator_apply hε]
  simp only [Complex.real_smul, ContinuousLinearMap.zero_apply, SchwartzMap.zero_app
  ring

lemma position_comp_radiusRegPow_commute (hε : 0 < ε) (i : Fin d) :
  ⊖[i] ∘L ⊖[ε,s] = ⊖[ε,s] ∘L ⊖[i] := by
  rw [← sub_eq_zero]
  exact position_commutation_radiusRegPow hε _

lemma radiusRegPow_commutation_radiusRegPow (hε : 0 < ε) :
  {radiusRegPowOperator (d := d) ε s, radiusRegPowOperator (d := d) ε t} = 0 := by
  dsimp only [Bracket.bracket]
  simp only [ContinuousLinearMap.mul_def, radiusRegPowOperator_comp_eq hε, add_comm,

/-
## Momentum / momentum commutators
-/

/-- Momentum operators commute:  $[p_i, p_j] = 0$ . -/
lemma momentum_commutation_momentum {d : ℕ} (i j : Fin d) : {⊖[i], ⊖[j]} = 0 := by
  dsimp only [Bracket.bracket]
  ext ψ x
  simp only [coe_sub', coe_mul, Pi.sub_apply, Function.comp_apply, SchwartzMap.sub_a
```

```

ContinuousLinearMap.zero_apply, SchwartzMap.zero_apply, momentumOperator
rw [Space.deriv_const_smul _ ?_, Space.deriv_const_smul _ ?_]
· rw [Space.deriv_commute _ (ψ.smooth _), sub_self]
· exact Space.deriv_differentiable (ψ.smooth _) i
· exact Space.deriv_differentiable (ψ.smooth _) j

lemma momentum_comp_commute {d : ℕ} (i j : Fin d) :  $\square[i] \circ_L \square[j] = \square[j] \circ_L$ 
  rw [← sub_eq_zero]
  exact momentum_commutation_momentum i j

lemma momentumSqr_commutation_momentum {d : ℕ} (i : Fin d) :
  {momentumOperatorSqr (d := d),  $\square[i]$ } = 0 := by
dsimp only [Bracket.bracket, momentumOperatorSqr]
rw [Finset.mul_sum, Finset.sum_mul, ← Finset.sum_sub_distrib]
conv_lhs =>
  enter [2, j]
  simp only [ContinuousLinearMap.mul_def]
  rw [comp_assoc]
  rw [momentum_comp_commute j i, ← comp_assoc]
  rw [momentum_comp_commute j i, comp_assoc]
  rw [sub_self]
  rw [Finset.sum_const_zero]

lemma momentumSqr_comp_momentum_commute {d : ℕ} (i : Fin d) :  $\square^2 \circ_L \square[i] =$ 
  rw [← sub_eq_zero]
  exact momentumSqr_commutation_momentum i

/-
## Position / momentum commutators
-/

/-- The canonical commutation relations:  $[x_i, p_j] = i\hbar \delta_{ij}$  -
/
lemma position_commutation_momentum {d : ℕ} (i j : Fin d) : { $\square[i], \square[j]$ } =
  (Complex.I *  $\hbar$  *  $\delta[i, j]$ ) • ContinuousLinearMap.id  $\mathbb{C} \square(\text{Space } d, \mathbb{C}) :=$  by
dsimp only [Bracket.bracket, kroneckerDelta]
ext ψ x
simp only [ContinuousLinearMap.smul_apply, SchwartzMap.smul_apply, coe_id,
  coe_sub', coe_mul, Pi.sub_apply, Function.comp_apply, SchwartzMap.sub_a]
rw [positionOperator_apply, momentumOperator_apply_fun]
rw [momentumOperator_apply, positionOperator_apply_fun]
simp only [neg_mul, Pi.smul_apply, smul_eq_mul, mul_neg, sub_neg_eq_add]

have h : (fun x ↦  $\uparrow(x \ i)$  * ψ x) = (fun (x : Space d) ↦ x i) • ψ := rfl
rw [h]
rw [Space.deriv_smul (by fun_prop) (SchwartzMap.differentiableAt ψ)]

```


1.88.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/COMMUTATION.LEAN

```

    rw [Space.deriv_component, ite_cond_symm j i]
    simp only [mul_add, Complex.real_smul, ite_smul, one_smul, zero_smul, mul_ite, mul_ite_mul, zero_mul]
    ring

lemma momentum_comp_position_eq (i j : Fin d) :  $\varphi[j] \circ_L \varphi[i] = \varphi[i] \circ_L \varphi[j] - (\text{Complex.I} * \hbar * \delta[i,j]) \cdot \text{ContinuousLinearMap.id } \mathbb{C} \varphi(\text{Space } d, \mathbb{C})$  =
  rw [← position_commutation_momentum]
  dsimp only [Bracket.bracket]
  simp only [ContinuousLinearMap.mul_def, sub_sub_cancel]

lemma position_position_commutation_momentum {d : ℕ} (i j k : Fin d) :  $\{\varphi[i] \circ_L \varphi[j] - (\text{Complex.I} * \hbar * \delta[i,k]) \cdot \varphi[j] + (\text{Complex.I} * \hbar * \delta[j,k]) \cdot \varphi[i]\} :=$  by
  rw [liebniz_lie]
  rw [position_commutation_momentum, position_commutation_momentum]
  rw [ContinuousLinearMap.comp_smul, ContinuousLinearMap.smul_comp]
  rw [id_comp, comp_id]
  rw [add_comm]

lemma position_commutation_momentum_momentum {d : ℕ} (i j k : Fin d) :  $\{\varphi[i], \varphi[j] \circ (\text{Complex.I} * \hbar * \delta[i,k]) \cdot \varphi[j] + (\text{Complex.I} * \hbar * \delta[i,j]) \cdot \varphi[k]\} :=$  by
  rw [lie_leibniz]
  rw [position_commutation_momentum, position_commutation_momentum]
  rw [ContinuousLinearMap.comp_smul, ContinuousLinearMap.smul_comp]
  rw [id_comp, comp_id]

lemma position_commutation_momentumSqr {d : ℕ} (i : Fin d) :  $\{\varphi[i], \varphi^2\} = (2 * \text{Complex.I} * \hbar) \cdot \varphi[i] :=$  by
  unfold momentumOperatorSqr
  rw [lie_sum]
  simp only [position_commutation_momentum_momentum]
  dsimp only [kroneckerDelta]
  simp only [mul_ite_zero, ite_zero_smul, Finset.sum_add_distrib, Finset.sum_ite_eq, Finset.mem_univ, ↯reduceIte]
  ext  $\psi$  x
  simp only [ContinuousLinearMap.add_apply, coe_smul', Pi.smul_apply, SchwartzMap.add_apply, SchwartzMap.smul_apply, smul_eq_mul]
  ring

lemma radiusRegPow_commutation_momentum (hε : 0 < ε) (i : Fin d) :
  {radiusRegPowOperator (d := d) ε s,  $\varphi[i]$ } = (s * Complex.I *  $\hbar$ ) *  $\varphi[\varepsilon, s - 2] \circ_L \varphi[i] :=$  by
  dsimp only [Bracket.bracket]
  ext  $\psi$  x
  simp only [coe_sub', coe_mul, Pi.sub_apply, Function.comp_apply, SchwartzMap.sub_apply, coe_comp', Pi.smul_apply, SchwartzMap.smul_apply, smul_eq_mul]

```

```

simp only [momentumOperator_apply, positionOperator_apply, radiusRegPowOp

have hne : ∀ x : Space d, ||x|| ^ 2 + ε ^ 2 ≠ 0 := by
  intro x
  apply ne_of_gt
  exact add_pos_of_nonneg_of_pos (sq_nonneg _) (sq_pos_of_pos hε)

have h : (fun x ↦ (||x|| ^ 2 + ε ^ 2) ^ (s / 2) • ψ x) =
  (fun (x : Space d) ↦ (||x|| ^ 2 + ε ^ 2) ^ (s / 2)) • ψ := rfl
have h' : ∂[i] (fun x ↦ (||x|| ^ 2 + ε ^ 2) ^ (s / 2)) =
  fun x ↦ s * (||x|| ^ 2 + ε ^ 2) ^ (s / 2 - 1) * x i := by
  trans ∂[i] ((fun x ↦ x ^ (s / 2)) ∘ (fun x ↦ ||x|| ^ 2 + ε ^ 2))
  · congr
  ext x
  rw [Space.deriv_eq, fderiv_comp]
  · simp only [fderiv_add_const, fderiv_norm_sq_apply, comp_smul, coe_smul,
    coe_innerSL_apply, Pi.smul_apply, Function.comp_apply, Space.inner_
    fderiv_eq_smul_deriv, smul_eq_mul, nsmul_eq_mul, Nat.cast_ofNat]
  rw [deriv_rpow_const]
  · simp only [deriv_id'', one_mul]
    ring
  · fun_prop
  · left
  exact hne _
  · exact Real.differentiableAt_rpow_const_of_ne (s / 2) (hne x)
  · exact Differentiable.differentiableAt (by fun_prop)

rw [h, Space.deriv_smul]
· rw [h']
  simp only [neg_mul, smul_neg, Complex.real_smul, Complex.ofReal_mul, su
  ring_nf
· refine DifferentiableAt.rpow ?_ (by fun_prop) (hne _)
  exact Differentiable.differentiableAt (by fun_prop)
· fun_prop

lemma momentum_comp_radiusRegPow_eq (hε : 0 < ε) (i : Fin d) :
  [][i] ∘L [][ε,s] = [][ε,s] ∘L [][i] - (s * Complex.I * ħ) • [][ε,s-
  2] ∘L [][i] := by
  rw [← radiusRegPow_commutation_momentum hε]
  dsimp only [Bracket.bracket]
  simp only [ContinuousLinearMap.mul_def, sub_sub_cancel]

lemma radiusRegPow_commutation_momentumSqr (hε : 0 < ε) :
  [radiusRegPowOperator (d := d) ε s, momentumOperatorSqr (d := d)] =
  (2 * s * Complex.I * ħ) • [][ε,s-2] ∘L ∑ i, [][i] ∘L [][i]
  + (s * ħ ^ 2) • ((d + s - 2) • [][ε,s-2] - (ε ^ 2 * (s - 2)) • [][ε,s-

```

1.88.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/COMMUTATION.LEAN

```
4)) := by
  unfold momentumOperatorSqr
  rw [lie_sum]
  conv_lhs =>
    enter [2, i]
    rw [lie_leibniz, radiusRegPow_commutation_momentum hε]
    rw [comp_smul, ← comp_assoc, momentum_comp_radiusRegPow_eq hε]
    rw [sub_comp, comp_assoc, momentum_comp_position_eq]
    simp only [kroneckerDelta, ↓reduceIte, mul_one]
  simp only [smul_comp, comp_sub, comp_smul, comp_id, smul_sub, comp_assoc,
    Finset.sum_add_distrib, Finset.sum_sub_distrib, ← Finset.smul_sum, Finset.sum_co
    Finset.card_univ, Fintype.card_fin, ← ContinuousLinearMap.comp_finset_sum]
  rw [positionOperatorSqr_eq hε, comp_sub, radiusRegPowOperator_comp_eq hε, comp_smu
  rw [← Nat.cast_smul_eq_nsmul ℂ]
  ext ψ x
  simp only [Complex.ofReal_sub, Complex.ofReal_ofNat, sub_add_cancel, coe_sub', Pi.
    ContinuousLinearMap.add_apply, coe_smul', coe_comp', coe_sum', Pi.smul_apply,
    Function.comp_apply, Finset.sum_apply, map_sum, SchwartzMap.sub_apply, SchwartzM
    SchwartzMap.smul_apply, SchwartzMap.sum_apply, smul_eq_mul, Complex.real_smul,
    Complex.ofReal_pow, Complex.ofReal_add, Complex.ofReal_natCast, Complex.ofReal_m
  ring_nf
  rw [Complex.I_sq]
  ring

/-
## Angular momentum / position commutators
-/

lemma angularMomentum_commutation_position {d : ℕ} (i j k : Fin d) : {□[i,j], □[k]}
  (Complex.I * ħ * δ[i,k]) • □[j] - (Complex.I * ħ * δ[j,k]) • □[i] := by
  unfold angularMomentumOperator
  rw [sub_lie]
  rw [leibniz_lie, leibniz_lie]
  rw [position_commutation_position, position_commutation_position]
  rw [← lie_skew, position_commutation_momentum]
  rw [← lie_skew, position_commutation_momentum]
  rw [kroneckerDelta_symm k i, kroneckerDelta_symm k j]
  simp only [ContinuousLinearMap.comp_neg, ContinuousLinearMap.comp_smul, comp_id, z
    add_zero, add_comm, sub_neg_eq_add, ← sub_eq_add_neg]

lemma angularMomentum_commutation_radiusRegPow (hε : 0 < ε) (i j : Fin d) :
  {□[i,j], radiusRegPowOperator (d := d) ε s} = 0 := by
  dsimp only [Bracket.bracket]
  unfold angularMomentumOperator
  simp only [sub_mul, ContinuousLinearMap.mul_def, ContinuousLinearMap.comp_assoc]
  repeat rw [momentum_comp_radiusRegPow_eq hε]
```

```

simp only [comp_sub, comp_smulsl, RingHom.id_apply, ← ContinuousLinearMap
repeat rw [position_comp_radiusRegPow_commute hε]
simp only [ContinuousLinearMap.comp_assoc]
rw [position_comp_commute]
simp only [sub_sub_sub_cancel_right, sub_self]

lemma angularMomentumSqr_commutation_radiusRegPow (hε : 0 < ε) :
  {angularMomentumOperatorSqr (d := d), radiusRegPowOperator (d := d) ε s
  unfold angularMomentumOperatorSqr
  simp only [sum_lie, smul_lie, leibniz_lie, angularMomentum_commutation_ra
  comp_zero, zero_comp, add_zero, smul_zero, Finset.sum_const_zero]

/-
## Angular momentum / momentum commutators
-/

lemma angularMomentum_commutation_momentum {d : ℕ} (i j k : Fin d) : {[] [i,j]
  (Complex.I * ħ * δ[i,k]) • [] [j] - (Complex.I * ħ * δ[j,k]) • [] [i] := by
  unfold angularMomentumOperator
  rw [sub_lie]
  rw [leibniz_lie, leibniz_lie]
  rw [momentum_commutation_momentum, momentum_commutation_momentum]
  rw [position_commutation_momentum, position_commutation_momentum]
  simp only [ContinuousLinearMap.smul_comp, id_comp, comp_zero, zero_add]

lemma momentum_comp_angularMomentum_eq {d : ℕ} (i j k : Fin d) : [] [k] •L [] [
  [] [i,j] •L [] [k] - (Complex.I * ħ * δ[i,k]) • [] [j] + (Complex.I * ħ * δ[j
  rw [← sub_eq_zero, sub_add]
  rw [← angularMomentum_commutation_momentum]
  dsimp only [Bracket.bracket]
  simp only [ContinuousLinearMap.mul_def, sub_sub_cancel, sub_eq_zero]

lemma angularMomentum_commutation_momentumSqr {d : ℕ} (i j : Fin d) :
  {[] [i,j], momentumOperatorSqr (d := d)} = 0 := by
  unfold momentumOperatorSqr
  conv_lhs =>
  rw [lie_sum]
  enter [2, k]
  rw [lie_leibniz, angularMomentum_commutation_momentum]
  simp only [comp_sub, comp_smulsl, RingHom.id_apply, sub_comp, smul_comp]
  rw [momentum_comp_commute _ i, momentum_comp_commute j _]
  dsimp only [kroneckerDelta]
  simp only [Finset.sum_add_distrib, Finset.sum_sub_distrib, mul_ite, mul_z
  zero_smul, Finset.sum_ite_eq, Finset.mem_univ, ↓reduceIte, sub_self, ad

lemma momentumSqr_comp_angularMomentum_commute {d : ℕ} (i j : Fin d) :

```

1.88.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/COMMUTATION.LEAN

```

    []2 •L [][i,j] = [][i,j] •L []2 := by
  apply Eq.symm
  rw [← sub_eq_zero]
  exact angularMomentum_commutation_momentumSqr i j

lemma angularMomentumSqr_commutation_momentumSqr {d : ℕ} :
  {angularMomentumOperatorSqr (d := d), momentumOperatorSqr (d := d)} = 0 := by
  unfold angularMomentumOperatorSqr
  simp only [smul_lie, sum_lie, leibniz_lie]
  simp [angularMomentum_commutation_momentumSqr]

/-
### Angular momentum / angular momentum commutators
-/

lemma angularMomentum_commutation_angularMomentum {d : ℕ} (i j k l : Fin d) : {[] [i,j]
  (Complex.I * ħ * δ[i,k]) • [][j,l] - (Complex.I * ħ * δ[i,l]) • [][j,k]
  - (Complex.I * ħ * δ[j,k]) • [][i,l] + (Complex.I * ħ * δ[j,l]) • [][i,k] := by
  nth_rw 2 [angularMomentumOperator]
  rw [lie_sub]
  rw [lie_leibniz, lie_leibniz]
  rw [angularMomentum_commutation_momentum, angularMomentum_commutation_position]
  rw [angularMomentum_commutation_momentum, angularMomentum_commutation_position]
  dsimp only [angularMomentumOperator, kroneckerDelta]
  simp only [ContinuousLinearMap.comp_sub, ContinuousLinearMap.sub_comp,
    ContinuousLinearMap.comp_smul, ContinuousLinearMap.smul_comp]
  ext ψ x
  simp only [mul_ite, mul_one, mul_zero, ite_smul, zero_smul, coe_sub', Pi.sub_apply,
    ContinuousLinearMap.add_apply, SchwartzMap.sub_apply, SchwartzMap.add_apply, smul
  ring

lemma angularMomentumSqr_commutation_angularMomentum {d : ℕ} (i j : Fin d) :
  {angularMomentumOperatorSqr (d := d), [] [i,j]} = 0 := by
  unfold angularMomentumOperatorSqr
  conv_lhs =>
    simp only [smul_lie, sum_lie, leibniz_lie, angularMomentum_commutation_angularMo
  dsimp only [kroneckerDelta]
  simp only [comp_add, comp_sub, add_comp, sub_comp, comp_smul, smul_comp, mul_ite,
    mul_one]
  simp only [ite_smul, zero_smul]

-- Split into individual terms to do one of the sums, then recombine
simp only [Finset.sum_add_distrib, Finset.sum_sub_distrib, Finset.sum_ite_irrel,
  Finset.sum_const_zero, Finset.sum_ite_eq', Finset.mem_univ, ↑reduceIte]
simp only [← Finset.sum_add_distrib, ← Finset.sum_sub_distrib]

```

```

ext  $\psi$  x
simp only [angularMomentumOperator_antisymm _ i, angularMomentumOperator_
  neg_comp, comp_neg, neg_neg, smul_neg, sub_neg_eq_add]
simp only [ContinuousLinearMap.sum_apply, ContinuousLinearMap.add_apply,
  ContinuousLinearMap.sub_apply, ContinuousLinearMap.smul_apply, Continuo
  ContinuousLinearMap.neg_apply, ContinuousLinearMap.zero_apply, Schwartz
  SchwartzMap.sum_apply, SchwartzMap.sub_apply, SchwartzMap.smul_apply, S
  SchwartzMap.zero_apply]
ring_nf
rw [Finset.sum_const_zero, smul_zero]

end
end QuantumMechanics

```

1.89 PhysLean/QuantumMechanics/DDimensions/Operators/Momentum.1

```

/-
Copyright (c) 2026 Gregory J. Loges. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Gregory J. Loges
-/
import PhysLean.SpaceAndTime.Space.Derivatives.Basic
import PhysLean.QuantumMechanics.PlanckConstant
/-!

# Momentum vector operator

In this module we define:
- The momentum operator on Schwartz maps, component-wise.
- The momentum squared operator on Schwartz maps.

-/

namespace QuantumMechanics
noncomputable section
open Constants
open Space
open ContDiff SchwartzMap

/-- Component `i` of the momentum operator is the continuous linear map
from  $\square(\text{Space } d, \mathbb{C})$  to itself which maps  $\psi$  to  $-i\hbar \partial_i \psi$ . -
/
def momentumOperator {d :  $\mathbb{N}$ } (i : Fin d) :  $\square(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \square(\text{Space } d, \mathbb{C})$ 
  (- Complex.I *  $\hbar$ ) • (SchwartzMap.evalCLM  $\mathbb{C}(\text{Space } d) \mathbb{C}(\text{basis } i)$ ) ◦ L

```

1.90.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/POSITION.lean

```

(SchwartzMap.fderivCLM  $\mathbb{C}$  (Space d)  $\mathbb{C}$ )

@[inherit_doc momentumOperator]
macro "[[" i:term "]" : term => `(momentumOperator $i)

lemma momentumOperator_apply_fun {d :  $\mathbb{N}$ } (i : Fin d) ( $\psi$  :  $\square$ (Space d,  $\mathbb{C}$ )) :
   $\square[i] \psi = (- \text{Complex.I} * \hbar) \cdot \partial[i] \psi := \text{rfl}$ 

lemma momentumOperator_apply {d :  $\mathbb{N}$ } (i : Fin d) ( $\psi$  :  $\square$ (Space d,  $\mathbb{C}$ )) (x : Space d) :
   $\square[i] \psi x = - \text{Complex.I} * \hbar * \partial[i] \psi x := \text{rfl}$ 

/-- The square of the momentum operator,  $\square^2 = \sum_i \square_i \circ \square_i$ . -
/
def momentumOperatorSqr {d :  $\mathbb{N}$ } :  $\square$ (Space d,  $\mathbb{C}$ )  $\rightarrow$  L[ $\mathbb{C}$ ]  $\square$ (Space d,  $\mathbb{C}$ ) :=  $\sum i, \square[i] \circ L$ 

@[inherit_doc momentumOperatorSqr]
notation " $\square^2$ " => momentumOperatorSqr

lemma momentumOperatorSqr_apply {d :  $\mathbb{N}$ } ( $\psi$  :  $\square$ (Space d,  $\mathbb{C}$ )) (x : Space d) :
   $\square^2 \psi x = \sum i, \square[i] (\square[i] \psi) x := \text{by}$ 
  dsimp only [momentumOperatorSqr]
  rw [← SchwartzMap.coe_coeHom]
  simp only [ContinuousLinearMap.coe_sum', ContinuousLinearMap.coe_comp', Finset.sum
    Function.comp_apply, map_sum]

end
end QuantumMechanics

```

1.90 PhysLean/QuantumMechanics/DDimensions/Operators/Position.lean

```

/-
Copyright (c) 2026 Gregory J. Loges. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Gregory J. Loges
-/
import PhysLean.SpaceAndTime.Space.Derivatives.Basic
/-!

# Position operators

In this module we define:
- The position vector operator on Schwartz maps, component-
wise.
- The (regularized) real powers of the radius operator on Schwartz maps.

```

```

-/

namespace QuantumMechanics
noncomputable section
open Space
open Function SchwartzMap ContDiff

/-
## Position vector operator
-/

/-- Component `i` of the position operator is the continuous linear map
from  $\square(\text{Space } d, \mathbb{C})$  to itself which maps  $\psi$  to  $x_i \psi$ . -
/
def positionOperator (i : Fin d) :  $\square(\text{Space } d, \mathbb{C}) \rightarrow L[\mathbb{C}] \square(\text{Space } d, \mathbb{C}) :=$ 
  SchwartzMap.smulLeftCLM  $\mathbb{C}$  (Complex.ofReal  $\circ$  coordCLM i)

@[inherit_doc positionOperator]
macro " $\square$ [" i:term "]" : term => `(positionOperator $i)

lemma positionOperator_apply_fun (i : Fin d) ( $\psi$  :  $\square(\text{Space } d, \mathbb{C})$ ) :
   $\square[i] \psi = (\text{fun } x \mapsto x \ i * \psi \ x) := \text{by}$ 
  unfold positionOperator
  ext x
  rw [SchwartzMap.smulLeftCLM_apply_apply]
  · rw [Function.comp_apply, smul_eq_mul]
  · rw [coordCLM_apply, coord_apply]
  · fun_prop

lemma positionOperator_apply (i : Fin d) ( $\psi$  :  $\square(\text{Space } d, \mathbb{C})$ ) (x : Space d) :
   $\square[i] \psi \ x = x \ i * \psi \ x := \text{by rw [positionOperator_apply_fun]}$ 

-/

## Radius operator

-/
TODO "ZGCNP" "Incorporate normRegularizedPow into Space.Norm"

/-- Power of regularized norm,  $(\|x\|^2 + \varepsilon^2)^{(s/2)}$  -/
private def normRegularizedPow ( $\varepsilon \ s$  :  $\mathbb{R}$ ) : Space d  $\rightarrow \mathbb{R} :=$ 
  fun x  $\mapsto (\|x\| ^ 2 + \varepsilon ^ 2) ^ (s / 2)$ 

private lemma normRegularizedPow_hasTemperateGrowth (h $\varepsilon$  :  $0 < \varepsilon$ ) :
  HasTemperateGrowth (normRegularizedPow (d := d)  $\varepsilon \ s) := \text{by}$ 

```


1.90.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/POSITION **LEAN**

```

-- Write `normRegularizedPow` as the composition of three simple functions
-- to take advantage of `hasTemperateGrowth_one_add_norm_sq_rpow`
let f1 := fun (x : ℝ) ↦ (ε ^ 2) ^ (s / 2) * x
let f2 := fun (x : Space d) ↦ (1 + ||x|| ^ 2) ^ (s / 2)
let f3 := fun (x : Space d) ↦ ε-1 • x

have h123 : normRegularizedPow (d := d) ε s = f1 ∘ f2 ∘ f3 := by
  unfold normRegularizedPow f1 f2 f3
  ext x
  simp only [Function.comp_apply, norm_smul, norm_inv, Real.norm_eq_abs]
  rw [← Real.mul_rpow (sq_nonneg _) ?_]
  · rw [mul_pow, mul_add, mul_one, ← mul_assoc, inv_pow, sq_abs]
    rw [IsUnit.mul_inv_cancel ?_]
    · rw [one_mul, add_comm]
    · rw [pow_two, isUnit_mul_self_iff, isUnit_iff_ne_zero]
      exact ne_of_gt hε
  · exact add_nonneg (zero_le_one' _) (sq_nonneg _)

rw [h123]
fun_prop

/-- The radius operator to power `s`, regularized by `ε ≠ 0`, is the continuous linear
    from `(Space d, ℂ)` to itself which maps `ψ` to `(||x||2 + ε2)(s/2) • ψ`. -
/
def radiusRegPowOperator (ε s : ℝ) : (Space d, ℂ) → L[ℂ] (Space d, ℂ) :=
  SchwartzMap.smulLeftCLM ℂ (Complex.ofReal ∘ normRegularizedPow ε s)

@[inherit_doc radiusRegPowOperator]
macro "⌈[" ε:term "," s:term "]" : term => `(radiusRegPowOperator $ε $s)

lemma radiusRegPowOperator_apply_fun (hε : 0 < ε) :
  ⌈[ε,s] ψ = fun x ↦ (||x|| ^ 2 + ε ^ 2) ^ (s / 2) • ψ x := by
  unfold radiusRegPowOperator
  ext x
  rw [smulLeftCLM_apply_apply]
  · unfold normRegularizedPow
    rw [comp_apply, smul_eq_mul, Complex.real_smul]
  · exact HasTemperateGrowth.comp (by fun_prop) (normRegularizedPow_hasTemperateGrow

lemma radiusRegPowOperator_apply (hε : 0 < ε) :
  ⌈[ε,s] ψ x = (||x|| ^ 2 + ε ^ 2) ^ (s / 2) • ψ x := by
  rw [radiusRegPowOperator_apply_fun hε]

lemma radiusRegPowOperator_comp_eq (hε : 0 < ε) (s t : ℝ) :
  (radiusRegPowOperator (d := d) ε s) ∘L ⌈[ε,t] = ⌈[ε,s+t] := by
  unfold radiusRegPowOperator

```

```

ext ψ x
simp only [ContinuousLinearMap.coe_comp', comp_apply]
repeat rw [smulLeftCLM_apply_apply ?_]
· unfold normRegularizedPow
  simp only [comp_apply, smul_eq_mul]
  rw [← mul_assoc, ← Complex.ofReal_mul]
  rw [← Real.rpow_add]
  · congr
    ring
  · exact add_pos_of_nonneg_of_pos (sq_nonneg _) (sq_pos_of_pos hε)
repeat exact HasTemperateGrowth.comp (by fun_prop) (normRegularizedPow_ha

lemma radiusRegPowOperator_zero (hε : 0 < ε) :
  [ε, 0] = ContinuousLinearMap.id ℂ (Space d, ℂ) := by
  ext ψ x
  rw [radiusRegPowOperator_apply hε, zero_div, Real.rpow_zero, one_smul,
    ContinuousLinearMap.coe_id', id_eq]

lemma positionOperatorSqr_eq (hε : 0 < ε) : ∑ i, [i] •L [i] =
  [ε, 2] - ε ^ 2 • ContinuousLinearMap.id ℂ (Space d, ℂ) := by
  ext ψ x
  simp only [ContinuousLinearMap.coe_sum', Finset.sum_apply, SchwartzMap.su
    ContinuousLinearMap.comp_apply, ContinuousLinearMap.sub_apply, Schwartz
    ContinuousLinearMap.smul_apply, ContinuousLinearMap.id_apply, SchwartzM
  simp only [positionOperator_apply_fun, radiusRegPowOperator_apply_fun hε]
  simp only [← mul_assoc, ← Finset.sum_mul, ← Complex.ofReal_mul]
  rw [div_self (by norm_num), Real.rpow_one, ← sub_smul, add_sub_cancel_rig
  rw [Space.norm_sq_eq, Complex.real_smul, Complex.ofReal_sum]
  simp only [pow_two]

end
end QuantumMechanics

```

1.91 PhysLean/QuantumMechanics/DDimensions/Operators/Unbounded.

```

/-
Copyright (c) 2026 Gregory J. Loges. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Gregory J. Loges
-/
import PhysLean.Mathematics.InnerProductSpace.Submodule
import PhysLean.QuantumMechanics.DDimenstions.SpaceDHilbertSpace.SchwartzSub
/-!

```

1.91.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNBOUNDED. LEAN

Unbounded operators

In this file we define

- ``UnboundedOperator``: an unbounded operator with domain a submodule of a generic Hilbert space. All unbounded operators are assumed to be both densely defined and closable.
- The closure, ``UnboundedOperator.closure``, and adjoint, ``UnboundedOperator.adjoint``. That ``U† = U.adjoint``. That ``U†`` is densely defined is guaranteed by the closability of `U`.
- The concept of a generalized eigenvector in ``IsGeneralizedEigenvector``.

We prove some basic relations, making use of the density and closability assumptions

- ``U.closure† = U†`` in ``closure_adjoint_eq_adjoint``
- ``U†† = U.closure`` in ``adjoint_adjoint_eq_closure``

References

- K. Schmüdgen, (2012). "Unbounded self-adjoint operators on Hilbert space" (Vol. 26) <https://doi.org/10.1007/978-94-007-4753-1>

-/

namespace QuantumMechanics

open LinearPMap Submodule

`/-- An `UnboundedOperator` is a linear map from a submodule of the Hilbert space to the Hilbert space, assumed to be both densely defined and closable. -`

`/`

`@[ext]`

`structure UnboundedOperator`

`(HS : Type*) [NormedAddCommGroup HS] [InnerProductSpace ℂ HS] [CompleteSpace HS]`
`extends LinearPMap ℂ HS HS where`

`/-- The domain of an unbounded operator is dense in the Hilbert space. -`

`/`

`dense_domain : Dense (domain : Set HS)`

`/-- An unbounded operator is closable. -/`

`is_closable : toLinearPMap.IsClosable`

namespace UnboundedOperator

variable

`{HS : Type*} [NormedAddCommGroup HS] [InnerProductSpace ℂ HS] [CompleteSpace HS]`
`(U : UnboundedOperator HS)`

`lemma ext' (U T : UnboundedOperator HS) (h : U.toLinearPMap = T.toLinearPMap) : U =`
`apply UnboundedOperator.ext`

`· exact toSubMulAction_inj.mp (congrArg toSubMulAction (congrArg domain h))`

```

· exact congr_arg_heq toFun h

lemma ext_iff' (U T : UnboundedOperator HS) : U = T ↔ U.toLinearPMap = T.to
  refine {?_, UnboundedOperator.ext' U T}
  intro h
  rw [h]

/-!
### Construction of unbounded operators
-/

variable {E : Submodule ℂ HS} {hE : Dense (E : Set HS)}

/-- An `UnboundedOperator` constructed from a symmetric linear map on a den
/
def ofSymmetric (f : E →ℂ E) (hf : f.IsSymmetric) : UnboundedOperator HS
  toLinearPMap := LinearPMap.mk E (E.subtype ∘ℂ f)
  dense_domain := hE
  is_closable := by
    refine isClosable_iff_exists_closed_extension.mpr ?_
    use (LinearPMap.mk E (E.subtype ∘ℂ f))†
    exact ⟨adjoint_isClosed hE, IsFormalAdjoint.le_adjoint hE hf⟩

@[simp]
lemma ofSymmetric_apply {f : E →ℂ E} {hf : f.IsSymmetric} (ψ : E) :
  (ofSymmetric (hE := hE) f hf).toLinearPMap ψ = E.subtypeL (f ψ) := rfl

/-!
### Closure
-/

section Closure

/-- The closure of an unbounded operator. -/
noncomputable def closure : UnboundedOperator HS where
  toLinearPMap := U.toLinearPMap.closure
  dense_domain := Dense.mono (HasCore.le_domain (closureHasCore U.toLinearP
  is_closable := IsClosed.isClosable (IsClosable.closure_isClosed U.is_clos

@[simp]
lemma closure_toLinearPMap : U.closure.toLinearPMap = U.toLinearPMap.closur

/-- An unbounded operator is closed iff the graph of its defining LinearPMap
/
def IsClosed : Prop := U.toLinearPMap.IsClosed

```

1.91.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/OPERATORS/UNBOUNDED. LEAN

```
lemma closure_isClosed : U.closure.IsClosed := IsClosable.closure_isClosed U.is_closable
```

```
end Closure
```

```
/-!  
### Adjoints  
-/  

```

```
section Adjoints
```

```
open InnerProductSpaceSubmodule
```

```
/-- The adjoint of a densely defined, closable `LinearPMap` is densely defined. -  
/
```

```
lemma adjoint_isClosable_dense (f : LinearPMap  $\mathbb{C}$  HS HS) (h_dense : Dense (f.domain : Set HS))  
  (h_closable : f.IsClosable) : Dense (f.adjoint.domain : Set HS) := by  
  by_contra hd  
  have :  $\exists x \in f.adjoint.domain, x \neq 0$  := by  
    apply not_forall.mp at hd  
    rcases hd with ⟨y, hy⟩  
    have hnetop : f.adjoint.domain  $\neq \top$  := by  
      rw [orthogonal_orthogonal_eq_closure]  
      exact Ne.symm (ne_of_mem_of_not_mem' trivial hy)  
    have hnebot : f.adjoint.domain  $\neq \perp$  := by  
      by_contra  
      apply hnetop  
      rwa [orthogonal_eq_top_iff]  
      exact exists_mem_ne_zero_of_ne_bot hnebot  
    rcases this with ⟨x, hx, hx'⟩  
    apply hx'  
    apply graph_fst_eq_zero_snd f.closure _ rfl  
    rw [← IsClosable.graph_closure_eq_closure_graph h_closable,  
      mem_submodule_closure_iff_mem_submoduleToLp_closure,  
      ← orthogonal_orthogonal_eq_closure,  
      ← mem_submodule_adjoint_adjoint_iff_mem_submoduleToLp_orthogonal_orthogonal,  
      ← LinearPMap.adjoint_graph_eq_graph_adjoint h_dense,  
      mem_submodule_adjoint_iff_mem_submoduleToLp_orthogonal]  
    rintro ⟨y, Uy⟩ hy  
    simp only [neg_zero, WithLp.prod_inner_apply, inner_zero_right, add_zero]  
    exact hx y (mem_domain_of_mem_graph hy)
```

```
/-- The adjoint of an unbounded operator, denoted as `U†`. -  
/
```

```
noncomputable def adjoint : UnboundedOperator HS where  
  toLinearPMap := U.toLinearPMap.adjoint  
  dense_domain := adjoint_isClosable_dense U.toLinearPMap U.dense_domain U.is_closable
```

```

    is_closable := IsClosed.isClosable (adjoint_isClosed U.dense_domain)

@[inherit_doc]
scoped postfix:1024 "†" => UnboundedOperator.adjoint

noncomputable instance instStar : Star (UnboundedOperator HS) where
  star := fun U => U.adjoint

@[simp]
lemma adjoint_toLinearPMap : U†.toLinearPMap = U.toLinearPMap† := rfl

lemma isSelfAdjoint_def : IsSelfAdjoint U ↔ U† = U := Iff.rfl

lemma isSelfAdjoint_iff : IsSelfAdjoint U ↔ IsSelfAdjoint U.toLinearPMap :=
  rw [isSelfAdjoint_def, LinearPMap.isSelfAdjoint_def, ← adjoint_toLinearPMap,
    UnboundedOperator.ext_iff']

lemma adjoint_isClosed : (U†).IsClosed := LinearPMap.adjoint_isClosed U.dense_domain

lemma closure_adjoint_eq_adjoint : U.closure† = U† := by
  -- Reduce to statement about graphs using density and closability assumption
  apply UnboundedOperator.ext'
  apply LinearPMap.eq_of_eq_graph
  rw [adjoint_toLinearPMap, adjoint_graph_eq_graph_adjoint U.closure.dense_domain]
  rw [adjoint_toLinearPMap, adjoint_graph_eq_graph_adjoint U.dense_domain]
  rw [closure_toLinearPMap, ← IsClosable.graph_closure_eq_closure_graph U.isClosable]
  ext f
  rw [mem_submodule_closure_adjoint_iff_mem_submoduleToLp_closure_orthogonal_closure,
    mem_submodule_adjoint_iff_mem_submoduleToLp_orthogonal_closure]

lemma adjoint_adjoint_eq_closure : U†† = U.closure := by
  -- Reduce to statement about graphs using density and closability assumption
  apply UnboundedOperator.ext'
  apply LinearPMap.eq_of_eq_graph
  rw [adjoint_toLinearPMap, adjoint_graph_eq_graph_adjoint U†.dense_domain]
  rw [adjoint_toLinearPMap, adjoint_graph_eq_graph_adjoint U.dense_domain]
  rw [closure_toLinearPMap, ← IsClosable.graph_closure_eq_closure_graph U.isClosable]
  ext f
  rw [mem_submodule_adjoint_adjoint_iff_mem_submoduleToLp_orthogonal_orthogonal_closure,
    mem_submodule_closure_iff_mem_submoduleToLp_orthogonal_closure]

end Adjoints

/-!
### Generalized eigenvectors
-/

```

1.92.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/SPACEDHILBERTSPACE/BASIC.LEAN

```

/-- A map `F : D(U) → L[ℂ] ℂ` is a generalized eigenvector of an unbounded operator
    if there is an eigenvalue `c` such that for all `ψ ∈ D(U)`, `F (U ψ) = c • F ψ`. -
/
def IsGeneralizedEigenvector (F : U.domain → L[ℂ] ℂ) (c : ℂ) : Prop :=
  ∀ ψ : U.domain, ∃ ψ' : U.domain, ψ' = U.toFun ψ ∧ F ψ' = c • F ψ

lemma isGeneralizedEigenvector_ofSymmetric_iff
  {f : E → L[ℂ] E} (hf : f.IsSymmetric) (F : E → L[ℂ] ℂ) (c : ℂ) :
  IsGeneralizedEigenvector (ofSymmetric (hE := hE) f hf) F c ↔ ∀ ψ : E, F (f ψ) =
  constructor <=> intro h ψ
  · obtain ⟨ψ', hψ', hψ''⟩ := h ψ
    apply SetLike.coe_eq_coe.mp at hψ'
    subst hψ'
    exact hψ''
  · use f ψ
    exact ⟨by simp, h ψ⟩

end UnboundedOperator
end QuantumMechanics

```

1.92 PhysLean/QuantumMechanics/DDimensions/SpaceDHilbertSpace/Basic.lean

```

/-
Copyright (c) 2026 Gregory J. Loges. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Gregory J. Loges
-/
import Mathlib.Analysis.InnerProductSpace.Dual
import Mathlib.MeasureTheory.Function.L2Space
import PhysLean.SpaceAndTime.Space.Basic
/-!

# Hilbert space for quantum mechanics on Space d

-/

namespace QuantumMechanics

noncomputable section

/-- The Hilbert space for single-particle quantum mechanics on `Space d` is defined
    `L²(Space d, ℂ)`, the space of almost-everywhere equal equivalence classes of square
    integrable

```

```

    functions from `Space d` to ` $\mathbb{C}$ `. -/
abbrev SpaceDHilbertSpace (d :  $\mathbb{N}$ ) := MeasureTheory.Lp ( $\alpha$  := Space d)  $\mathbb{C}$  2

namespace SpaceDHilbertSpace
open MeasureTheory
open InnerProductSpace

/-- The anti-linear map from the Hilbert space to its dual. -
/
def toBra {d :  $\mathbb{N}$ } : SpaceDHilbertSpace d  $\rightarrow_{\text{sl}}$  [starRingEnd  $\mathbb{C}$ ] (StrongDual  $\mathbb{C}$  (
    toDual  $\mathbb{C}$  (SpaceDHilbertSpace d)

@[simp]
lemma toBra_apply (f g : SpaceDHilbertSpace d) : toBra f g =  $\langle f, g \rangle_{\mathbb{C}}$  := rfl

/-- The anti-linear map `toBra` taking a ket to its corresponding bra is surjective. -
/
lemma toBra_surjective : Function.Surjective (toBra (d := d)) :=
    (toDual  $\mathbb{C}$  (SpaceDHilbertSpace d)).surjective

/-- The anti-linear map `toBra` taking a ket to its corresponding bra is injective. -
/
lemma toBra_injective : Function.Injective (toBra (d := d)) := by
    intro f g h
    simpa [toBra] using h

/-!
## Member of the Hilbert space as a property
-/

/-- The proposition `MemHS f` for a function `f : Space d  $\rightarrow$   $\mathbb{C}$ ` is defined
    to be true if the function `f` can be lifted to the Hilbert space. -
/
def MemHS (f : Space d  $\rightarrow$   $\mathbb{C}$ ) : Prop := MemLp f 2

lemma aeStronglyMeasurable_of_memHS {f : Space d  $\rightarrow$   $\mathbb{C}$ } (h : MemHS f) : AEStronglyMeasurable f :=
    MemLp.aeStronglyMeasurable h

/-- A function `f` satisfies `MemHS f` if and only if it is almost everywhere
    strongly measurable and square integrable. -/
lemma memHS_iff {f : Space d  $\rightarrow$   $\mathbb{C}$ } : MemHS f  $\leftrightarrow$ 
    AEStronglyMeasurable f  $\wedge$  Integrable (fun x  $\mapsto$   $\|f x\|^2$ ) := by
    rw [MemHS, MemLp, and_congr_right]
    intro h
    rw [eLpNorm_lt_top_iff_lintegral_rpow_enorm_lt_top
        (NeZero.ne' 2).symm ENNReal.top_ne_ofNat.symm]

```


1.92.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/SPACEDHILBERTSPACE/BASIC.LEAN

```

simp only [ENNReal.toReal_ofNat, ENNReal.rpow_ofNat]
have h' : AESTronglyMeasurable (fun x ↦ ||f x|| ^ 2) :=
  AESTronglyMeasurable.pow (AESTronglyMeasurable.norm h) 2
simp [Integrable, h', HasFiniteIntegral]

@[simp]
lemma zero_memHS : MemHS (d := d) 0 := by
  rw [memHS_iff]
  simp only [Pi.zero_apply, norm_zero, ne_eq, OfNat.ofNat_ne_zero, not_false_eq_true,
    integrable_zero, and_true]
  fun_prop

@[simp]
lemma zero_fun_memHS : MemHS (fun _ : Space d ↦ (0 : ℂ)) := zero_memHS

lemma memHS_add {f g : Space d → ℂ} (hf : MemHS f) (hg : MemHS g) : MemHS (f + g) :=
  MemLp.add hf hg

lemma memHS_const_smul {f : Space d → ℂ} {c : ℂ} (hf : MemHS f) : MemHS (c • f) :=
  MemLp.const_smul hf c

lemma memHS_of_ae {g : Space d → ℂ} (f : Space d → ℂ) (hf : MemHS f) (hfg : f =ᵐ[vol]
  MemHS g := MemLp.ae_eq hfg hf

/-!
## Construction of elements of the Hilbert space
-/

lemma aeEqFun_mk_mem_iff (f : Space d → ℂ) (hf : AESTronglyMeasurable f volume) :
  AEEqFun.mk f hf ∈ SpaceDHilbertSpace d ↔ MemHS f := by
  rw [Lp.mem_Lp_iff_memLp]
  exact memLp_congr_ae (AEEqFun.coeFn_mk f hf)

/-- Given a function `f : Space d → ℂ` such that `MemHS f` is true via `hf`,
  `SpaceDHilbertSpace.mk f` is the element of the Hilbert space defined by `f`. -
/
def mk {f : Space d → ℂ} (hf : MemHS f) : SpaceDHilbertSpace d :=
  ⟨AEEqFun.mk f hf.1, (aeEqFun_mk_mem_iff f hf.1).mpr hf⟩

lemma coe_hilbertSpace_memHS (f : SpaceDHilbertSpace d) : MemHS (f : Space d → ℂ) :=
  rw [← aeEqFun_mk_mem_iff f (Lp.aestronglyMeasurable f)]
  have hf : f = AEEqFun.mk f (Lp.aestronglyMeasurable f) := (AEEqFun.mk_coeFn _).symm
  exact hf ▸ f.2

lemma mk_surjective (f : SpaceDHilbertSpace d) :
  ∃ (g : Space d → ℂ) (hg : MemHS g), mk hg = f := by

```

```

use f, coe_hilbertSpace_memHS f
simp [mk]

lemma coe_mk_ae {f : Space d → ℂ} (hf : MemHS f) : (mk hf) ==[volume] f :=
  AEEqFun.coeFn_mk f hf.1

lemma inner_mk_mk {f g : Space d → ℂ} (hf : MemHS f) (hg : MemHS g) :
  [mk hf, mk hg]_ℂ = ∫ x : Space d, starRingEnd ℂ (f x) * g x := by
  apply integral_congr_ae
  filter_upwards [coe_mk_ae hf, coe_mk_ae hg] with x hf hg
  simp [hf, hg, mul_comm]

@[simp]
lemma eLpNorm_mk {f : Space d → ℂ} {hf : MemHS f} : eLpNorm (mk hf) 2 = eLp
  eLpNorm_congr_ae (coe_mk_ae hf)

lemma mem_iff {f : Space d → ℂ} (hf : AESTronglyMeasurable f volume) :
  AEEqFun.mk f hf ∈ SpaceDHilbertSpace d ↔ Integrable (fun x ↦ ||f x|| ^ 2)
  rw [Lp.mem_Lp_iff_memLp, MemLp, eLpNorm_aeqFun]
  have h1 := AEEqFun.aestronglyMeasurable (AEEqFun.mk f hf)
  have h2 : AESTronglyMeasurable (fun x ↦ norm (f x) ^ 2) :=
    AESTronglyMeasurable.pow (continuous_norm.comp_aestronglyMeasurable hf)
  simp only [h1]
  simp only [eLpNorm_lt_top_iff_lintegral_rpow_enorm_lt_top (NeZero.ne' 2),
    (ENNReal.top_ne_ofNat).symm, ENNReal.toReal_ofNat, ENNReal.rpow_ofNat]
  simp [h2, Integrable, HasFiniteIntegral]

@[simp]
lemma mk_add {f g : Space d → ℂ} {hf : MemHS f} {hg : MemHS g} :
  mk (memHS_add hf hg) = mk hf + mk hg := rfl

@[simp]
lemma mk_const_smul {f : Space d → ℂ} {c : ℂ} {hf : MemHS f} :
  mk (memHS_const_smul (c := c) hf) = c • mk hf := rfl

lemma mk_eq_iff {f g : Space d → ℂ} {hf : MemHS f} {hg : MemHS g} :
  mk hf = mk hg ↔ f ==[volume] g := by simp [mk]

lemma ext_iff {f g : SpaceDHilbertSpace d} :
  f = g ↔ (f : Space d → ℂ) ==[volume] (g : Space d → ℂ) := Lp.ext_iff

end SpaceDHilbertSpace
end
end QuantumMechanics

```

1.93.

PHYSLEAN/QUANTUMMECHANICS/DDIMENSIONS/SPACEDHILBERTSPACE.SWARTZSUBMODULE.LEAN

1.93 PhysLean/QuantumMechanics/DDimensions/SpaceDHilbertSpace/Schwartz

```
/-
Copyright (c) 2026 Gregory J. Loges. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Gregory J. Loges
-/
import Mathlib.Analysis.Distribution.SchwartzSpace.Basic
import PhysLean.QuantumMechanics.DDimensions.SpaceDHilbertSpace.Basic
/-!

# Schwartz submodule of the Hilbert space

-/

namespace QuantumMechanics
namespace SpaceDHilbertSpace

noncomputable section

open MeasureTheory
open InnerProductSpace
open SchwartzMap

/-- The continuous linear map including Schwartz functions into `SpaceDHilbertSpace`
/
def schwartzIncl {d : ℕ} :  $\mathcal{L}(\text{Space } d, \mathbb{C}) \rightarrow \mathcal{L}(\mathbb{C})$  SpaceDHilbertSpace d := toLpCLM  $\mathbb{C}$  (E

lemma schwartzIncl_injective {d : ℕ} : Function.Injective (schwartzIncl (d := d)) :=
  injective_toLp (E := Space d) 2

lemma schwartzIncl_coe_ae {d : ℕ} (f :  $\mathcal{L}(\text{Space } d, \mathbb{C})$ ) : f.1 ==[volume] (schwartzIncl
  (coeFn_toLp f 2)).symm

lemma schwartzIncl_inner {d : ℕ} (f g :  $\mathcal{L}(\text{Space } d, \mathbb{C})$ ) :
   $\langle \text{schwartzIncl } f, \text{schwartzIncl } g \rangle_{\mathbb{C}} = \int x : \text{Space } d, \text{starRingEnd } \mathbb{C} (f \ x) * g \ x :=
  \text{apply integral_congr_ae}$ 
  filter_upwards [schwartzIncl_coe_ae f, schwartzIncl_coe_ae g] with _ hf hg
  rw [← hf, ← hg, RCLike.inner_apply, mul_comm]
  rfl

/-- The submodule of `SpaceDHilbertSpace d` consisting of Schwartz functions. -
/
abbrev schwartzSubmodule (d : ℕ) := (schwartzIncl (d := d)).range

lemma schwartzSubmodule_dense {d : ℕ} :
```

```

    Dense (schwartzSubmodule d : Set (SpaceDHilbertSpace d)) :=
      denseRange_toLpCLM ENNReal.top_ne_ofNat.symm

end
end SpaceDHilbertSpace
end QuantumMechanics

```

1.94 PhysLean/QuantumMechanics/FiniteTarget/Basic.lean

```

NormedSpace.exp (-(Complex.I * t / ħ) • A.Ham)

```

1.95 PhysLean/QuantumMechanics/OneDimension/HilbertSpace/PlaneWave

```

import Mathlib.Analysis.Distribution.TemperedDistribution
open MeasureTheory SchwartzMap TemperedDistribution
  (TemperedDistribution.delta k : SchwartzMap ℝ ℂ → L[ℂ] ℂ) ∘ L (SchwartzMap.
  apply (FourierTransform.fourierCLE ℂ (ℝ, ℂ)).injective

```

1.96 PhysLean/QuantumMechanics/OneDimension/HilbertSpace/Position

```

import Mathlib.Analysis.Distribution.TemperedDistribution
def positionState (x : ℝ) : ℂ(ℝ, ℂ) → L[ℂ] ℂ := TemperedDistribution.delta x

```

1.97 PhysLean/QuantumMechanics/OneDimension/HilbertSpace/SchwartzSpace

```

import Mathlib.Analysis.Distribution.SchwartzSpace.Fourier

```

1.98 PhysLean/QuantumMechanics/OneDimension/Operators/Momentum

```

toFun ψ := (- Complex.I * ħ) • SchwartzMap.derivCLM ℂ ℂ ψ
  (SchwartzMap.fourierTransformCLM ℂ) ((SchwartzMap.derivCLM ℂ ℂ) ψ) k)
  (SchwartzMap.differentiable (ψ)) (SchwartzMap.integrable ((SchwartzMap.
  ((-Complex.I * ↑↑ħ) * (SchwartzMap.derivCLM ℂ ℂ) (ψ2) x)
rw [← fderiv_apply_one_eq_deriv]
  change (starL' ℝ) (SchwartzMap.derivCLM ℂ ℂ (ψ1) x)
exact SchwartzMap.integrable ((SchwartzMap.derivCLM ℂ ℂ) (ψ1))
  (fun x => SchwartzMap.derivCLM ℂ ℂ (ψ2) x) T MeasureTheory.volume
exact SchwartzMap.memLp_top ((SchwartzMap.derivCLM ℂ ℂ) (ψ2))

```

1.99.

PHYSLEAN/QUANTUMMECHANICS/ONEDIMENSION/OPERATORS/PARITY.lean

1.99 PhysLean/QuantumMechanics/OneDimension/Operators/Parity.lean

```
simp [ContinuousLinearMap.norm_id]
simp only [Nat.succ_eq_add_one, fderiv_id', Function.comp_apply,
  LinearIsometryEquiv.norm_map, ge_iff_le]
```

1.100 PhysLean/Relativity/LorentzAlgebra/ExponentialMap.lean

```
NormedSpace.exp (A.1T) = η * (NormedSpace.exp (-A.1)) * η := by
  erw [NormedSpace.exp_units_conj P_gl (-A.1)]
theorem exp_mem_lorentzGroup (A : lorentzAlgebra) : NormedSpace.exp A.1 ∈ LorentzGroup A := by
  (η * NormedSpace.exp (-A.1) * η) * η * NormedSpace.exp A.1
  _ = η * NormedSpace.exp (-A.1) * (η * η) * NormedSpace.exp A.1 := by noncomm_rinv
  _ = η * NormedSpace.exp (-A.1) * 1 * NormedSpace.exp A.1 := by rw [minkowskiMatr
  _ = η * NormedSpace.exp (-A.1 + A.1) := by
  _ = η * NormedSpace.exp 0 := by rw [neg_add_cancel]
NormedSpace.exp (A.submatrix e.symm e.symm) = reindex e e (NormedSpace.exp A) :=
exact (NormedSpace.map_exp f.toAlgHom h_cont A).symm
LorentzGroup.IsProper (NormedSpace.exp A.1, exp_mem_lorentzGroup A) := by
LorentzGroup.IsOrthochronous (NormedSpace.exp A.1, exp_mem_lorentzGroup A) := by
let exp_γ : Path (1 : LorentzGroup 3) (NormedSpace.exp A.1, exp_mem_lorentzGroup A) :=
{ toFun := fun t => (NormedSpace.exp (γ t).val, exp_mem_lorentzGroup (γ t)),
  have h_joined : Joined (1 : LorentzGroup 3) (NormedSpace.exp A.1, exp_mem_lorentzGroup A) :=
  have h_connected : (NormedSpace.exp A.1, exp_mem_lorentzGroup A) ∈ connectedCompon
    ((NormedSpace.exp A.1, exp_mem_lorentzGroup A) : LorentzGroup 3) ∈
```

1.101 PhysLean/Relativity/LorentzGroup/Boosts/Generalized.lean

```
have hx := Velocity.one_add_minkowskiProduct_ne_zero u v
exact Velocity.one_add_minkowskiProduct_ne_zero u v
exact Velocity.one_add_minkowskiProduct_ne_zero u v
have h1 := Velocity.one_add_minkowskiProduct_ne_zero u v
exact Velocity.one_add_minkowskiProduct_ne_zero u v
  have h1 := Velocity.one_add_minkowskiProduct_ne_zero u v
apply (smul_right_inj (Velocity.one_add_minkowskiProduct_ne_zero u v)).mp
  have := (Velocity.one_add_minkowskiProduct_ne_zero u v)
apply (smul_right_inj (Velocity.one_add_minkowskiProduct_ne_zero u v)).mp
apply (smul_right_inj (Velocity.one_add_minkowskiProduct_ne_zero u v)).mp
  exact fun x => Velocity.one_add_minkowskiProduct_ne_zero u x
  exact fun x => Velocity.one_add_minkowskiProduct_ne_zero _ _
apply (smul_right_inj (Velocity.one_add_minkowskiProduct_ne_zero v u)).mp
  · have h1 := Velocity.one_add_minkowskiProduct_ne_zero u v
/-- The time component of a generalised boost.
```

```

    v.1.timeComponent • u.1.spatialPart || ^ 2 / (1 + ||u.1, v.1||m) := by
rw [generalizedBoost_apply_eq_toCoord]
simp only [Matrix.one_apply_eq, inl_0_inl_0, one_mul]
have h := Velocity.one_add_minkowskiProduct_ne_zero u v
rw [norm_sub_sq_real, norm_smul, norm_smul, Real.norm_eq_abs, Real.norm_eq_abs,
    Velocity.timeComponent_abs u, Velocity.timeComponent_abs v,
    real_inner_smul_left, real_inner_smul_right]
simp only [timeComponent, minkowskiProduct_eq_timeComponent_spatialPart]
field_simp [h]
nlinarith [mul_pow (u.1 (Sum.inl 0)) (||v.1.spatialPart||) 2,
    mul_pow (v.1 (Sum.inl 0)) (||u.1.spatialPart||) 2,
    Velocity.norm_spatialPart_sq_eq u, Velocity.norm_spatialPart_sq_eq v,
    real_inner_comm (u.1.spatialPart) (v.1.spatialPart)]

```

1.102 PhysLean/Relativity/Tensors/Color/Lift.lean

```

map η := LaxBraidedFunctor.homMk (repNatTransOfColor η)
refine LaxBraidedFunctor.hom_ext ?_
    LaxBraidedFunctor.homMk_hom_hom, LaxBraidedFunctor.id_hom, NatTrans.id_hom
refine LaxBraidedFunctor.hom_ext ?_
    LaxBraidedFunctor.homMk_hom_hom, LaxBraidedFunctor.comp_hom, LaxMonoid.comp_app
map η := Discrete.natTrans fun c => η.hom.hom.app (incl.obj c)

```

1.103 PhysLean/Relativity/Tensors/ComplexTensor/Metrics/Basic.lean

```

congr 1
exact Lorentz.coMetric_apply_one
congr 1
exact Lorentz.contrMetric_apply_one
congr 1
exact Fermion.leftMetric_apply_one
congr 1
exact Fermion.rightMetric_apply_one
congr 1
exact Fermion.altLeftMetric_apply_one
congr 1
exact Fermion.altRightMetric_apply_one

```

1.104 PhysLean/Relativity/Tensors/ComplexTensor/Metrics/Lemmas.lean

```

exact contrT_metricTensor_metricTensor_eq_dual_unit

```

1.105.

PHYSLEAN/RELATIVITY/TENSORS/COMPLEXTENSOR/OF RAT. LEAN 97

```
exact contrT_metricTensor_metricTensor_eq_dual_unit
exact contrT_metricTensor_metricTensor_eq_dual_unit
exact contrT_metricTensor_metricTensor_eq_dual_unit
exact contrT_metricTensor_metricTensor_eq_dual_unit
exact contrT_metricTensor_metricTensor_eq_dual_unit
```

1.105 PhysLean/Relativity/Tensors/ComplexTensor/OfRat.lean

```
ofRat_basis_repr_apply, Fin.val_cast, mul_one,
simp only [Fin.val_cast, ne_eq] at hy
```

1.106 PhysLean/Relativity/Tensors/ComplexTensor/Units/Basic.lean

```
congr 1
exact Lorentz.coContrUnit_apply_one
congr 1
exact Lorentz.contrCoUnit_apply_one
congr 1
exact Fermion.altLeftLeftUnit_apply_one
congr 1
exact Fermion.leftAltLeftUnit_apply_one
congr 1
exact Fermion.altRightRightUnit_apply_one
congr 1
exact Fermion.rightAltRightUnit_apply_one
```

1.107 PhysLean/Relativity/Tensors/ComplexTensor/Weyl/Two.lean

```
exact congrArg _ (Basis.tensorProduct_apply leftBasis leftBasis i j)
exact congrArg _ (Basis.tensorProduct_apply altLeftBasis altLeftBasis i j)
exact congrArg _ (Basis.tensorProduct_apply leftBasis altLeftBasis i j)
exact congrArg _ (Basis.tensorProduct_apply altLeftBasis leftBasis i j)
exact congrArg _ (Basis.tensorProduct_apply rightBasis rightBasis i j)
exact congrArg _ (Basis.tensorProduct_apply altRightBasis altRightBasis i j)
exact congrArg _ (Basis.tensorProduct_apply rightBasis altRightBasis i j)
exact congrArg _ (Basis.tensorProduct_apply altRightBasis rightBasis i j)
exact congrArg _ (Basis.tensorProduct_apply altLeftBasis altRightBasis i j)
exact congrArg _ (Basis.tensorProduct_apply leftBasis rightBasis i j)
```

1.108 PhysLean/Relativity/Tensors/Contraction/Basis.lean

```
simp only [ofFin, dropPair, Pure.dropPairEmb_ne_fst, ↓reduceDItte, Pure.dr
simp only [Fin.val_cast]
```

1.109 PhysLean/Relativity/Tensors/Contraction/Products.lean

```
simp_all only [Fin.succAbove, Fin.lt_def, Fin.val_castSucc, ne_eq]
· simp_all only [Fin.succAbove, Fin.lt_def, Fin.val_castSucc, ne_eq, ite_
· simp only [Fin.val_castAdd, Fin.val_natAdd]
· simp only [Fin.val_castAdd, Fin.val_natAdd]
  simp only [Function.comp_apply, Fin.ext_iff, Fin.val_natAdd, add_righ
  simp only [Fin.ext_iff, Fin.val_natAdd, Fin.natAdd_mk] at h ⊢
```

1.110 PhysLean/Relativity/Tensors/Contraction/Pure.lean

```
lemma fst_ne_dropPairEmb_pre (i j : Fin (n + 1 + 1)) (m : Fin n) :
lemma dropPairEmb_ne_fst (i j : Fin (n + 1 + 1)) (m : Fin n) :
lemma snd_ne_dropPairEmb_pre (i j : Fin (n + 1 + 1)) (m : Fin n) :
  exact fst_ne_dropPairEmb_pre j i m
lemma dropPairEmb_ne_snd (i j : Fin (n + 1 + 1)) (m : Fin n) :
  exact Ne.symm (fst_ne_dropPairEmb_pre i j m)
```

1.111 PhysLean/Relativity/Tensors/Evaluation.lean

```
@[simp]
lemma evalT_pure {n : ℕ} {c : Fin (n + 1) → C} (i : Fin (n + 1))
  (b : Fin (S.repDim (c i))) (p : Pure S c) :
  evalT i b p.toTensor = Pure.evalP i b p := by
simp only [evalT, Pure.toTensor]
change _ = Pure.evalPMultilinear i b p
conv_rhs => rw [← PiTensorProduct.lift.tprod]
rfl
```

1.112 PhysLean/Relativity/Tensors/Product.lean

```
simp only [Sum.elim_inl, finCongr_symm, finCongr_apply, Fin.val_cast]
  finCongr_apply, Fin.val_cast]
simp only [prodAssocMap, finSumFinEquiv_apply_right, Fin.ext_iff, Fin.val
simp only [prodAssocMap', finSumFinEquiv_apply_right, Fin.ext_iff, Fin.va
```


1.113 PhysLean/Relativity/Tensors/RealTensor/Basic.lean

```
eq_mpr_eq_cast, Fin.val_cast, e]
simp_all only [ne_eq, Fin.ext_iff, Finset.mem_univ, Fin.val_cast,
```

1.114 PhysLean/Relativity/Tensors/RealTensor/Metrics/Basic.lean

```
congr 1
exact Lorentz.preCoMetric_apply_one
congr 1
exact Lorentz.preContrMetric_apply_one
```

1.115 PhysLean/Relativity/Tensors/RealTensor/ToComplex.lean

```
# Complex Lorentz tensors from real Lorentz tensors
## i. Overview
```

In this module we describe how to pass from real Lorentz tensors to complex Lorentz in a functorial way.
Specifically, we construct a canonical equivariant semilinear map

```
* `toComplex :  $\mathbb{RT}(3, c) \rightarrow_{\text{sl}} [\text{Complex.ofRealHom}] \mathbb{CT}(\text{colorToComplex} \circ c)`$ 
```

which is compatible with the natural operations on tensors (permutations of indices, tensor products, contractions and evaluations).

```
## ii. Key results
```

The main definitions and statements are:

- * ``colorToComplex`` upgrades the colour of a real Lorentz tensor to the corresponding complex Lorentz colour.
- * ``TensorSpecies.Tensor.ComponentIdx.complexify`` transports component indices along ``colorToComplex``.
- * ``toComplex`` is the basic semilinear map from real to complex Lorentz tensors.
- * ``toComplex_basis`` and ``toComplex_pure_basisVector`` show that ``toComplex`` sends basis tensors to basis tensors.
- * ``toComplex_eq_zero_iff`` and ``toComplex_injective`` show that ``toComplex`` is injective.
- * ``toComplex_equivariant`` states that ``toComplex`` is equivariant for the action of the complexified Lorentz group.

```
* `permT_toComplex`, `prodT_toComplex`, `contrT_toComplex` and `evalT_toComplex`
  express that `toComplex` commutes with the basic tensor operations.
```

```
## iii. Table of contents
```

```
* A. Colours and component indices
* B. The semilinear map `toComplex`
  * B.1. Expression in the tensor basis
  * B.2. Behaviour on basis vectors and injectivity
  * B.3. Equivariance under the Lorentz action
* C. Compatibility with permutations: `permT`
* D. Compatibility with tensor products: `prodT`
* E. Compatibility with contraction: `contrT`
* F. Compatibility with evaluation: `evalT`
```

```
## iv. References
```

The general formalism of Lorentz tensors and their operations is developed in other parts of the library; here we only specialise to the passage from real to complex tensors.

```
## A. Colours and component indices
```

We first explain how the Lorentz colour data and component indices for real tensors are transported to the complex setting.

```
-/
```

```
@[simp]
```

```
lemma ComponentIdx.complexify_apply {n} {c : Fin n → realLorentzTensor.Color}
  (f : ComponentIdx (S := realLorentzTensor) c) (j : Fin n) :
  (ComponentIdx.complexify f) j = Fin.cast (by
    simp only [repDim_eq_one_plus_dim, Nat.reduceAdd, Function.comp_apply]
    generalize c j = cj
    match cj with
    | .up => rfl
    | .down => rfl) (f j) :=
  rfl
```

```
@[simp]
```

```
lemma ComponentIdx.complexify_toFun_apply {n} {c : Fin n → realLorentzTensor.Color}
  (f : ComponentIdx (S := realLorentzTensor) c) (j : Fin n) :
  (ComponentIdx.complexify.toFun f) j = (ComponentIdx.complexify f) j :=
  rfl
```

```
/-!
```

B. The semilinear map `toComplex`

We now define the basic semilinear map from real Lorentz tensors to complex Lorentz tensors. It is characterised by sending the standard tensor basis on the real side to the corresponding basis on the complex side, and is therefore determined by the behaviour on components.

-/

/-- `toComplex` sends basis elements to basis elements. -

/

@[simp]

lemma toComplex_basis {n} {c : Fin n → realLorentzTensor.Color}

(i : ComponentIdx (S := realLorentzTensor) c) :

toComplex (c := c) ((Tensor.basis (S := realLorentzTensor) c) i) =

(Tensor.basis (S := complexLorentzTensor) (colorToComplex ∘ c)) i.complexify :

classical

simp only [toComplex, LinearMap.coe_mk, AddHom.coe_mk]

rw [Basis.repr_self]

simp_rw [Finsupp.single_apply]

-- collapse the sum: only the `i`-term survives

refine (Fintype.sum_eq_single i ?_).trans ?_

· intro j hj

have hij : i ≠ j := Ne.symm hj

simp [hij]

· -- now the remaining term is the `i`-term

simp

/-- `toComplex` on a pure basis vector. -/

@[simp]

lemma toComplex_pure_basisVector {n} {c : Fin n → realLorentzTensor.Color}

(b : ComponentIdx (S := realLorentzTensor) c) :

toComplex (c := c) (Pure.basisVector c b |>.toTensor)

=

(Pure.basisVector (colorToComplex ∘ c) b.complexify).toTensor := by

classical

-- rewrite pure basis vector back to tensor basis, use `toComplex\_basis`, then rew

rw [← Tensor.basis_apply (S := realLorentzTensor) (c := c) b]

rw [toComplex_basis (c := c) b]

rw [Tensor.basis_apply (S := complexLorentzTensor) (c := (colorToComplex ∘ c)) b.c

lemma toComplex_map_smul {n} (c : Fin n → realLorentzTensor.Color) (r : ℝ) (t : ℝT(3

toComplex (c := c) (r • t) = (Complex.ofReal r) • toComplex (c := c) t :=

(toComplex (c := c)).map_smul_{s1} r t

```
/-!
```

```
### B.3. Equivariance under the Lorentz action
```

Finally we record that ``toComplex`` is equivariant for the natural action of ``SL(2, ℂ)`` (and hence the induced Lorentz action) on tensors.

```
-/
```

```
## C. Compatibility with permutations: `permT`
```

We first show that complexification is compatible with permutation of tensor slots. On colours this is encoded in the ``PermCond`` predicate, and on tensors by the operator ``permT``.

```
/-- `permT` sends basis vectors to basis vectors. -/
```

```
@[simp] lemma permT_basis_real {n m : ℕ}
  {c : Fin n → realLorentzTensor.Color} {c1 : Fin m → realLorentzTensor.Color}
  {σ : Fin m → Fin n} (h : PermCond c c1 σ)
  (b : ComponentIdx (S := realLorentzTensor) c) :
  permT (S := realLorentzTensor) σ h ((Tensor.basis (S := realLorentzTensor) c))
  =
  (Tensor.basis (S := realLorentzTensor) c1)
  (fun j => Fin.cast (by simp [repDim_eq_one_plus_dim]) (b (σ j))) := by
  classical
  simp [Tensor.basis_apply, permT_pure, Pure.permP_basisVector]
```

```
@[simp] lemma permT_basis_complex {n m : ℕ}
  {c : Fin n → complexLorentzTensor.Color} {c1 : Fin m → complexLorentzTensor.Color}
  {σ : Fin m → Fin n} (h : PermCond c c1 σ)
  (b : ComponentIdx (S := complexLorentzTensor) c) :
  permT (S := complexLorentzTensor) σ h ((Tensor.basis (S := complexLorentzTensor) c))
  =
  (Tensor.basis (S := complexLorentzTensor) c1)
  (fun j => Fin.cast
    (by
      -- from the color agreement we get the repDim agreement
      -- if one has `h.2 j : c1 j = c (σ j)`, then replace it with `(h.2 j).
      simp using congrArg (fun col => (complexLorentzTensor).repDim col)
      (b (σ j))) := by
  classical
  simp [Tensor.basis_apply, permT_pure, Pure.permP_basisVector]
```

```
classical
```

```
let h' : PermCond (colorToComplex ∘ c) (colorToComplex ∘ c1) σ :=
  permCond_colorToComplex (c := c) (c1 := c1) h
```

1.115.

PHYSLEAN/RELATIVITY/TENSORS/REALTENSOR/TOCOMPLEX.LEAN 103

```

let P :  $\mathbb{RT}(3, c) \rightarrow \text{Prop} := \text{fun } t \Rightarrow
  \text{toComplex (permT (S := realLorentzTensor) } \sigma \text{ h } t)
  =
  \text{permT (S := complexLorentzTensor) } \sigma \text{ h' (toComplex (c := c) t)}
change P t
apply induction_on_basis
· intro b
  dsimp [P, h']

  -- permT on (real/complex) basis + toComplex on basis
  simp (config := { failIfUnchanged := false })
    [permT_basis_real, permT_basis_complex, toComplex_basis]

  -- index equality
  apply congrArg (Tensor.basis (S := complexLorentzTensor) (colorToComplex . c1))
  funext j
  simp [TensorSpecies.Tensor.ComponentIdx.complexify, colorToComplex, Function.com
· simp [P]
· intro r t ht
  dsimp [P] at ht ⊢
  refine (by
    simp [map_smul, ht])
· intro t1 t2 h1 h2
  dsimp [P] at h1 h2 ⊢
  refine (by
    simp [map_add, h1, h2])$ 
```

/-!

D. Compatibility with tensor products: `prodT`

```

-/
lemma permCond_prodTColorToComplex {n m :  $\mathbb{N}$ }
  {c : Fin n → realLorentzTensor.Color} {c1 : Fin m → realLorentzTensor.Color} :
  PermCond (Fin.append (colorToComplex . c) (colorToComplex . c1))
    (colorToComplex . Fin.append c c1)
    (id : Fin (n + m) → Fin (n + m)) := by
  -- For `σ = id`, `PermCond.on_id` reduces the goal to pointwise color equality.
  -- Here that equality is exactly `colorToComplex_append`.
  apply (PermCond.on_id
    (c := Fin.append (colorToComplex . c) (colorToComplex . c1))
    (c1 := colorToComplex . Fin.append c c1)).2
  intro x
  -- `colorToComplex_append` states the two color functions are extensionally equal,
  -- but with the sides reversed, so we use its symmetric form.
  have hx := congrArg (fun f => f x)

```

```

    (colorToComplex_append (c := c) (c1 := c1)).symm
  simp [Function.comp_apply] using hx

  fun x y =>
    permT (S := complexLorentzTensor) (σ := (id : Fin (n + m) → Fin (n + m))
      (permCond_prodTColorToComplex (c := c) (c1 := c1))
      (prodT (S := complexLorentzTensor) x y))

private lemma cast_componentIdx_apply {n : ℕ} {c c' : Fin n → complexLorentzTensor.Color}
  (h : c' = c) (f : ComponentIdx (S := complexLorentzTensor) c') (x : Fin n) :
  (cast (congr_arg ComponentIdx h) f) x =
    Fin.cast (congr_arg (fun c => complexLorentzTensor.repDim (c x)) h) (f x)
  subst h
  rfl

@[simp]
private lemma cast_componentIdx_eq_fun {n : ℕ}
  {c c' : Fin n → complexLorentzTensor.Color}
  (h : c' = c) (f : ComponentIdx (S := complexLorentzTensor) c') :
  cast (congr_arg ComponentIdx h) f =
    (fun x =>
      Fin.cast (congr_arg (fun col => complexLorentzTensor.repDim (col x)) h) (f x))
  funext x
  exact cast_componentIdx_apply (c := c) (c' := c') h f x

/-- `complexify` commutes with `prod` of component indices. -
/
@[simp]
lemma complexify_prod {n m : ℕ}
  {c : Fin n → realLorentzTensor.Color} {c1 : Fin m → realLorentzTensor.Color}
  (b : ComponentIdx (S := realLorentzTensor) c)
  (b1 : ComponentIdx (S := realLorentzTensor) c1) :
  ComponentIdx.complexify (c := Fin.append c c1) (b.prod b1)
  =
  cast (congr_arg ComponentIdx (colorToComplex_append c c1).symm)
    ((ComponentIdx.complexify (c := c) b).prod (ComponentIdx.complexify (c1 := c1) b1))
  ext x
  obtain ⟨i, rfl⟩ := finSumFinEquiv.surjective x
  cases i with
  | inl i =>
    rw [ComponentIdx.complexify_apply, ComponentIdx.prod_apply_finSumFinEquiv]
    have cast_apply_rhs : (cast (congr_arg ComponentIdx (colorToComplex_append c c1).symm)
      ((ComponentIdx.complexify b).prod (ComponentIdx.complexify b1)))
      (finSumFinEquiv (Sum.inl i)) =
      Fin.cast (congr_arg (fun c =>
        complexLorentzTensor.repDim (c (finSumFinEquiv (Sum.inl i)))))

```

1.115.

PHYSLEAN/RELATIVITY/TENSORS/REALTENSOR/TOCOMPLEX.LEAN 105

```

      (colorToComplex_append c c1).symm)
    (((ComponentIdx.complexify b).prod (ComponentIdx.complexify b1))
      (finSumFinEquiv (Sum.inl i))) :=
  cast_componentIdx_apply (colorToComplex_append c c1).symm _ _
rw [cast_apply_rhs,
  ComponentIdx.prod_apply_finSumFinEquiv (ComponentIdx.complexify b)
    (ComponentIdx.complexify b1) (Sum.inl i)]
congr 1
| inr j =>
  rw [ComponentIdx.complexify_apply, ComponentIdx.prod_apply_finSumFinEquiv b b1]
  have cast_apply_rhs : (cast (congr_arg ComponentIdx (colorToComplex_append c c1))
    (((ComponentIdx.complexify b).prod (ComponentIdx.complexify b1)))
      (finSumFinEquiv (Sum.inr j))) =
    Fin.cast (congr_arg (fun c => complexLorentzTensor.repDim
      (c (finSumFinEquiv (Sum.inr j))))) (colorToComplex_append c c1).symm)
      (((ComponentIdx.complexify b).prod (ComponentIdx.complexify b1))
        (finSumFinEquiv (Sum.inr j))) :=
  cast_componentIdx_apply (colorToComplex_append c c1).symm _ _
rw [cast_apply_rhs,
  ComponentIdx.prod_apply_finSumFinEquiv (ComponentIdx.complexify b)
    (ComponentIdx.complexify b1) (Sum.inr j)]
congr 1
classical
-- Induction on the first tensor using the tensor basis.
let P :  $\mathbb{RT}(3, c) \rightarrow \text{Prop}$  := fun t =>
   $\forall t1 : \mathbb{RT}(3, c1),$ 
    toComplex (c := Fin.append c c1) (prodT (S := realLorentzTensor) t t1)
    =
    prodTColorToComplex (c := c) (c1 := c1)
      (toComplex (c := c) t) (toComplex (c := c1) t1)
have hP : P t := by
  -- `induction_on_basis` over the first tensor.
  apply
    induction_on_basis
      (c := c)
      (P := P)
      (t := t)
  · -- basis case for the first tensor and we must show the property for all `t1`
    intro b t1
    -- Define the property on the second tensor, with the first fixed to a basis v
    let P1 :  $\mathbb{RT}(3, c1) \rightarrow \text{Prop}$  := fun t1' =>
      toComplex (c := Fin.append c c1)
        (prodT (S := realLorentzTensor)
          ((Tensor.basis (S := realLorentzTensor) c) b) t1'))
      =
      prodTColorToComplex (c := c) (c1 := c1)

```

```

      (toComplex (c := c) ((Tensor.basis (S := realLorentzTensor) c) b)
      (toComplex (c := c1) t1'))
have hP1 : P1 t1 := by
  -- Induction on the second tensor using the tensor basis.
  apply
    induction_on_basis
      (c := c1)
      (P := P1)
      (t := t1)
  · -- basis case for the second tensor
    intro b1
    -- Unfold `P1` and compute both sides explicitly on pure basis te
    dsimp [P1]
    simp (config := { failIfUnchanged := false })
      [prodTColorToComplex,
       prodT_pure,
       permT_pure,
       Pure.prodP_basisVector,
       Pure.permP_basisVector,
       Tensor.basis_apply,
       toComplex_pure_basisVector,
       colorToComplex_append]
  · -- zero tensor in the second argument
    simp [P1, prodTColorToComplex]
  · -- scalar multiplication in the second argument
    intro r t1' ht'
    dsimp [P1] at ht' ⊢
    refine (by
      simp [map_smul, ht', prodTColorToComplex])
  · -- addition in the second argument
    intro t1' t2' h1 h2
    dsimp [P1] at h1 h2 ⊢
    refine (by
      simp [map_add, h1, h2, prodTColorToComplex])
  -- Apply the resulting property to `t1`.
  exact hP1
· -- zero tensor in the first argument
  intro t1
  simp [prodTColorToComplex]
· -- scalar multiplication in the first argument
  intro r t ht t1
  dsimp [P] at ht ⊢
  refine (by
    simp [map_smul, ht, prodTColorToComplex])
· -- addition in the first argument
  intro t1 t2 h1 h2 t1'

```


1.115.

PHYSLEAN/RELATIVITY/TENSORS/REALTENSOR/TOCOMPLEX.LEAN 107

```

    dsimp [P] at h1 h2 ⊢
    refine (by
      simp [map_add, h1 t1', h2 t1', prodTColorToComplex])
  -- Apply the resulting property to `t1`.
  exact hP t1

/-!

### E. Compatibility with contraction: `contrT`

-/
/-- `complexify` commutes with precomposition by `dropPairEmb`.
  We use `fun k => b (Pure.dropPairEmb i j k)` and direct application
  `(ComponentIdx.complexify b) (Pure.dropPairEmb i j m)` rather than composition so
  dependent `ComponentIdx` types unify correctly (avoiding `Function.comp` type mism
  /
@[simp]
lemma ComponentIdx.complexify_comp_dropPairEmb
  {n : ℕ} {c : Fin (n + 1 + 1) → realLorentzTensor.Color}
  {i j : Fin (n + 1 + 1)} (b : ComponentIdx (S := realLorentzTensor) c) (m : Fin n
  (ComponentIdx.complexify (c := c ∘ Pure.dropPairEmb i j)
    (fun k => b (Pure.dropPairEmb i j k))) m =
  (ComponentIdx.complexify (c := c) b) (Pure.dropPairEmb i j m) := by
  simp only [ComponentIdx.complexify_apply, Function.comp_apply]

/-- For a real basis vector, `toComplex(contrP(basisVector c b))` equals
  `contrP(basisVector (colorToComplex ∘ c) (complexify b))` (complex species). -
  /
lemma toComplex_contrP_basisVector {n : ℕ} {c : Fin (n + 1 + 1) → realLorentzTensor.
  {i j : Fin (n + 1 + 1)} (h : i ≠ j ∧ (realLorentzTensor).τ (c i) = c j)
  (b : ComponentIdx (S := realLorentzTensor) c) :
  toComplex (c := c ∘ Pure.dropPairEmb i j)
    (Pure.contrP (S := realLorentzTensor) i j h (Pure.basisVector c b))
  =
  Pure.contrP (S := complexLorentzTensor) i j
  (by
    simpa [Function.comp_apply] using And.intro h.1
      (by simpa [tau_colorToComplex] using congrArg colorToComplex h.2))
    (Pure.basisVector (colorToComplex ∘ c) (ComponentIdx.complexify b)) := by
  let c' := c ∘ Pure.dropPairEmb i j
  simp only [Pure.contrP]
  rw [toComplex_map_smul c' (Pure.contrPCoeff i j h (Pure.basisVector c b))
    ((Pure.dropPair i j h.1 (Pure.basisVector c b)).toTensor),
    Pure.dropPair_basisVector (c := c),
    ← Tensor.basis_apply (S := realLorentzTensor) c' (fun k => b (Pure.dropPairEmb i
    toComplex_basis (c := c') (i := fun k => b (Pure.dropPairEmb i j k)))]

```

```

congr 1
· -- contrPCoeff: real and complex both equal 0 or 1 with same condition
  have h_real := realLorentzTensor.contr_basis (b i) (Fin.cast (by rw [h.2])
  have h_complex := complexLorentzTensor.basis_contr ((colorToComplex ∘ c) i)
    ((ComponentIdx.complexify b) i)
    (Fin.cast (by simp [tau_colorToComplex, h.2]) ((ComponentIdx.complexify b) i))
  dsimp only [Pure.contrPCoeff]
  simp only [Pure.basisVector]
  erw [← realLorentzTensor.basis_congr (h.2) (Fin.cast (by rw [h.2]) (b j))]
  have heq := (TensorSpecies.castToField_eq_self (S := realLorentzTensor)
    (v := (realLorentzTensor.contr.app { as := c i }).hom
      ((realLorentzTensor.basis (c i)) (b i)
        ⊗t[ $\mathbb{R}$ ] (realLorentzTensor.basis (realLorentzTensor.τ (c i))
          (Fin.cast (by rw [h.2]) (b j)))))).symm
  have hcoeq : ↑((realLorentzTensor.contr.app { as := c i }).hom
    ((realLorentzTensor.basis (c i)) (b i)
      ⊗t[ $\mathbb{R}$ ] (realLorentzTensor.basis (realLorentzTensor.τ (c i))
        (Fin.cast (by rw [h.2]) (b j)))))) =
    ↑(realLorentzTensor.castToField ((realLorentzTensor.contr.app { as := c i }
      ((realLorentzTensor.basis (c i)) (b i)
        ⊗t[ $\mathbb{R}$ ] (realLorentzTensor.basis (realLorentzTensor.τ (c i))
          (Fin.cast (by rw [h.2]) (b j)))))) := congrArg (↑·) heq
  erw [hcoeq, h_real]
  erw [← complexLorentzTensor.basis_congr (by simp [tau_colorToComplex, h.2])
    (Fin.cast (by simp [tau_colorToComplex, h.2])
      ((ComponentIdx.complexify b) j))]
  have hc := (TensorSpecies.castToField_eq_self (S := complexLorentzTensor)
    (v := (complexLorentzTensor.contr.app { as := (colorToComplex ∘ c) i }
      ((complexLorentzTensor.basis ((colorToComplex ∘ c) i) (ComponentIdx.complexify b) i)
        ⊗t[ $\mathbb{C}$ ]
          (complexLorentzTensor.basis (complexLorentzTensor.τ ((colorToComplex ∘ c) i)
            (Fin.cast (by simp [tau_colorToComplex, h.2]) ((ComponentIdx.complexify b) j))
          ))
  erw [hc, h_complex]
  simp only [ComponentIdx.complexify_apply, Fin.val_cast]; split_ifs <=>
· -- complexify(fun k => b (dropPairEmb k)) = (complexify b) ∘ dropPairEmb
  conv_rhs => enter [1]; rw [Pure.dropPair_basisVector (S := complexLorentzTensor)
    (c := colorToComplex ∘ c) h.1 (b := ComponentIdx.complexify b)]
  conv_rhs => rw [← Tensor.basis_apply (S := complexLorentzTensor)
    (c := (colorToComplex ∘ c) ∘ Pure.dropPairEmb i j)
    (b := fun m => (ComponentIdx.complexify b) (Pure.dropPairEmb i j m))]
  refine congr_arg _ (funext fun m => ComponentIdx.complexify_comp_dropPairEmb m)

/-- The map `toComplex` commutes with `contrT`. -/
  simp [tau_colorToComplex] using congrArg colorToComplex h.2))
classical
-- We prove the statement by induction on the tensor `t` using the tensor

```

1.115.

PHYSLEAN/RELATIVITY/TENSORS/REALTENSOR/TOCOMPLEX.LEAN 109

```

-- After contracting two indices, the resulting colour function lives on `Fin n`.
let c' : Fin n → realLorentzTensor.Color := c ∘ Pure.dropPairEmb i j
have hP :
  ∀ t : ℝT(3, c),
    toComplex (c := c') (contrT (S := realLorentzTensor) n i j h t) =
      contrT (S := complexLorentzTensor) n i j
      (by
        -- transport the colour relation along `colorToComplex`
        simpa [Function.comp_apply] using
          And.intro h.1
          (by
            simpa [tau_colorToComplex] using congrArg colorToComplex h.2))
    (toComplex (c := c) t) := by
intro t
-- Work with the property as a predicate for `induction_on_basis`.
let P : ℝT(3, c) → Prop := fun t =>
  toComplex (c := c') (contrT (S := realLorentzTensor) n i j h t) =
    contrT (S := complexLorentzTensor) n i j
    (by
      simpa [Function.comp_apply] using
        And.intro h.1
        (by
          simpa [tau_colorToComplex] using congrArg colorToComplex h.2))
    (toComplex (c := c) t)
have hP' : P t := by
  -- `induction_on_basis` over the tensor `t`.
  apply
    induction_on_basis
      (c := c)
      (P := P)
      (t := t)
  · -- basis case
    intro b
    -- (Tensor.basis c) b = (Pure.basisVector c b).toTensor;
    -- then equate both sides via toComplex_contrP_basisVector.
    rw [Tensor.basis_apply (S := realLorentzTensor) c b]
    show toComplex (c := c')
      (contrT (S := realLorentzTensor) n i j h ((Pure.basisVector c b).toTensor))
      = contrT (S := complexLorentzTensor) n i j _
      (toComplex (c := c) ((Pure.basisVector c b).toTensor))
    rw [contrT_pure (S := realLorentzTensor) (p := Pure.basisVector c b),
      toComplex_pure_basisVector (c := c) b,
      contrT_pure (S := complexLorentzTensor)
        (p := Pure.basisVector (colorToComplex ∘ c) (ComponentIdx.complexify b))
      exact toComplex_contrP_basisVector h b
  · -- zero tensor

```

```

      dsimp [P]
      simp
    · -- scalar multiplication
      intro r t ht
      dsimp [P] at ht ⊢
      refine (by
        simp [map_smul, ht])
    · -- addition
      intro t1 t2 h1 h2
      dsimp [P] at h1 h2 ⊢
      refine (by
        simp [map_add, h1, h2])
  exact hP'
exact hP t

/-!

### F. Compatibility with evaluation: `evalT`

-/

/-- `complexify` commutes with precomposition by `succAbove`. -
/
@[simp]
lemma ComponentIdx.complexify_comp_succAbove
  {n : ℕ} {c : Fin (n + 1) → realLorentzTensor.Color} (i : Fin (n + 1))
  (b : ComponentIdx (S := realLorentzTensor) c) (m : Fin n) :
  (ComponentIdx.complexify (c := c ∘ i.succAbove) (fun k => b (i.succAbove k)))
  (ComponentIdx.complexify (c := c) b) (i.succAbove m) := by
  simp only [ComponentIdx.complexify_apply, Function.comp_apply]
  simp [hci, colorToComplex, Function.comp_apply, complex_repDim_down]
/- For a real basis vector, `toComplex(evalP(basisVector c b))` equals
`evalP(basisVector (colorToComplex ∘ c) (complexify b))` (complex species)
/
lemma toComplex_evalP_basisVector {n : ℕ} {c : Fin (n + 1) → realLorentzTensor.Color}
  (i : Fin (n + 1)) (b : Fin ((realLorentzTensor).repDim (c i)))
  (b' : ComponentIdx (S := realLorentzTensor) c) :
  toComplex (c := c ∘ i.succAbove)
  (Pure.evalP (S := realLorentzTensor) i b (Pure.basisVector c b'))
  =
  permT (S := complexLorentzTensor) (σ := (id : Fin n → Fin n))
  (by simp [Function.comp_apply])
  (Pure.evalP (S := complexLorentzTensor) i (evalIdxToComplex (c := c)
    (Pure.basisVector (colorToComplex ∘ c) (ComponentIdx.complexify b'))))
  simp only [Pure.evalP]
  have hdrop : (Pure.basisVector c b').drop i =

```

1.115.

PHYSLEAN/RELATIVITY/TENSORS/REALTENSOR/TOCOMPLEX.LEAN 111

```

    Pure.basisVector (c ◦ i.succAbove) (fun k => b' (i.succAbove k)) := by
      ext j; simp only [Pure.drop, Pure.basisVector, Function.comp_apply]
  rw [hdrop, toComplex_map_smul (c ◦ i.succAbove) (Pure.evalPCoeff i b (Pure.basisVe
    ((Pure.basisVector (c ◦ i.succAbove)) (fun k => b' (i.succAbove k)) |>.toTensor)
  · -- evalPCoeff: real and complex match; then tensor equality
    simp only [Pure.evalPCoeff, Pure.basisVector, Basis.repr_self, Finsupp.single_ap
      ComponentIdx.complexify_apply, evalIdxToComplex]
  · by_cases h : b' i = b
    · simp [h]
      have hdrop' : (Pure.basisVector (colorToComplex ◦ c) (ComponentIdx.complexify
        Pure.basisVector (colorToComplex ◦ (c ◦ i.succAbove))
          (ComponentIdx.complexify (c := c ◦ i.succAbove) (fun k => b' (i.succAbov
        ext j; simp only [Pure.drop, Pure.basisVector, ComponentIdx.complexify_app
          Function.comp_apply]
      rw [hdrop']
      exact (permT_id_self (S := complexLorentzTensor) (c := colorToComplex ◦ (c ◦
        (t := (Pure.basisVector (colorToComplex ◦ (c ◦ i.succAbove))
          (ComponentIdx.complexify (c := c ◦ i.succAbove) (fun k => b'
            (i.succAbove k))))).toTensor)).symm
    · simp [h]

classical
let c' := c ◦ i.succAbove
let P : ℝT(3, c) → Prop := fun t =>
  toComplex (c := c')
  ((TensorSpecies.Tensor.evalT (S := realLorentzTensor) (c := c) i b) t) =
  evalTColorToComplex (c := c) i b (toComplex (c := c) t)
have hP : ∀ t, P t := by
  intro t
  apply induction_on_basis (c := c) (P := P) (t := t)
  · intro b'
    rw [Tensor.basis_apply (S := realLorentzTensor) c b']
    simp only [evalTColorToComplex, P]
    rw [evalT_pure (S := realLorentzTensor) (p := Pure.basisVector c b'),
      toComplex_pure_basisVector (c := c) b',
      evalT_pure (S := complexLorentzTensor)
        (p := Pure.basisVector (colorToComplex ◦ c) (ComponentIdx.complexify b'))]]
    exact toComplex_evalP_basisVector i b b'
  · dsimp [P]
    simp only [evalTColorToComplex, map_zero]
  · intro r t' ht'
    dsimp [P] at ht'
    rw [LinearMap.map_smul, toComplex_map_smul (c ◦ i.succAbove) r
      ((evalT (S := realLorentzTensor) i b) t'),
      ht', toComplex_map_smul (c := c) r t']
    simp only [evalTColorToComplex, LinearMap.map_smul]

```

```

· intro t1 t2 h1 h2
  dsimp [P] at h1 h2 ⊢
  rw [LinearMap.map_add, map_add, h1, h2]
  simp only [evalTColorToComplex, LinearMap.map_add]
exact hP t

```

1.116 PhysLean/Relativity/Tensors/RealTensor/Vector/Basic.lean

```
import Mathlib.Geometry.Manifold.IsManifold.Basic
```

1.117 PhysLean/Relativity/Tensors/RealTensor/Velocity/Basic.lean

```
lemma one_add_minkowskiProduct_ne_zero (u v : Velocity d) :
```

1.118 PhysLean/Relativity/Tensors/TensorSpecies/Basic.lean

```
lemma castToField_eq_self {S : TensorSpecies k C G} {c}
  (v : (↑((⊔_ (Discrete C ⊔ Rep k G)).obj { as := c })).V)) :
  S.castToField v = v := rfl

```

1.119 PhysLean/SpaceAndTime/Space/Basic.lean

```
lemma volume_closedBall_ne_zero {d : ℕ} (x : Space d.succ) (r : ℝ) (hr : 0
lemma volume_closedBall_ne_top {d : ℕ} (x : Space d.succ) (r : ℝ) :
```

1.120 PhysLean/SpaceAndTime/Space/ConstantSliceDist.lean

```

change (SchwartzMap.evalCLM ℝ (Space d.succ) ℝ (((slice i).symm (1, 0))
  (SchwartzMap.fderivCLM ℝ (Space d.succ) ℝ) η (((slice i).symm (r, x))
    * ||(1 + ||t||) ^ (rt)||-1)
· exact Filter.univ_mem' (hF (F x₀ 0))

```

1.121 PhysLean/SpaceAndTime/Space/Derivatives/Basic.lean

```

/-- Space derivatives on scalar product of functions. -/
lemma deriv_smul [NormedAddCommGroup M] [NormedSpace ℝ M] [NontriviallyNorm
  [NormedAlgebra ℝ []] [NormedSpace [] M] {c : Space d → []} {f : Space d →

```

```

    (hc : DifferentiableAt ℝ c x) (hf : DifferentiableAt ℝ f x) :
      ∂[u] (c • f) x = c x • ∂[u] f x + ∂[u] c x • f x := by
    unfold deriv
    rw [fderiv_smul hc hf]
    rfl

/-- Space derivatives on scalar times function. -/
lemma deriv_const_smul [NormedAddCommGroup M] [NormedSpace ℝ M] [Semiring R]
  [Module R M] [SMulCommClass ℝ R M] [ContinuousConstSMul R M] {f : Space d → M} (
  (h : Differentiable ℝ f) : ∂[u] (c • f) = c • ∂[u] f := by
  rw [ContinuousLinearMap.coe_smul', Pi.smul_apply, Pi.smul_apply]
  ∂[u] (fun x => k * f x i) x = k * ∂[u] (fun x => f x i) x := by
lemma schwartzMap_fderiv_comm {d}
  ((SchwartzMap.evalCLM ℝ (Space d) ℝ (basis μ))
  ((fderivCLM ℝ (Space d) ℝ) ((SchwartzMap.evalCLM ℝ (Space d) ℝ (basis ν))
  ((fderivCLM ℝ (Space d) ℝ) η)))) x =
  ((SchwartzMap.evalCLM ℝ (Space d) ℝ (basis ν))
  ((fderivCLM ℝ (Space d) ℝ) ((SchwartzMap.evalCLM ℝ (Space d) ℝ (basis μ))
  ((fderivCLM ℝ (Space d) ℝ) η)))) x := by

```

1.122 PhysLean/SpaceAndTime/Space/Derivatives/Curl.lean

```

distCurl f η 0 = - f (SchwartzMap.evalCLM ℝ Space ℝ (basis 2) (fderivCLM ℝ Space
+ f (SchwartzMap.evalCLM ℝ Space ℝ (basis 1) (fderivCLM ℝ Space ℝ η)) 2 := by
distCurl f η 1 = - f (SchwartzMap.evalCLM ℝ Space ℝ (basis 0) (fderivCLM ℝ Space
+ f (SchwartzMap.evalCLM ℝ Space ℝ (basis 2) (fderivCLM ℝ Space ℝ η)) 0 := by
distCurl f η 2 = - f (SchwartzMap.evalCLM ℝ Space ℝ (basis 1) (fderivCLM ℝ Space
+ f (SchwartzMap.evalCLM ℝ Space ℝ (basis 0) (fderivCLM ℝ Space ℝ η)) 1 := by
| 0 => - f (SchwartzMap.evalCLM ℝ Space ℝ (basis 2) (fderivCLM ℝ Space ℝ η)) 1
+ f (SchwartzMap.evalCLM ℝ Space ℝ (basis 1) (fderivCLM ℝ Space ℝ η)) 2
| 1 => - f (SchwartzMap.evalCLM ℝ Space ℝ (basis 0) (fderivCLM ℝ Space ℝ η)) 2
+ f (SchwartzMap.evalCLM ℝ Space ℝ (basis 2) (fderivCLM ℝ Space ℝ η)) 0
| 2 => - f (SchwartzMap.evalCLM ℝ Space ℝ (basis 1) (fderivCLM ℝ Space ℝ η)) 0
+ f (SchwartzMap.evalCLM ℝ Space ℝ (basis 0) (fderivCLM ℝ Space ℝ η)) 1 := by
change ((SchwartzMap.evalCLM ℝ Space ℝ _)
  ((fderivCLM ℝ Space ℝ) ((SchwartzMap.evalCLM ℝ Space ℝ _) ((fderivCLM ℝ Space
- ((SchwartzMap.evalCLM ℝ Space ℝ _)
  ((fderivCLM ℝ Space ℝ) ((SchwartzMap.evalCLM ℝ Space ℝ _) ((fderivCLM ℝ Space

```

1.123 PhysLean/SpaceAndTime/Space/Derivatives/Div.lean

```

(((SchwartzMap.evalCLM ℝ (Space dm1.succ) ℝ (basis i))
  ((fderivCLM ℝ (Space dm1.succ) ℝ) η)) x • f x) j) volume := by

```

1.124 PhysLean/SpaceAndTime/Space/Derivatives/Grad.lean

```

rw [deriv_const_smul]
  Integrable (fun x => (((SchwartzMap.evalCLM ℝ (Space dm1.succ) ℝ (basis i)) (fderivCLM ℝ (Space dm1.succ) ℝ) η)) x • f x) j) volume := by
distGrad f η =
  ∑ i, - f (SchwartzMap.evalCLM ℝ (Space d) ℝ (basis i)) (fderivCLM ℝ (Space d) ℝ) η
  □ ∑ i, - f (SchwartzMap.evalCLM ℝ (Space d) ℝ (basis i)) (fderivCLM ℝ (Space d) ℝ) η
  □ ∑ i, - f (SchwartzMap.evalCLM ℝ (Space d) ℝ (basis i)) (fderivCLM ℝ (Space d) ℝ) η
  (∑ i, - f (SchwartzMap.evalCLM ℝ (Space d) ℝ (basis i)) (fderivCLM ℝ (Space d) ℝ) η)
  (∑ i, - f (SchwartzMap.evalCLM ℝ (Space d) ℝ (basis i)) (fderivCLM ℝ (Space d) ℝ) η)
  EuclideanSpace.single i 1))

```

1.125 PhysLean/SpaceAndTime/Space/LengthUnit.lean

```

lemma val_ne_zero (x : LengthUnit) : x.val ≠ 0 := by
lemma div_ne_zero (x y : LengthUnit) : ¬ x / y = (0 : ℝ≥0) := by
  · exact Ne.symm (div_ne_zero x y)
simp [div_eq_val, x.val_ne_zero]

```

1.126 PhysLean/SpaceAndTime/Space/Norm.lean

```

simp only [distGrad_inner_eq, Distribution.fderivD_apply, distOfFunction_
simp only [distGrad_inner_eq, Distribution.fderivD_apply, distOfFunction_
  simp [fderiv_smul_const, iteratedFDeriv_succ_eq_comp_right,
    ContinuousLinearMap.norm_id]
  · exact (integrable ((derivCLM ℝ ℝ) (η' n))).integrableOn

```

1.127 PhysLean/SpaceAndTime/Space/Translations.lean

```

simp only [translateSchwartz_apply, map_neg, sub_neg_eq_add,
  LinearIsometryEquiv.symm_apply_apply]
have h1 : ((translateSchwartz (-a)) ((SchwartzMap.evalCLM ℝ (Space d) ℝ (basis i)) (fderivCLM ℝ (Space d) ℝ) η))) =
  ((SchwartzMap.evalCLM ℝ (Space d) ℝ (basis i)) (fderivCLM ℝ (Space d) ℝ) ((translateSchwartz (-a)) η))) := by

```


1.128 PhysLean/SpaceAndTime/SpaceTime/Derivatives.lean

```

- f ((SchwartzMap.evalCLM  $\mathbb{R}$  (SpaceTime d)  $\mathbb{R}$  (Lorentz.Vector.basis  $\mu$ ))
  ((fderivCLM  $\mathbb{R}$  (SpaceTime d)  $\mathbb{R}$ )  $\varepsilon$ )) := by
f ((SchwartzMap.evalCLM  $\mathbb{R}$  (SpaceTime d)  $\mathbb{R}$  (Lorentz.Vector.basis  $\mu$ ))
  ((fderivCLM  $\mathbb{R}$  (SpaceTime d)  $\mathbb{R}$ )  $\varepsilon$ )) =

```

1.129 PhysLean/SpaceAndTime/Time/TimeTransMan.lean

```

have hx : x.val  $\neq$  0 := x.val_ne_zero
simp only [ne_eq, TimeUnit.val_ne_zero, not_false_eq_true, inv_mul_cancel_left₀]
· apply Continuous.add
· apply Continuous.fun_mul
· fun_prop
· apply Differentiable.continuous ( $\square$  :=  $\mathbb{R}$ )
  fun_prop
· fun_prop

```

1.130 PhysLean/SpaceAndTime/Time/TimeUnit.lean

```

lemma val_ne_zero (x : TimeUnit) : x.val  $\neq$  0 := by
lemma div_ne_zero (x y : TimeUnit) :  $\neg$  x / y = (0 :  $\mathbb{R}_{\geq 0}$ ) := by
· exact Ne.symm (div_ne_zero x y)
simp [div_eq_val, x.val_ne_zero]

```

1.131 PhysLean/SpaceAndTime/TimeAndSpace/Basic.lean

```

(distTimeDeriv f)  $\varepsilon$  =
-f (SchwartzMap.evalCLM  $\mathbb{R}$  (Time  $\times$  Space d)  $\mathbb{R}$  (1, 0)
  ((fderivCLM  $\mathbb{R}$  (Time  $\times$  Space d)  $\mathbb{R}$ )  $\varepsilon$ )) := by
f (SchwartzMap.evalCLM  $\mathbb{R}$  (Time  $\times$  Space d)  $\mathbb{R}$  (1, 0) ((fderivCLM  $\mathbb{R}$  (Time  $\times$  Space d)
- (distTimeDeriv f)  $\varepsilon$  := by
- f ((SchwartzMap.evalCLM  $\mathbb{R}$  (Time  $\times$  Space d)  $\mathbb{R}$  (0, basis i))
  ((fderivCLM  $\mathbb{R}$  (Time  $\times$  Space d)  $\mathbb{R}$ )  $\varepsilon$ )) := by
f ((SchwartzMap.evalCLM  $\mathbb{R}$  (Time  $\times$  Space d)  $\mathbb{R}$  (0, basis i))
  ((fderivCLM  $\mathbb{R}$  (Time  $\times$  Space d)  $\mathbb{R}$ )  $\varepsilon$ )) =

```

1.132 PhysLean/SpaceAndTime/TimeAndSpace/ConstantTimeDist.lean

```

simp only [Prod.norm_mk, norm_pow, Real.norm_eq_abs, norm_iteratedFDeriv_one,

```

```

    mul_eq_mul_right_iff, norm_eq_zero]
  left
    (ContinuousLinearMap.id  $\mathbb{R}$  (Space d))) *  $\|(1 + \|t\|)^{-1}\|$ 
  · exact Filter.univ_mem' (hF (F x₀ 0))
  simp only [Prod.norm_mk, norm_pow, Real.norm_eq_abs, norm_iteratedFDeriv]
    mul_eq_mul_right_iff, norm_eq_zero]
  left
  suffices h : (timeIntegralSchwartz ((SchwartzMap.evalCLM  $\mathbb{R}$  (Time  $\times$  Space
    ((fderivCLM  $\mathbb{R}$  (Time  $\times$  Space d)  $\mathbb{R}$ )  $\eta$ ))) = 0 by

```

1.133 PhysLean/StatisticalMechanics/BoltzmannConstant.lean

```

lemma kB_ne_zero : kB  $\neq$  0 := by

```

1.134 PhysLean/StatisticalMechanics/CanonicalEnsemble/Finite.lean

```

simp_all only [neg_mul, ne_eq, not_false_eq_true]

```

1.135 PhysLean/StatisticalMechanics/CanonicalEnsemble/Lemmas.lean

```

have hkB : (kB :  $\mathbb{R}$ )  $\neq$  0 := kB_ne_zero
  field_simp [Constants.kB_ne_zero, Tne]
have hkB_ne_zero := kB_ne_zero

```

1.136 PhysLean/Thermodynamics/Temperature/Basic.lean

```

field_simp [kB_ne_zero]
  field_simp [kB_ne_zero]
  · exact kB_ne_zero
  simp [mul_eq_zero, kB_ne_zero, hx0]
    field_simp [pow_two, mul_comm, mul_left_comm, mul_assoc, kB_ne_zero]

```

1.137 PhysLean/Thermodynamics/Temperature/TemperatureUnits.lean

```

lemma val_ne_zero (x : TemperatureUnit) : x.val  $\neq$  0 := by
lemma div_ne_zero (x y : TemperatureUnit) :  $\neg$  x / y = (0 :  $\mathbb{R}_{\geq 0}$ ) := by
  · exact Ne.symm (div_ne_zero x y)
  simp [div_eq_val, x.val_ne_zero]

```

1.138 PhysLean/Units/Basic.lean

```
lemma dimScale_ne_zero (u1 u2 : UnitChoices) (d : Dimension) :
  · exact Ne.symm (dimScale_ne_zero u1 u2 d)
```

1.139 PhysLean/Units/Examples.lean

```
mul_eq_zero, NNReal.coe_eq_zero, dimScale_ne_zero, or_self, or_false, eq_iff_iff
```

1.140 README.md

```
[](https://github.com/leanprover/)
```

1.141 docs/MaintainerTeam.md

The Maintainer Team

The default policies around the maintainer team are those of Mathlib, and where conflict or disagreement arises these are the fall-back.

Role

The role of the maintainer team is to:

1. Give final reviews of pull-requests.
2. Help structure and guide the project.
3. Help foster a pleasant and active community around the project.

Becoming a maintainer

Maintainers have research-level knowledge of physics or mathematics, and have experience with either the PhysLean or Mathlib review process.

To become a maintainer, you either need an invitation from current maintainers, or need to nominate yourself to a current maintainer.

Term of a maintainer

A maintainer will serve for a period for one-year, which will renew unless they are inactive for a period of 6 months before renewal or wish to step down (which they can do at any point).

1.142 docs/_data/perturbationTheory.yml

```
rw [timeOrder_superCommute_anPart_ofFieldOp_ne_time]
```

1.143 lake-manifest.json

```
"rev": "a41d5ebefbfa77afe737fec8de8ad03fc8b08fdff",
"inputRev": "v4.28.0",
"rev": "8f9d9cff6bd728b17a24e163c9402775d9e6a365",
"inputRev": "v4.28.0",
"rev": "4f10f47646cb7d5748d6f423f4a07f98f7bbcc9e",
"rev": "ff04f5c424e50e23476d3539c7c0cc4956e971ad",
"rev": "058ada3acad7dd0d55657476bf292c8e02a2f650",
"rev": "7e097e9a076d5fbe48aa39aceee871af0d011101",
"rev": "55c8532eb21ec9f6d565d51d96b8ca50bd1fbef3",
"rev": "c5d5b8fe6e5158def25cd28eb94e4141ad97c843",
"rev": "85b59af46828c029a9168f2f9c35119bd0721e6e",
"rev": "be3b2e63b1bbf496c478cef98b86972a37c1417d",
"inputRev": "v0.0.87",
"rev": "f642a64c76df8ba9cb53dba3b919425a0c2aeaf1",
"rev": "b8f98e9087e02c8553945a2c5abf07cec8e798c3",
"rev": "495c008c3e3f4fb4256ff5582ddb3abf3198026f",
```

1.144 lakefile.toml

```
rev = "v4.28.0"
rev = "v4.28.0"
```

1.145 lean-toolchain

```
leanprover/lean4:v4.28.0
```